


UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

Corso di Laurea Triennale in Fisica

# Restricted Boltzmann Machines For Fermions in a Three Dimensional Box

**Relatore:**
Carlo Barbieri

**Tesi di Laurea di:**
Gabriele Borroni
Matricola 14768A

Anno accademico 2024/2025

# Contents


**1 Introduction** **1**

**2 Theoretical background** **5**
2.1 Classical mechanics . . . . . 5
2.1.1 Principles . . . . . 5
2.1.2 Analytical mechanics . . . . . 6
2.2 Quantum mechanics . . . . . 7
2.2.1 Postulates of quantum mechanics . . . . . 8
2.3 many–body quantum systems . . . . . 9
2.3.1 The hydrogen atom . . . . . 10
2.3.2 The helium atom . . . . . 11
2.3.3 Second quantization . . . . . 12
2.4 Nuclear physics . . . . . 15
2.4.1 Fundamental Interactions . . . . . 15
2.4.2 Nuclear potentials . . . . . 16
2.4.3 The Argonne potential . . . . . 17
2.5 Statistical Physics . . . . . 19
2.5.1 The Ising model . . . . . 20

**3 Numerical Approaches** **21**
3.1 Monte Carlo Methods . . . . . 21
3.1.1 The Metropolis-Hastings algorithm . . . . . 22
3.1.2 Variational Monte Carlo . . . . . 24
3.1.3 Stochastic gradient descent . . . . . 26

**4 Machine learning** **29**
4.1 Artificial neural networks . . . . . 30
4.1.1 Perceptrons . . . . . 31
4.1.2 Training neural networks . . . . . 31

**5 Restricted Boltzmann Machines** **35**
5.1 RBMs as universal approximators . . . . . 36
5.2 Wavefunction representation with RBMs . . . . . 37

**6 Neural Network quantum states** **41**
6.1 State of the art . . . 41
6.2 Many electron atoms wavefunction . . . 42
6.2.1 Kinetic energy . . . 43
6.2.2 Coulomb interaction . . . 44
6.3 Deuteron wavefunction . . . 44
6.3.1 ArgonneV6 potential . . . 46
6.4 Complete algorithm . . . 50

**7 Results** **51**
7.1 Atoms . . . 51
7.1.1 learning rate study . . . 51
7.1.2 Spatial distribution plots . . . 57
7.1.3 Helium with antiparallel spins . . . 63
7.1.4 Lithium . . . 69
7.2 Deuteron . . . 76

**8 Conclusions and outlook** **81**

**A Code** **83**
A.1 Readoptions.cc . . . 83
A.2 optH2.cc . . . 84
A.3 nqs.cc . . . 85
A.4 Sampler.cc . . . 86
A.5 SamplerDeut.cc . . . 87
A.6 Coulomb3d.cc . . . 88
A.7 ArgonneV6.cc . . . 89
A.8 Variational.cc . . . 90
A.9 SGD.cc . . . 91

# List of Figures


4.1 A chart of the most common neural network architectures. . . . . 30
4.2 A chart of the most common activation functions. . . . . 32

7.1 Heatmap energy/error for $n_h = 5$ with $p = 0.5$ and $p = 1$ . . . . . 52
7.2 Heatmap energy/error for $n_h = 5$ with $p = 2$ . . . . . 53
7.3 Heatmap energy/error for $n_h = 7$ with different learning rate decay power laws $p$ 54
7.4 Heatmap energy/error for $n_h = 3$ with different learning rate decay power laws $p$ 55
7.6 Energy and error during training (first 4000 iterations) for Helium atom with both spins up and $n_h = 7$ . . . . . 57
7.7 $\rho(r)$ for Helium atom with both spins up . . . . . 59
7.8 $\rho(r)$ for Helium atom with both spins up with $n_h = 30$ . . . . . 60
7.9 $P(r)$ for Helium atom with both spins up . . . . . 61
7.10 $g(r)$ for Helium atom with both spins up . . . . . 62
7.11 Final energies and error for RBM with different hidden layer sizes on helium with antiparallel spins . . . . . 63
7.12 average energy and errors during training (first 4000 iterations) for RBM with $n_h = 7$ on helium with antiparallel spins . . . . . 64
7.13 $\rho(r)$ for Helium atom with antiparallel spins . . . . . 65
7.14 $P(r)$ for Helium atom with antiparallel spins. . . . . 66
7.15 $g(r)$ for Helium atom with antiparallel spins. . . . . 67
7.16 Motion of a spin up electron on the equatorial $xy$ plane with a spin down one on the nucleus. Here $x_2 = y_2 = z_2 = z_1 = 5$. . . . . 68
7.17 heatmap of the section on $xy$ equatorial plane. . . . . 68
7.18 Motion of a spin up electron on the equatorial $xy$ plane with a spin up one on the nucleus. Here $x_2 = y_2 = z_2 = z_1 = 5$. . . . . 68
7.19 heatmap of the section on $xy$ equatorial plane. . . . . 68
7.20 Motion of one spin up and one spin down electrons on a vertical line passing through the nucleus. Here $x_2 = y_2 = x_1 = y_1 = 5$ . . . . . 69
7.21 heatmap of the section on a vertical line passing through the nucleus for antiparallel spins. . . . . 69
7.22 Motion of two spin up electrons on a vertical line passing through the nucleus. Here $x_2 = y_2 = x_1 = y_1 = 5$. . . . . 69
7.23 heatmap of the section on a vertical line passing through the nucleus for parallel spins. . . . . 69

7.24 last energy values with error bars for RBM with different hidden layer size trained on lithium atom. . . . . 70
7.25 Energy and error during training (first 4000 iterations) for lithium with $n_h = 10$. 70
7.26 Redaelli's study [1] on different ground state energy eigenvalue, compared with the RBM spatial parameters. . . . . 71
7.27 $\rho(r)$ for lithium with two spin up electrons and a spin down one. . . . . 72
7.28 $P(r)$ for lithium with two spin up electrons and a spin down one. . . . . 73
7.29 $g(r)$ for lithium with two spin up electrons and a spin down one, plotted for parallel and antiparallel pairs. . . . . 74
7.30 Motion of a spin up electron on the equatorial $xy$ plane. Here $x_2 = y_2 = z_2 = x_3 = y_3 = z_3 = 5$ (two electrons on the nucleus) and $z_1 = 5$. . . . . 75
7.31 heatmap of the section on $xy$ equatorial plane. . . . . 75
7.32 Motion of two spin up electrons on a vertical line passing through the nucleus. Here $x_1 = x_2 = x_3 = y_1 = y_2 = y_3 = z_3 = 5$. . . . . 75
7.33 heatmap of the section on a vertical line passing through the nucleus. . . . . 75
7.34 Motion of one spin up and one spin down electrons on a vertical line passing through the nucleus. Here $x_1 = x_2 = x_3 = y_1 = y_2 = y_3 = z_3 = 5$. . . . . 76
7.35 heatmap of the section on a vertical line passing through the nucleus. . . . . 76
7.36 Energy during training for deuteron with ArgonneV6 potential. . . . . 76
7.37 Energy during training for deuteron with ArgonneV6 potential in the energy drop region. . . . . 77
7.38 $\rho(r)$ for deuteron in different stages of training. . . . . 77
7.39 $\rho(r)$ for the deuteron, separated by particle type, with logarithmic scale on both axis and an approximate exponential fit. Weights taken at $n_{\text{iter}} = 6000$. 78
7.40 $P(r)$ for deuteron in different stages of training. . . . . 78
7.41 $g(r)$ for deuteron in different stages of training. . . . . 79

# Abstract

Complex systems simulations in physics are undergoing a major transformation thanks to the employment of machine learning methods. Building on previous work, this thesis continues the study of fermionic systems in a three dimensional lattice box using neural network based Monte Carlo techniques. Our approach adopts a restricted Boltzmann machine (RBM) as the variational ansatz within the framework of neural network quantum states (NQS). The implemented code is validated by computing ground state energies and spatial distribution plots of many–body systems of interest. The first part focuses on the application of such framework to many electron atoms such as helium and lithium. Meanwhile, The second part focuses on the extension to nuclear systems, in particular the deuteron, by implementing the Argonne V6 nucleon–nucleon potential and a sampling procedure that exploits symmetries in the deuteron. This study highlights the promising capabilities of RBM-based NQS models in computational physics and quantum chemistry, where the most promising feature of this neural network architecture is that the number of parameters is largely independent from the number of bodies involved, which opens the way to hypothesize transfer learning capability across related quantum many–body problems.

# Chapter 1

# Introduction

Machine learning and artificial intelligence have undergone a drastic development during the recent years. These technologies marked a shift of perspective across various scientific fields as well as in our daily life. In particular, modern learning algorithms can be used not only to improve existing numerical methods, for example by accelerating expensive calculations or denoising experimental data, but also to tackle intrinsically hard computational tasks, such as high dimensional optimization and sampling problems, where traditional approaches quickly become inefficient.

Physics is a natural playground for these ideas. Many physical systems are described by a large number of degrees of freedom and by highly non trivial correlations, which make their numerical treatment extremely challenging. As a consequence, machine learning has been successfully applied across several branches of physics, from particle and nuclear physics to condensed matter and astrophysics, and it had a severe impact on the development of our understanding of the world. The scientific community acknowledged this step forward, as evidenced by the Nobel prize for physics assigned to Geoffrey Hinton and John J. Hopfield [2] for their pioneering work on neural networks.

More recently, machine learning has started to be used at a deeper level, by embedding it directly into first principles numerical methods. In quantum many–body physics, this idea is particularly appealing: the Schrödinger equation can be reformulated as a variational optimization problem, in which one searches for the ground state by minimizing the energy over a suitable family of wavefunctions. Replacing rigid ansätze (such as Slater forms) with flexible neural network representations offers the possibility of capturing strong correlations with a compact set of parameters, allowing to handle the exponentially large Hilbert space into a learnable model.

A major step in this direction was made in 2017 by Carleo and Troyer [3], who combined Monte Carlo methods and machine learning to introduce the so called *Neural Network Quantum States* (NQS). In this framework, the many–body wavefunction is represented by a neural network, and its parameters are optimized via variational Monte Carlo (VMC) using samples drawn from the probability distribution defined by the network itself. The key conceptual shift is that the system is not described by explicitly constructing a basis or an expansion, but by defining a trainable model, which enables for a more expressive and unbiased ansätze. This pioneering work is having a great impact on the scientific community, and has already lead to promising results:

- In *Fermionic neural network states for ab initio electronic structure* [4], NQS are extended to interacting electronic systems in second quantization. The fermionic degrees of freedom are mapped to spin variables, and different neural-network ansätze are benchmarked on quantum chemistry problems.

- In *Dilute neutron star matter from neural network quantum states* [5], low density neutron matter is modeled with hidden-nucleon NQS combined with variational Monte Carlo and stochastic optimization, showing that neural networks can also be used in nuclear many–body problems.

- In *Solving quasiparticle band spectra of real solids using neural network quantum states* [6], the authors demonstrate that NQS can achieve chemical accuracy for the ground state energies of periodic solids in one, two and three dimensions, and can be used to extract quasiparticle band structures.

- In *Variational Monte Carlo Calculations of $A \leq 4$ Nuclei with an Artificial Neural Network Correlator Ansatz* [7], binding energies and point nucleon densities of light nuclei are computed using an ANN correlator ansatz and compared with more traditional Jastrow type parametrizations.

In parallel, several deep learning architectures have been proposed to represent fermionic wavefunctions directly in real space. Among the most prominent examples, PauliNet [8] combines a Slater–Jastrow ansatz with a deep neural network, FermiNet [9] uses permutation–equivariant deep network to construct sets of orbitals whose determinants are summed to form the full antisymmetric state. More recent developments include hybrid architectures that merge ideas from graph neural networks and FermiNet–like structures, and Transformer–based ansätze such as the Psiformer, which further improve accuracy on molecular benchmarks. A comprehensive overview of these methods and of their application to ab initio quantum chemistry can be found in the review *Ab initio quantum chemistry with neural network wavefunctions* [10].

This thesis focuses on the implementation of NQS to describe **wavefunctions** of fermionic systems such as many electron atoms and simple nuclei within a unified framework.

Although there are several architectures to be chosen to tackle this problem, this work focuses on an implementation based on **restricted Boltzmann machines** (RBMs). Although their implementation and training can be more challenging, these types of architectures show the promising feature of having a fixed number of parameters, largely independent from the number of bodies involved in the problem. This opens the way to the **transfer learning hypothesis**, that could be an extremely powerful feature for going in many–body quantum systems simulations. Moreover, RBMs naturally operate on discretized spatial grids and are therefore well suited to Hamiltonians expressed in position space with predominantly local operators. This work is built on three previous theses [11, 12, 1]. Starting from the most recent implementation, we first resolved the technical issues that previously prevented the use of hidden layers, enabling us to obtain detailed results for helium and lithium. In addition, the code has been extended to incorporate the Argonne V6 potential, an approximate but realistic representation of the nucleon–nucleon interaction derived from the Argonne V18 model [13].

This thesis covers all the concepts needed for such an implementation. Chapter 2 provides a summary of the theoretical background of the work, including quantum, nuclear and statistical physics; chapter 3 presents an overview on Monte Carlo approaches and stochastic variational optimization; chapter 4 introduces machine learning basic concepts; chapter 5 presents the theoretical details of RBMs, including universal approximation theorems, and then highlights how these architectures can be used to represent wavefunctions. Chapter 6 gives an insight of the implementation of RBMs for fermionic systems within the NQS framework. Finally, chapter 7 presents the results, including analysis on learning rate and hidden layers size along with spatial distribution plots and wavefunction sections. Here both lithium and helium are presented as atomic systems, while for nuclear ones we present energy and spatial plots of interest for deuteron. Chapter 8 contains the conclusions drawn from this work, and an outlook on possible future applications, along with a technical explanation of each component of the code in the appendix.

# Chapter 2

# Theoretical background

## 2.1 Classical mechanics

### 2.1.1 Principles

Classical mechanics studies the movement of bodies through the action of **forces** applied on them. It is the foundation of all physics, originally developed by Isaac Newton.
Classical mechanics relies on three principles, known as the three Newton's laws of dynamics:

- *First law*: an object remains at rest or in uniform motion on a straight line, unless it is acted upon by an external force.
- *Second law*: the acceleration of an object is directly proportional to the net force acting on it and inversely proportional to its mass:

$$\vec{F} = m\vec{a}$$

- *Third law*: for every force $\vec{F}$ exerted by object A on object B, object B exerts an equal and opposite force $-\vec{F}$ on object A.

In the classical framework, a material point is described by a position vector in the **configuration space** $\mathbb{R}^3$. A force acting on the particle is described as a vector field on the configurations space:

$$\vec{F} : \mathbb{R}^3 \times \mathbb{R} \to \mathbb{R}^3$$

This vector field associates to each configuration a force vector, generally dependent on 3 spatial coordinates and time.

To describe the motion of a classical system, **dynamic variables** are introduced. A dynamic variable is any physical quantity that describes the state of a system and evolves in time according to the laws of motion. The fundamental dynamical variables in classical mechanics are position $\vec{x}(t)$ and velocity $\vec{v}(t) = \frac{d\vec{x}}{dt} = \dot{\vec{x}}(t)$.

Newton's second law provides, in principle, a way to determine the evolution of each dynamical variable, and thus a description of the motion of any object, by solving the equation:

$$\vec{F} = m\ddot{\vec{x}}(t)$$

Another pivotal role in Newtonian mechanics is played by **energy**. Energy can be defined as the ability to perform work. The work of a force along a path $\gamma$ is defined as:

$$W = \int_\gamma \vec{F}(\vec{r}, t) \cdot d\vec{s}$$

The main types of energy include the following:

- *Kinetic energy*: for each particle of mass $m$ moving with velocity $v$, the kinetic energy is defined as:
$$T = \frac{1}{2}mv^2$$
- *Potential energy*: for **conservative forces**, which are associated with conservative fields, there exists a scalar potential function, depending on the position position of the partice $\vec{r}$, such that:
$$\vec{F} = -\nabla V(\vec{r})$$

An important result in classical mechanics is that the **total mechanical energy** $E = T + V$ is conserved during the time evolution of the system.

### 2.1.2 Analytical mechanics

#### Lagrangian formalism

Analytical mechanics is the bridge between classical and modern physics. Lagrange's intuition was to rewrite classical mechanics in a general way, introducing a more elegant and efficient framework relying no longer on vectors like forces, which become increasingly difficult to describe as systems become more complex, but on scalar and more general quantities such as energy.

The analytical mechanics framework revolves around the **Lagrangian function**, defined as the difference between the kinetic and potential energy of the system. For conservative forces:

$$\mathcal{L}(q, \dot{q}, t) = T(q, \dot{q}) - V(q)$$

Here, $q = q(t)$ denotes a generalized coordinate and $\dot{q}(t)$ the generalized velocity.

by projecting Newton's second law of motion onto each generalized coordinate, it is possible to obtain the **Euler-Lagrange equations**, which govern the dynamics of the system. For a system composed of N particles:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = 0 \qquad i = 1, \dots, N$$

The solutions of these equations coincide with the laws of motion of the system, namely, it is possible to derive $q_i(t), \dot{q}_i(t)\ \forall i = 1, \dots, N, \forall t \in \mathbb{R}^+$.

The main novelty is that this formalism uses a different underlying space: for a single unconstrained particle, Newton's equations are second order ODEs on the configuration space $Q = \mathbb{R}^3$, while Euler-Lagrange dynamics live in the **state space**, the tangent bundle $TQ \cong \mathbb{R}^3 \times \mathbb{R}^3$, which is the set of pairs $(q, \dot{q})$ of generalized coordinates and velocities, so that the Lagrangian is a function:

$$\mathcal{L} : TQ \to \mathbb{R}$$

### Hamiltonian formalism

A further extension of Lagrangian dynamics, more compact and general, is the Hamiltonian formalism.

This approach relies on the **phase space**, the set of pairs $(q, p)$ of generalized coordinates and conjugate momenta:

$$p = \frac{\partial \mathcal{L}}{\partial \dot{q}}$$

By applying the **Legendre transform** on the Lagrangian, that enables to express a function in terms of its own derivatives, one obtains the **Hamiltonian function**:

$$H(q_i, p_i, t) = \sum_i p_i \dot{q}_i - \mathcal{L}(q_i, \dot{q}_i, t)$$

For natural Lagrangian systems, where $\mathcal{L} = T - V$ and $\frac{\partial \mathcal{L}}{\partial t} = 0$, the Hamiltonian coincides with the total energy of the system, $H = T + V$, thus it is a constant of motion. Moreover, it is possible to show that the Euler-Lagrange equations are equivalent to the Hamilton equations:

$$\dot{q}^i = \frac{\partial H}{\partial p_i}, \qquad \dot{p}_i = -\frac{\partial H}{\partial q^i}, \qquad i = 1, \ldots, n.$$

In this framework it is sufficient to solve 2N first order ODEs to obtain the laws of motion for a system of N particles.

This abstract, energy based formalism is the foundation of modern physics, since it can be efficiently used to study complex systems, such as quantum or many–body ones, without solving second order vector differential equations. Moreover, this formalism has the powerful feature of highlighting and exploiting symmetries through the Noether theorem. Finally, the principle of least action leads to a formulation of the dynamics of every classical system, making the analytical mechanics the most flexible and general framework in physics.

## 2.2 Quantum mechanics

Classical mechanics successfully describes the motion of macroscopic systems, but fails to describe the behavior of nature at atomic or subatomic scales. Phenomena such as the black-body radiation, the photoelectric effect and atomic spectra revealed the intrinsic limits of classical theory, showing that physical observables are not continuous but quantized. Quantum physics emerged as a new and more general framework for describing nature, in which classical theories emerge as the macroscopic limit of quantum laws. This new language overturned the certainties of classical determinism, replacing exact predictions with probability amplitudes, and marking a new Copernician revolution across scientific disciplines. Because it is conceptually complex and counterintuitive, quantum physics faced many skeptics in its early development. Yet, it is proven to be the most accurate and successful theory in physics, as an overwhelming number of experiments confirm the extraordinary agreement between its predictions and the experimental results.

### 2.2.1 Postulates of quantum mechanics

Quantum mechanics is based upon 4 fundamental postulates:

- *Postulate 1*: The state of an isolated physical system is fully described by a vector $|\psi\rangle$ belonging to a Hilbert space $\mathcal{H}$, a complex vector space equipped with an inner product.
- *Postulate 2*: every physical observable is represented by a Hermitian operator $\hat{A}$ acting on $\mathcal{H}$. The possible outcomes of the measurements of $\hat{A}$ correspond to the spectrum of such operator. An equivalent formulation of this postulate is that the possible results of a measurement are associated with exhaustive and exclusive states that constitute an orthonormal basis for $\mathcal{H}$.
- *Postulate 3*: when an observable $\hat{A}$ is measured in a system in the state $|\psi\rangle$, the probability of obtaining the eigenvalue $a_n$ is:

  $$P_{a_n} = |\langle a_n|\psi\rangle|^2$$

  After the measurement, the state of the system collapses into the corresponding eigenstate $|a_n\rangle$.
- *Postulate 4*: the time evolution of an isolated system is generated by the action of a hermitian operator $\hat{H}(t)$. That is, the time evolution operator is unitary and has the infinitesimal form:

  $$\hat{S}(t+\epsilon, t) = \mathbb{I} - \frac{i\epsilon}{\hbar}\hat{H}(t)$$

The time evolution of a quantum system can be deduced by solving the **Schrödinger equation**:

$$i\hbar\frac{d}{dt}|\psi(t)\rangle = \hat{H}(t)|\psi(t)\rangle$$

where $\hat{H}(t) = T+V$ is the Hamiltonian of the system, $i$ the imaginary unit, and $\hbar$ the reduced Planck constant. To solve the equation, one needs to project the state and the operators on a convenient basis. Taking, for instance, the position basis (which will be assumed from now on)$|x\rangle$:

$$i\hbar\frac{\partial}{\partial t}\langle x|\psi(t)\rangle = \langle x|\hat{H}(t)|\psi(t)\rangle$$

defining $\psi(x,t) := \langle x|\psi(t)\rangle$ the **wavefunction** of the system, and substituting $\hat{H} = \hat{T} + \hat{V}$, where

$$\hat{T} = \frac{\hat{p}^2}{2m} = -\frac{\hbar^2}{2m}\frac{\partial^2}{\partial x^2}$$

leads to:

$$i\hbar\frac{\partial\psi(x,t)}{\partial t} = -\frac{\hbar^2}{2m}\frac{\partial^2\psi(x,t)}{\partial x^2} + V(x,t)\psi(x,t)$$

The solution to this equation provides an expression of the wavefunction at any time $t$. This second order PDE can be extremely difficult to solve, although it can be heavily

simplified for some specific classes of systems. When the Hamiltonian is time independent, the solutions are called **stationary states**, because the spatial part of the wavefunction can be separated from the temporal part, so that the probability density $|\psi(x)|^2$ is time independent:

$$\psi(x,t) = \sum_n \phi_n(x) e^{-\frac{i}{\hbar}E_n t}$$

where $E_n$ are the energy eigenvalues, and $\phi_n(x)$ are the energy eigenfunctions, obtained by solving the **time independent Schrödinger equation**:

$$\hat{H}\phi_n(x) = E_n\phi_n(x)$$

Thus, finding the energy spectrum of a system is often sufficient to determine its dynamical nature. Depending on the boundary conditions of the system, the energy spectrum can be:

- Discrete: for bound systems, such as harmonic oscillators, only specific (negative) energy eigenvalues are allowed.
- Continuous: in unbound systems, such as free non-interacting particles, the energy can take any positive value.
- Mixed: systems that can have bound and unbound states, such as atoms.

## 2.3 many–body quantum systems

The state vector of a composite quantum system belongs to the **composite Hilbert space**, obtained as the tensor product of the individual Hilbert spaces of its subsystems:

$$|\Psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_N\rangle \in \mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \cdots \otimes \mathcal{H}_N.$$

In general, the wavefunction of an $N$-particle system cannot be written as the product of $N$ single-particle wavefunctions. When such a factorization is possible, the system is said to be **separable**, meaning that its subsystems evolve independently.

For a system described by a Hamiltonian of the form

$$\hat{H} = \sum_{i=1}^{N} \hat{H}_i,$$

the dynamics is separable if the operators $\hat{H}_i$ act on distinct subspaces and mutually commute:

$$[\hat{H}_i, \hat{H}_j] = 0 \quad \forall\, i \neq j.$$

In this case, the total wavefunction can be expressed as the product of individual ones:

$$\Psi(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \psi_1(\mathbf{x}_1)\, \psi_2(\mathbf{x}_2)\, \ldots\, \psi_N(\mathbf{x}_N).$$

### 2.3.1 The hydrogen atom

The hydrogen atom is one of the most important results in quantum mechanics, yet the simplest real world quantum mechanical problem. This is a two body problem subjected to the **Coulomb potential**:

$$V(r) = -\frac{Zq_e^2}{4\pi\epsilon_0 r}$$

where $q_e$ is the elementary charge, $Z$ the number of protons in the nucleus, $\epsilon_0$ the electrical permittivity of the vacuum.

The complete Hamiltonian of the system reads:

$$\hat{H} = -\frac{\hbar^2}{2m_e}\nabla_1^2 - \frac{\hbar^2}{2m_p}\nabla_2^2 - \frac{Ze^2}{4\pi\varepsilon_0|\mathbf{r}_1 - \mathbf{r}_2|}.$$

where $m_e$ and $m_p$ indicate the masses of electron and proton, respectively, and $-\hbar^2\nabla_i^2$ is the 3D expression of the squared momentum operator. This problem meets the separability conditions, so it can be effectively separated into a **radial part** $H_r$ and a **barycentric part** $H_B$ in the following way:

$$\hat{H} = -\frac{\hbar^2}{2\mu}\nabla^2 - \frac{Ze^2}{r},$$

where $\mu = \frac{m_e m_p}{m_e + m_p}$ is the reduced mass r the Euclidean distance between the particles and $e = \frac{q_e^2}{4\pi\epsilon_0}$.

The time independent Schrödinger equation for the system is separated (in spherical coordinates) in the same way as the Hamiltonian and reads:

$$\left(-\frac{\hbar^2}{2\mu}\frac{d^2}{dr^2} + \frac{\hbar^2\ell(\ell+1)}{2\mu r^2} - \frac{Ze^2}{r}\right)u(r) = E\,u(r).$$

where $\ell$ is the quantum number associated with the **angular momentum operator** eigenvalue:

$$\hat{L}^2|\ell\rangle = \hbar^2\ell(\ell+1)|\ell\rangle$$

and $u(r)$ is the radial wavefunction. The solution to this equation reads:

$$\psi(r,\theta,\phi) = \phi(r)Y_{l,m}(\theta,\phi)$$

where $\phi(r) = \frac{u(r)}{r}$ is the radial part of the wavefunction and the angular part is described with the **spherical harmonics** $Y_{l,m}(\theta,\phi)$, a complete orthonormal basis on the sphere that entirely describes the angular motion of the system. From the stationary equation it is also possible to derive the **energy spectrum** for the hydrogen-like atom:

$$E_n = -\frac{\mu Z^2 e^4}{2\hbar^2 n^2}$$

where $n \geq 1$ is called the **principal quantum number**, since it structures the energy levels. For each energy state, it is easily provable that there is a degeneracy- number of states with different quantum numbers $n, \ell$ that share the same energy eigenvalue- of $n^2$.

### 2.3.2 The helium atom

The Helium atom has two electrons, which means that Coulomb repulsion needs to be taken account of. The Hamiltonian of the system, expressed in terms of electrons' coordinates, is:

$$\hat{H} = -\frac{\hbar^2}{2m_e}\left(\nabla_1^2 + \nabla_2^2\right) - \frac{2e^2}{4\pi\varepsilon_0}\left(\frac{1}{r_1} + \frac{1}{r_2}\right) + \frac{e^2}{4\pi\varepsilon_0|\mathbf{r}_1 - \mathbf{r}_2|}.$$

The last term couples the electrons coordinates, making the problem impossible to be separated. For this reason, Helium atom, like all the other ones except for hydrogen, hasn't an analytical solution, and has to be tackled using approximated methods.

Moreover, other quantum effects arise when studying atoms with more than one electron, such as **spin**, a quantum degree of freedom without classical analogue. Spin can be seen as an intrinsic angular momentum of the particle, independent of any orbital motion. Spin operators satisfy the same commutation relations as angular momentum operators but act in a finite-dimensional Hilbert space. For the electron, this space has dimension 2, so that the third component of spin can take two possible values. Since the degeneracy associated with a spin quantum number $s$ is $2s+1$, electrons are **spin-$\frac{1}{2}$ particles**. More generally, particles with half-integer spin are called **fermions**, while those with integer spin are called **bosons**.

For **spin** $\frac{1}{2}$ systems, the spin state is described as a rank two complex vector called **spinor**. A basis on this vector space can be constructed by defining the "up" state ($s = +\frac{1}{2}$) and the "down" one ($s = -\frac{1}{2}$) as:

$$|\uparrow\rangle = \begin{pmatrix}1\\0\end{pmatrix} \quad |\downarrow\rangle = \begin{pmatrix}0\\1\end{pmatrix}$$

so that these are eigenstate of the third component projection of spin:

$$\hat{S}_z|\uparrow\rangle = +\frac{\hbar}{2}|\uparrow\rangle \quad \hat{S}_z|\downarrow\rangle = -\frac{\hbar}{2}|\downarrow\rangle$$

and the total spin is given by:

$$\hat{S}^2|\uparrow/\downarrow\rangle = \frac{3}{4}\hbar^2|\uparrow/\downarrow\rangle$$

In this scenario, spin operators can be expressed as $2\times 2$ Hermitian matrices, called the **Pauli matrices**:

$$\sigma_0 = \mathbb{I} \qquad \hat{S}_x = \frac{\hbar}{2}\sigma_x = \frac{\hbar}{2}\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}, \qquad \hat{S}_y = \frac{\hbar}{2}\sigma_y = \frac{\hbar}{2}\begin{pmatrix}0 & -i\\ i & 0\end{pmatrix}, \qquad \hat{S}_z = \frac{\hbar}{2}\sigma_z = \frac{\hbar}{2}\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}. \tag{2.1}$$

So that the Spin operator is defined as:

$$\vec{S} = \frac{\hbar}{2}\vec{\sigma}$$

In this formalism, spin operators are related to rotations in a two dimensional hilbert space, since $\vec{S} \in SU(2)$, and generate rotations of spinors through unitary evolutions.

When taking into account quantum indistinguishability, it can be shown that the wavefunction of a system composed of $N$ fermions must be antisymmetric under the exchange of two particles:

$$\Psi(\dots, \mathbf{r}_i, s_i, \dots, \mathbf{r}_j, s_j, \dots) = -\Psi(\dots, \mathbf{r}_j, s_j, \dots, \mathbf{r}_i, s_i, \dots),$$

whereas for $N$ bosons it must be symmetric under exchange.

In the general case, an $N$–fermion wavefunction is built by antisymmetrizing a product of $N$ one particle orbitals. Formally, if $\phi_k(x)$ denotes the $k$–th one particle state, the general fermionic many–body state reads

$$\Psi(x_1, \dots, x_N) = \frac{1}{\sqrt{N!}} \sum_{\pi \in S_N} \operatorname{sgn}(\pi)\, \phi_1(x_{\pi(1)}) \phi_2(x_{\pi(2)}) \cdots \phi_N(x_{\pi(N)}),$$

where the sum runs over all permutations of $N$ particles, and $\operatorname{sgn}(\pi)$ denotes the sign of the permutation.

This antisymmetrized sum is compactly written as the **Slater determinant**

$$\Psi(x_1, \dots, x_N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_N(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \cdots & \phi_N(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_N) & \phi_2(x_N) & \cdots & \phi_N(x_N) \end{vmatrix}.$$

If two fermions share the same one particle state, e.g. $\phi_i = \phi_j$ for some $i \neq j$, then two columns of this determinant coincide, and

$$\Psi(x_1, \dots, x_N) = 0.$$

Thus a fermionic many–body state becomes the null vector, which has no physical meaning, if two quantum numbers coincide. This result is known as the **Pauli exclusion principle**, which states that two fermions cannot occupy the same quantum state simultaneously.

This principle adds further complexity to the study of many electron atoms, and it has to be accounted in numerical methods.

### 2.3.3 Second quantization

The second quantization formalism is an efficient reformulation of quantum mechanics for many–body systems of indistinguishable particles, particularly efficient when dealing with many electron atoms or many nucleon nuclei. This framework marks a shift of perspective: instead of describing the state of the full quantum system in terms of physical coordinates, it characterizes it by counting how many particles occupy every single particle state, a quantity known as the **occupation number**. In this section, a brief introduction to the key concept is given, while a more in-depth analysis of Fock space fundamentals can be found in [14]

The state of the system in this formalism lives in the **Fock space**, which is built as the direct sum of all $n$-particle Hilbert spaces (symmetrized for bosons and antisymmetrized for fermions):

$$\mathcal{F} = \bigoplus_{n=0}^{\infty} \mathcal{H}^{(n)}.$$

Here the $n$-particle space is defined as

$$\mathcal{H}^{(n)} = \begin{cases} \mathrm{Sym}(\mathcal{H}^{\otimes n})\,, & \text{bosons,} \\ \mathrm{Alt}(\mathcal{H}^{\otimes n})\,, & \text{fermions,} \end{cases}$$

where $\mathcal{H}$ is the single particle Hilbert space and Sym/Alt denote the (anti)symmetrization projectors implementing indistinguishability. A generic ket $|\phi\rangle \in \mathcal{F}$ can be represented as:

$$|\phi\rangle = |n_1 \dots n_N\rangle$$

where $n_i$ represents the number of bodies occupying the single particle orbital $i$. It follows immediately that for fermions $n = 0, 1$, and so the Pauli principle is automatically satisfied. To operate on states defined like so, **creation and annihilation operators** are needed. The creation operator $a_i^\dagger$ acts on a state by adding one particle in the $i$-th orbital:

$$a_i^\dagger |n_1 \dots n_i \dots n_N\rangle = \sqrt{n_i + 1}\,|n_1 \dots n_i + 1 \dots n_N\rangle$$

Similarly, the annihilation operator $a_i$ acts on the state by removing a particle in the $i$-th orbital:

$$a_i |n_1 \dots n_i \dots n_N\rangle = \sqrt{n_i}\,|n_1 \dots n_i - 1 \dots n_N\rangle$$

It is important to note that the action of the annihilation operator on the **vacuum** (zero particle state) simply vanishes:

$$a_i |0\rangle = 0$$

While a general Fock space state can be constructed by repeatedly acting with those operators on the vacuum state:

$$|n_1, n_2, \dots, n_\alpha, \dots\rangle = \frac{1}{\sqrt{n_1!\, n_2! \cdots n_\alpha! \cdots}}\, (c_1^\dagger)^{n_1} (c_2^\dagger)^{n_2} \cdots (c_\alpha^\dagger)^{n_\alpha} \cdots |0\rangle.$$

For bosons, the order of these operations does not affect the final results, meanwhile for fermions exchanging two of these actions will result in a phase shift, due to symmetrization. When dealing with coordinates, it is possible to project the creation and annihilation operators on the coordinate basis. The operators $\psi(\vec{r})$ and $\psi^\dagger(\vec{r})$ annihilate and create a particle at position $\vec{r}$. Given a complete orthonormal set of single particle wavefunctions $\{u_i(\vec{r})\}$, the creation operator in position space can be expanded as

$$\psi^\dagger(\vec{r}) = \sum_i c_i^\dagger\, u_i^*(\vec{r}), \qquad c_i^\dagger = \int d\vec{r}\, \psi^\dagger(\vec{r})\, u_i(\vec{r}).$$

A single particle state associated with the one body wavefunction $u_i(\vec{r})$is obtained from

$$|i\rangle = c_i^\dagger |0\rangle = \int d\vec{r}\, u_i(\vec{r})\, |\vec{r}\rangle, \qquad |\vec{r}\rangle = \psi^\dagger(\vec{r})\, |0\rangle.$$

The $N$-particle position eigenstate is constructed by successive application of the creation operator in position space:

$$|\vec{r}_1, \vec{r}_2, \dots, \vec{r}_N\rangle = \frac{1}{\sqrt{N!}}\, \psi^\dagger(\vec{r}_1)\psi^\dagger(\vec{r}_2) \cdots \psi^\dagger(\vec{r}_N)\, |0\rangle.$$

The connection between occupation number states and the $N$-body wavefunction is

$$\langle \vec{r}_1, \vec{r}_2, \ldots, \vec{r}_N \mid n_1, n_2, \ldots \rangle = \Phi(\vec{r}_1, \vec{r}_2, \ldots, \vec{r}_N; \{n_i\}),$$

where $\Phi$ denotes the many–body wavefunction associated with the Fock space state specified by the set of occupation numbers $\{n_i\}$.
An analogue argument can be carried on for operators.

A one body operator of the form $O(\vec{r})$ is written as

$$O = \int d\vec{r}\, \psi^\dagger(\vec{r})\, O(\vec{r})\, \psi(\vec{r}) = \sum_{ij} o_{ij}\, c_i^\dagger c_j,$$

with matrix elements

$$o_{ij} = \int d\vec{r}\, u_i^*(\vec{r})\, O(\vec{r})\, u_j(\vec{r}).$$

A two body interaction symmetric under particle exchange can be written as

$$V = \frac{1}{4} \sum_{ijkl} v_{ijkl}\; c_i^\dagger c_j^\dagger c_l c_k,$$

where the two body matrix elements are

$$v_{ijkl} = \int d\vec{r}_1 \int d\vec{r}_2\; u_i^*(\vec{r}_1)\, u_j^*(\vec{r}_2)\, V(\vec{r}_1, \vec{r}_2) \left[ u_k(\vec{r}_1) u_l(\vec{r}_2) \pm u_l(\vec{r}_1) u_k(\vec{r}_2) \right].$$

Where the plus sign applies to bosons, and the minus sign to fermions.

### Fermionic wavefunctions on a discretized lattice

The second quantization tools presented can be used to represent the wavefunction for a fermionic system on a discretized lattice. Consider a discretized lattice with $N_v$ sites. A configuration of the system is represented by a binary vector

$$\vec{v}^T = (0, 0, 1, 0, 1, 0, \ldots), \qquad \vec{v} \in \{0, 1\}^{N_v}, \tag{2.2}$$

where $v_i$ denotes the occupation of the site $x_i$.

A many–body basis state corresponding to a configuration $\vec{v}$ is constructed as

$$|\vec{v}\rangle = \prod_{i=0}^{N_v - 1} \left[ c_{s_i}^\dagger(x_i) \right]^{v_i} |0\rangle, \tag{2.3}$$

where $c_{s_i}^\dagger(x_i)$ creates a fermion at site $x_i$ with spin $s_i$, and the exponent $v_i \in \{0, 1\}$ selects whether the corresponding operator acts or not.

## 2.4 Nuclear physics

Nuclear physics applies quantum mechanics to the study of matter at **Fermi [fm]** scales ($1fm = 10^{-15}m$), where atomic nuclei can be found. Nuclei are bound states of protons and neutrons. While the proton is a charged particle ($q_p = |q_e|$), the neutron is not. The main nuclear phenomena involve energies in the range of **mega electronvolt (MeV)**($1MeV = 10^6eV = 1.6 \times 10^{-13}J$), where an electronvolt is defined as the kinetic energy gained by an electron accelerated through a potential difference of 1 volt. Nuclear masses are often expressed in terms of energy through the Einstein relation:

$$m = \frac{E}{c^2}$$

so that, for proton and neutron:

$$m_p = 938.280\frac{MeV}{c^2} \qquad m_n = 938.573\frac{MeV}{c^2} \tag{2.4}$$

Protons and neutrons are both **fermions**, which means that they obey Pauli principle. Because of their similar properties, Werner Heisenberg (1932) proposed to treat them as two different states of a single particle species, introducing the concept of **isospin**. This is formalized by introducing a new quantum operator with the same mathematical structure as the spin one (also described with the Pauli matrices): the nucleon is assigned a total isospin $T = \frac{1}{2}$, while the proton and neutron correspond to the two possible projections of the isospin along the third axis:

$$|p\rangle = |T = \frac{1}{2}, T_z = +\frac{1}{2}\rangle \quad |n\rangle = |T = \frac{1}{2}, T_z = -\frac{1}{2}\rangle$$

This formalism allows to account more efficiently for nuclear interaction symmetries and for Pauli principle.

### 2.4.1 Fundamental Interactions

Modern nuclear and subnuclear physics revolve around the concept of the **four fundamental interactions** of nature. Each interaction between particles occurs through the exchange of spin-1 particles named **bosons**, which are also referred to as **mediators**. The four fundamental interactions include:

- *Electromagnetic interaction*: mediated by the photon, the quantum of the electromagnetic field. It acts between charged particles, being repulsive for like charges and attractive for opposite ones. Its range is infinite, and the corresponding potential decreases as $V_C(r) \propto \frac{1}{r} \quad \text{for} \quad r \to \infty$.

  The potential associated with this interaction has a closed analytical form, the Coulomb potential.

- *Weak (or electroweak) interaction*: a short-range interaction mediated by the heavy bosons $W^{\pm}$ and $Z^0$. It is responsible for subatomic processes such as quark flavour change and nuclear decays.

- *Strong interaction*: the strongest of all fundamental forces, acting at distances of the order of a few femtometers. It binds quarks inside hadrons (composite particles such nucleons) through the exchange of **gluons**. At the hadronic level, its residual effect — the **nuclear force** — binds protons and neutrons together within nuclei.

- *Gravitational interaction*: an interaction of infinite range acting on mass and energy. However, its strength at nuclear scales is negligible compared to those of the other forces.

At nuclear scale, the dominant interactions are the electromagnetic and the strong ones. While the former admits an analytical expression for its potential, the latter does not.

At the fundamental level, the strong interaction acts between quarks and gluons and is described by Quantum Chromodynamics (QCD). However, at nuclear energies the relevant force is not the fundamental QCD interaction itself, but its **residual effect** between nucleons. This residual strong interaction can be effectively modeled through massive bosons exchange, as first proposed by Hideki Yukawa (1935). Yukawa proposed that the short range of the nuclear force arises from the exchange of a massive mediator about 200 times heavier than the electron, the **pion** $\pi$, and derived an analytical form for the nuclear interaction potential, which brings his name:

$$V_Y(r) = -g^2 \frac{e^{-\frac{m_\pi c}{\hbar} r}}{r} \tag{2.5}$$

where $g$ is the pion-nucleon coupling constant, $\frac{\hbar}{m_\pi c} \approx 1.4 fm$ is the characteristic length of the interaction and $r$ the distance between nucleons. It is worth noticing that in the case of $m = 0$ (photon) and $g = \frac{1}{4\pi\epsilon_0}$ this form reproduces the Coulomb potential, confirming that the finite range of the nuclear force originates from the nonzero mass of its mediator.

This interaction also **conserves the isospin**, while it is strongly spin dependent.

### 2.4.2 Nuclear potentials

Because of the complexity of including all the interactions acting on nuclear scale, there is no closed form of a potential that fully describes the nuclear motion, so that many **phenomenological potentials** have been developed.

One of the first and simplest ones is the **Woods-Saxon** potential, which is an empirical potential proposed in the framework of the nuclear shell model, which assumes a **mean field approximation**: every nucleon interacts through a field which is given by the mean field of the other $N - 1$ nucleons.

$$V(r) = -\frac{V_0}{1 + e^{\frac{r-R}{a}}}, \tag{2.6}$$

where $V_0$ is the potential depth (typically $V_0 \approx 50$ MeV), $R = r_0 A^{1/3}$ is the nuclear radius with $r_0 \approx 1.2$ fm, and $a \approx 0.5$ fm is the diffuseness parameter, which controls how smoothly the potential vanishes at the nuclear surface.

To reproduce the observed shell structure and magic numbers, a spin–orbit term is added:

$$V_{so}(r) = \lambda \left( \frac{1}{r} \frac{dV(r)}{dr} \right) \mathbf{L} \cdot \mathbf{S},$$

This formula is effective for reproducing the behaviour of nuclear interaction considering the nucleus as a whole through the mean field approximation. Modern potentials also account for the nucleon-nucleon interaction, and there are many versions of them available in literature.

### 2.4.3 The Argonne potential

The ArgonneV18 potential, named after the Argonne research group that formulated it in 1995, is the most widely used phenomenological potential describing nucleon-nucleon interactions, including spin, isospin, orbital and electromagnetic terms.

The full NN potential is defined as a sum of an electromagnetic part (EM), a one pion-exchange part (OPE) and a short-range phenomenological one (R):

$$v_{NN} = v_{NN}^{EM} + v_{NN}^{\pi} + v_{NN}^{R}$$

Where the electromagnetic term depends mainly on electric charge, while accounting for finite size effects (including electromagnetic form factors), dipole interactions, vacuum polarizability and a Darwin–Foldy relativistic correction. This part of the potential is most effective in the *pp* canals.

The one pion exchange term describes the nuclear interaction at its long range by the exchange of single pions. The effects of this interaction include: spin-spin and tensorial interaction, isospin interaction (attractive in $T = 0$ canals and repulsive in $T = 1$ ones) and isospin simmetry effects due to difference of mass in the pions. These operators are expressed as linear combinations of central Yukawa terms $Y_\pi$ (which have the form of (2.5)) and tensorial Yukawa terms $T_\pi$ obtained with derivatives of $Y_\pi$:

$$T_\pi(r) = \left(1 + \frac{3}{m_\pi r} + \frac{3}{(m_\pi r)^2}\right)\frac{e^{-m_\pi r}}{m_\pi r} \propto \left(\frac{1}{r}\frac{d}{dr} - \frac{d^2}{dr^2}\right) Y_\pi(r).$$

Finally, the phenomenological potential includes what is left by EM and OPE interactions, namely the intermediate–short range part of the nuclear potential. This term does not derive from first principles like the other ones, but it is constructed in a flexible and analytical manner to fit experimental data such as NN scattering and deuteron properties. These terms are expressed as properly weighted combinations of terms including the square of the tensorial OPE functions, which models the interactions due to multiple pion exchange:

$$T_\pi^2(r) = [T_\pi(r)]^2 = \left[\left(1 + \frac{3}{m_\pi r} + \frac{3}{(m_\pi r)^2}\right)\frac{e^{-m_\pi r}}{m_\pi r}\right]^2.$$

and their derivatives, which are used as an empirical correction, along with Woods–Saxon terms $W(r)$ in the form of (2.6). A detailed derivation of the operators is available in the original Argonne group article [13].

The Argonne V18 potential is most conveniently written by projecting the channel functions defined in $(T, S)$ onto an operator basis built from spin ($\boldsymbol{\sigma}$), isospin ($\boldsymbol{\tau}$),tensor $S_{12}$, and orbital operators involving $\mathbf{L}$ and $\mathbf{S}$. The first 14 (charge–independent) operators are

$$\begin{array}{llll} O_1 = 1, & O_2 = \boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2, & O_3 = S_{12}, & O_4 = \mathbf{L}\cdot\mathbf{S}, \\ O_5 = L^2, & O_6 = (\mathbf{L}\cdot\mathbf{S})^2, & O_7 = (\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2), & O_8 = (\boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2)(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2), \\ O_9 = S_{12}(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2), & O_{10} = (\mathbf{L}\cdot\mathbf{S})(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2), & O_{11} = L^2(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2), & O_{12} = (\mathbf{L}\cdot\mathbf{S})^2(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2). \end{array}$$

Here $S_{12} = 3(\boldsymbol{\sigma}_1\cdot\hat{\mathbf{r}})(\boldsymbol{\sigma}_2\cdot\hat{\mathbf{r}}) - \boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2$ is the tensor operator. The four remaining operators introduce Charge–dependence (CD) and charge–asymmetry (CA) effects:

$$\begin{aligned} &O_{13} = T_{ij}, && O_{14} = (\boldsymbol{\sigma}_i\cdot\boldsymbol{\sigma}_j)T_{ij}, && O_{15} = S_{ij}\,T_{ij}, \quad O_{16} = \tau_{zi} + \tau_{zj}, \\ &O_{17} = (\boldsymbol{\sigma}_i\cdot\boldsymbol{\sigma}_j)(\tau_{zi} + \tau_{zj}), && O_{18} = S_{ij}(\tau_{zi} + \tau_{zj}), \end{aligned}$$

where

$$T_{ij} = 3\,\tau_{zi}\tau_{zj} - \boldsymbol{\tau}_i\cdot\boldsymbol{\tau}_j$$

is the standard CD isospin operator.

To perform the projection from $(S, T)$ channels to the operator basis, introduce the spin and isospin projectors

$$P_{S=1} = \tfrac{1}{4}(3 + \boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2)\,, \quad P_{S=0} = \tfrac{1}{4}(1 - \boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2)\,, \qquad P_{T=1} = \tfrac{1}{4}(3 + \boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2)\,, \quad P_{T=0} = \tfrac{1}{4}(1 - \boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2)\,.$$

Starting from the channel form

$$V(r) = \sum_{S,T}\Big[v^c_{ST}(r) + v^t_{ST}(r)\,S_{12} + v^{ls}_{ST}(r)\,\mathbf{L}\cdot\mathbf{S} + v^{l^2}_{ST}(r)\,L^2 + v^{(ls)^2}_{ST}(r)\,(\mathbf{L}\cdot\mathbf{S})^2\Big]\,P_S P_T,$$

and expanding $P_S P_T$ on the scalar operator basis, one collects coefficients in front of $\{1,\ \boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2,\ \boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2,\ (\boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2)(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2)\}$ (and the analogous sets multiplying $S_{12}$, $\mathbf{L}\cdot\mathbf{S}$, $L^2$, $(\mathbf{L}\cdot\mathbf{S})^2$), obtaining the compact operator form

$$V(r) = \sum_{p=1}^{18} v_p(r)\,O_p,$$

with $v_p(r)$ the radial functions determined by OPE, EM, and the phenomenological terms.

An effective, most tractable form of this potential is the **ArgonneV6 potential**, which excludes the quadratic–orbital, spin–orbit and EM components of the interaction, which can eventually be added perturbatively. This approximation leads to an expansion over 6 operators, which describes the **static part** of the V18 potential, mainly dependent on strong interaction:

$$V = \sum_{i=1}^{6} V_i(r)\hat{O}_i \tag{2.7}$$

where

$$\begin{aligned} &O_1 = \mathbb{I}, && O_2 = \boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2, && O_3 = \boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2, \quad O_4 = (\boldsymbol{\sigma}_1\cdot\boldsymbol{\sigma}_2)(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2), \\ &O_5 = S_{12}, && O_6 = S_{12}(\boldsymbol{\tau}_1\cdot\boldsymbol{\tau}_2) \end{aligned} \tag{2.8}$$

The radial functions are obtained by re–fitting the experimental data for the AV6 potential, leading to different weights on the combinations that mainly include Yukawa, Woods–Saxon

and OPE terms:

$$
\begin{aligned}
V_1(r)\ \text{(central isoscalar)}: &\quad V_1 \ \propto\ T_\pi^2(r) \ +\ \frac{d}{dr}T_\pi^2(r) \ +\ \frac{d}{dr}\Big[\tfrac{T_\pi^2(r)}{r}\Big] \ +\ W(r),\\
V_2(r)\ \text{(spin–spin isoscalar)}: &\quad V_2 \ \propto\ T_\pi^2(r) \ +\ \frac{d}{dr}T_\pi^2(r) \ +\ \frac{d}{dr}\Big[\tfrac{T_\pi^2(r)}{r}\Big] \ +\ W(r),\\
V_3(r)\ \text{(central isovector)}: &\quad V_3 \ \propto\ T_\pi^2(r) \ +\ \frac{d}{dr}T_\pi^2(r) \ +\ \frac{d}{dr}\Big[\tfrac{T_\pi^2(r)}{r}\Big] \ +\ W(r),\\
V_4(r)\ \text{(spin–spin isovector)}: &\quad V_4^{\mathrm{OPE}} \ \propto\ Y_\pi(r) \ +\ \Big[T_\pi^2(r),\ \tfrac{d}{dr}T_\pi^2(r),\ \tfrac{d}{dr}\big(\tfrac{T_\pi^2}{r}\big),\ W(r)\Big],\\
V_5(r)\ \text{(tensor isoscalar)}: &\quad V_5 \ \propto\ T_\pi^2(r) \ +\ \frac{d}{dr}T_\pi^2(r) \ +\ \frac{d}{dr}\Big[\tfrac{T_\pi^2(r)}{r}\Big] \ +\ W(r),\\
V_6(r)\ \text{(tensor isovetcorial)}: &\quad V_6^{\mathrm{OPE}} \ \propto\ T_\pi(r) \ +\ \Big[T_\pi^2(r),\ \tfrac{d}{dr}T_\pi^2(r),\ \tfrac{d}{dr}\big(\tfrac{T_\pi^2}{r}\big),\ W(r)\Big].
\end{aligned}
$$

Finally, using the nucleon nucleon potential defined, it is possible to obtain a general expression of the Hamiltonian describing a nuclei with $A$ nucleons:

$$
H \ =\ \sum_{i=1}^{A}\frac{\mathbf{p}_i^{\,2}}{2m_N} \ +\ \sum_{1\le i<j\le A}\ \sum_{p=1}^{6} v_p(r_{ij})\,O_{ij}^{(p)}, \tag{2.9}
$$

where

$$
r_{ij}=|\mathbf{r}_i-\mathbf{r}_j|,\quad \hat{\mathbf{r}}_{ij}=\frac{\mathbf{r}_i-\mathbf{r}_j}{r_{ij}}.
$$

## 2.5 Statistical Physics

Statistical physics studies systems composed of an extremely large number of degrees of freedom. Treating such systems by following the microscopic dynamics of each particle is impossible in practice. The key idea is therefore to replace the detailed microscopic description with a statistical one: physical observables are obtained as averages over a suitable probability distribution on the space of microstates.

The central tools are the *statistical ensembles*. The basic postulate is that a given macroscopic state (fixed by constraints such as $T$, $V$, $N$) corresponds to a very large number of microstates $\{x\}$, each representing a configuration in phase space. In equilibrium, each microstate compatible with the macroscopic constraints is assigned a probability weight. For the canonical ensemble (fixed $T, V, N$) this weight is the Boltzmann factor $\beta$:

$$
P(x)=\frac{e^{-\beta E(x)}}{Z},\qquad \beta=\frac{1}{k_B T},
$$

where $E(x)$ is the energy of the microstate $x$ and

$$
Z=\sum_x e^{-\beta E(x)}
$$

is the **partition function**. The partition function normalizes the distribution and encodes all the statistical information of the system. Ensemble averages of physical observables are

then computed as

$$\langle \mathcal{O} \rangle = \sum_x \mathcal{O}(x)\, P(x).$$

Thermodynamic quantities can be derived from $Z$ using standard identities. For instance, the Helmholtz free energy is

$$F = -\frac{1}{\beta} \ln Z,$$

And the derivatives of $F$ yield all equilibrium thermodynamic observables. Thus, the statistical approach replaces the impossible task of tracking each microscopic trajectory with a well–defined probabilistic description, where macroscopic properties are obtained from averages over the ensemble, allowing to efficiently describe the properties of complex systems.

### 2.5.1 The Ising model

A paradigmatic model in statistical physics is the Ising model, originally introduced to describe ferromagnetism. It consists of $N$ two–state variables ("spins") $s_i = \pm 1$ located on the sites of a lattice, interacting through nearest neighbor couplings. The energy of a spin configuration $\{s_i\}$ in the presence of an external field $h$ is

$$E(\{s_i\}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i,$$

where $J$ is the coupling constant and $\langle i, j \rangle$ denotes nearest neighbors.

The probability of observing a configuration in the canonical ensemble is

$$P(\{s_i\}) = \frac{e^{-\beta E(\{s_i\})}}{Z}, \qquad Z = \sum_{\{s_i = \pm 1\}} e^{-\beta E(\{s_i\})}.$$

The Ising model displays a phase transition: in two and three dimensions, when the temperature is lowered below a critical value $T_c$, the system spontaneously magnetizes (ferromagnetic phase). Above $T_c$ it becomes paramagnetic. This nontrivial phenomenon emerges from the cooperative behavior of many interacting degrees of freedom, despite the simplicity of the microscopic variables.

Beyond magnetism, the Ising model is a minimal and widely used benchmark for complex interacting systems, as many problems can be tackled by mapping them into a spin lattice, from neural network models to image processing, and generally every system involving collective behavior in high dimensional spaces.

# Chapter 3

# Numerical Approaches

As physical systems become more complex, analytical solutions become exceedingly rare. In particular, the Schrödinger equation becomes rapidly intractable as the number of bodies $N$ increases, since the dimensionality of the associated Hilbert space grows exponentially with $N$. This computational explosion motivated the development of numerical approaches specifically designed for high–dimensional spaces.

In this chapter, a general overview of these methods is presented. Classical Markov Chain Monte Carlo techniques are introduced first, with the Metropolis algorithm. Then, Variational Monte Carlo (VMC) is discussed in the context of quantum systems.

## 3.1 Monte Carlo Methods

Monte Carlo methods are numerical approaches based on repeated random sampling to obtain numerical results. The theory behind this method is probability and stochastic processes, while the applicative tools include random numbers and sampling of random variables.

Since computers are deterministic machines, they cannot generate truly random numbers in the strict mathematical sense. However, there exist several algorithms that generate **pseudo–random numbers**: once the sequence is started, they appear unpredictable, but restarting the same algorithm with the same seed yields exactly the same sequence. The simplest (uniform) random number generator is the linear congruential generator, which is based on the formula:

$$x_i = (ax_{i-1} + c) \bmod m$$

where $x_i$ is the $i$-th random number of the sequence and $a, c, m$ are properly chosen integers. The seed of the sequence is the starting number $x_0$. This is useful as a very simple example of a pseudo random number generator, but it is outdated because modern generators have more sophisticated properties.

The important thing to mention is that building a generator which samples numbers uniformly distributed in $[0, 1)$ is sufficient to **sample numbers from any probability distribution**. This can be done by two fundamental methods:

**Inverse cumulative method.** Given a random variable $X$ with cumulative distribution function[1] $F(x)$, one samples a uniform random number $u \in [0, 1]$, and defines

$$x = F^{-1}(u).$$

Then $x$ is distributed according to the target distribution $F(x)$. In fact:

$$P\big(F^{-1}(U) \leq x\big) = P(U \leq F(x)) = F(x).$$

**Accept–Reject method.** Let $f(x)$ be the target pdf, and $g(x)$ a simpler one (such as the uniform distribution). Assume that there exists a constant $M$ such that

$$f(x) \leq M\, g(x) \quad \text{for all } x.$$

Algorithm:

1. draw $Y \sim g(x)$
2. draw $U \sim \text{Unif}(0, 1)$
3. if $U \leq \dfrac{f(Y)}{M\, g(Y)}$ then accept and set $X = Y$; otherwise reject and repeat

If the draw is accepted, then $X$ is distributed according to $f(x)$. The acceptance rate is $1/M$.

Monte Carlo methods are ultimately used to estimate **expectation values** of the form

$$\mathbb{E}_f[h(X)] = \int h(x) f(x)\, dx \;\approx\; \frac{1}{N} \sum_{i=1}^{N} h(x_i),$$

by replacing the integral with the sample average over the generated samples. The more samples, the lower the statistical error [2]. The key feature of these approaches is that the errors associated to Monte Carlo estimators converge as $1/\sqrt{N}$, where $N$ is the number of samples, independently of the dimensionality of the problem, which makes them extremely useful in evaluating high–dimensional integrals efficiently.

### 3.1.1 The Metropolis-Hastings algorithm

The **Metropolis algorithm** allows to sample from an arbitrary complex, n-dimensional probability distribution function, without knowing its normalization constant, which makes it the optimal framework for drawing samples from a wavefunction. This algorithm represents a theoretical step forward in Monte Carlo techniques, since it is a pioneering approach to Markov Chain Monte Carlo (MCMC) implementations [15].

[1] In probability theory, the cumulative function of a real-valued random variable $X$, evaluated at $x$, is defined as the probability that $X$ will take a value less than or equal to $x$: $F_X(x) = P(X \leq x) = \int_{-\infty}^{x} f_X(t)\, dt$.

[2] Central Limit Theorem: if $X_1, \ldots, X_N$ are i.i.d. with variance $\sigma^2$, then the sample mean $\bar{X}_N$ satisfies $\sqrt{N}(\bar{X}_N - \mathbb{E}[X]) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$. Therefore $\text{Var}(\bar{X}_N) = \sigma^2/N$.

**Markov Chains** A Markov chain is a stochastic process $\{X_n\}_{n\geq 0}$ such that

$$P(X_{n+1} = x_{n+1} \mid X_n = x_n, X_{n-1} = x_{n-1}, \ldots, X_0 = x_0) = P(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

Namely, the future depends only on the present state, not on the full history: a Markov chain represents a process with no **memory**. Here follow some useful definitions about a Markov chain:

- A Markov chain is **irreducible** if for every pair of states $i$ and $j$ there exists an integer $n \geq 1$ such that $P(X_n = j|X_0 = i) > 0$: the process explores all the possible different states.

- A Markov chain is **aperiodic** if no state has a fixed period greater than one. Equivalently, for every state $i$, the greatest common divisor of all $n$ such that $P(X_n = i|X_0 = i) > 0$ is equal to 1: the process never gets stuck in a loop.

It can be shown that if a Markov chain is aperiodic and irreducible, it is also **ergodic**, which means that it admits a unique stationary distribution and the distribution of $X_n$ converges to it as $n \to \infty$, independently of the initial state. A more intuitive definition of ergodicity comes from statistical mechanics: the chain is ergodic if, given enough time ($t \to \infty$), it explores the whole configuration space with its own probability density.

In this context, the Metropolis algorithm is based on the idea of creating an ergodic Markov chain through an accept/reject procedure. Starting from a state $x$, a move is towards a new state $y$ is proposed by using an arbitrary **transition proposal distribution** $T(x|y)$. The proposed move is then accepted or rejected depending on the comparison between $p(x)$ and $p(y)$, taking into account the respective transition probabilities.

It can be shown that a necessary condition to make the Metropolis Markov chain ergodic is that the transition probability satisfies the **detailed balance**:

$$p(x)T(y|x) = p(y)T(x|y)$$

which states that, at equilibrium, the probability of transitioning from $x \to y$ is the same as the one of transitioning from $y \to x$.

On the overall, the algorithm reads:

**Algorithm 1** Metropolis–Hastings Algorithm

**Require:** initial state $x^{(0)}$, target density $p(x)$, proposal $T(x'|x)$
**Ensure:** $\{x^{(r)}\}_{r=1,\dots,R}$ samples from $p(x)$

**for** $n = 0$ to $R$ **do**
  $x = x_n$
  generate proposal $x'$ from $T(x'|x_n)$
  compute

$$A(x'|x_n) = \min\left(1, \frac{T(x_n|x')\,p(x')}{T(x'|x_n)\,p(x_n)}\right)$$

  generate $r \sim U(0,1)$
  **if** $r \leq A(x'|x_n)$ **then**
    $x_{n+1} = x'$
  **else**
    $x_{n+1} = x_n$
  **end if**
**end for**

It is important to notice that this algorithm samples from the target distribution only asymptotically, so an initial equilibration (burn-in) phase is required in order for the chain to reach its stationary regime. Moreover, successive samples are typically correlated, which affects the statistical efficiency of the estimator.

### 3.1.2 Variational Monte Carlo

The difficulty of solving the schrodinger equation as the nuber of bodies grows has already been mentioned. In the Variational Monte Carlo (VMC) framework, the key idea is to reduce the problem of finding the wavefunction of a system into a **stochastic optimization** one.

**The variational principle** The energy functional $E[\psi]$ computed using an arbitrary state $\psi$ will always be greater, or at least equal, to the exact ground state energy $E_0$ of the system:

$$E[\psi] = \langle\psi|\hat{H}|\psi\rangle \geq E_0 \tag{3.1}$$

So that the problem of finding the wavefunction becomes an optimization problem:

$$\psi_0 = argmin_\psi E[\psi]$$

However, this again typically requires the evaluation of high dimensional integrals, so that a stochastic approach is valuable.

In general, the expectation value of any quantum operator $O$ can be written, using the closure relation $\sum_x |x\rangle\langle x| = \mathbb{I}$ , as:

$$\frac{\langle\Psi|O|\Psi\rangle}{\langle\Psi|\Psi\rangle} = \frac{\sum_{x,x'}\langle\Psi|x\rangle\langle x|O|x'\rangle\langle x'|\Psi\rangle}{\sum_x \langle\Psi|x\rangle\langle x|\Psi\rangle} = \frac{\sum_{x,x'}\Psi^\star(x)\,O_{xx'}\,\Psi(x')}{\sum_x |\Psi(x)|^2}\,. \tag{3.2}$$

If the operators is diagonal on the computational basis, then $O_{xx'} = \delta_{xx'}O(x)$, then the quantum expectation value become equivalent to statistical averages on samples drawn from $|\psi(x)|^2$:

$$\frac{\langle\Psi|O|\Psi\rangle}{\langle\Psi|\Psi\rangle} = \frac{\sum_x |\Psi(x)|^2 O(x)}{\sum_x |\Psi(x)|^2} \equiv \langle\langle O \rangle\rangle . \tag{3.3}$$

If the operator is not diagonal in the computational basis, it can be replaced with a *local* operator $O_{loc}$:

$$O_{loc}(x) = \sum_x O_{xx'} \frac{\psi(x')}{\psi(x)} \tag{3.4}$$

by applying this substitution, the local operator has the same property as a diagonal one:

$$\frac{\langle\Psi|O|\Psi\rangle}{\langle\Psi|\Psi\rangle} = \frac{\sum_x |\Psi(x)|^2 O_{\mathrm{loc}}(x)}{\sum_x |\Psi(x)|^2} \equiv \langle\langle O_{\mathrm{loc}} \rangle\rangle .$$

In this framework, VMC gives a way to evaluate stochastically every expectation value of interest by computing statistical averages over arbitrary wavefunctions. The sum over exponentially many states is thus replaced with a statistical average over a large but finite set of states sampled according to the probability distribution $|\psi(x)|^2$.

Since the Hamiltonian is, in general, an off-diagonal operator, the **local energy**, to be minimized, can be defined as:

$$E_{loc}(\mathbf{x}) = \sum_{\boldsymbol{x}'} H_{xx'} \frac{\psi(\mathbf{x}')}{\psi(\mathbf{x})} \tag{3.5}$$

so that:

$$\langle H \rangle = \langle\langle E_{loc} \rangle\rangle.$$

The optimization of the local energy can also be treated stochastically, by defining:

$$D_k = \frac{1}{\psi(\mathbf{x})} \frac{\partial \psi(\mathbf{x})}{\partial p_k}$$

where $p_k$ denotes the $k$-th parameter of $\psi$. It can be shown, with detailed calculations available at [16], that:

$$\frac{\partial}{\partial p_k}\langle H \rangle \approx \langle\langle E_{loc} D_k^* \rangle\rangle - \langle\langle E_{loc} \rangle\rangle \langle\langle D_k^* \rangle\rangle + const \approx \langle\langle G_k \rangle\rangle$$

so that the **gradient estimator** $G_k$ has the form:

$$G_k(\mathbf{x}) = 2\,\mathrm{Re}\big[\big(E_{\mathrm{loc}}(\mathbf{x}) - \langle E_{\mathrm{loc}} \rangle\big)\, D_k^\star(\mathbf{x})\big] .$$

### 3.1.3 Stochastic gradient descent

The simplest optimization scheme to tackle functions with a high number of parameters is the **stochastic gradient descent** (SGD). It is based on the classical gradient descent method, which seeks the minimum of a function by taking steps in the direction opposite to its gradient.

At each iteration $i$, each variational parameter $p_k^i$ is updated according to:

$$p_k^{i+1} = p_k^i - \eta \frac{\partial}{\partial p_k} \langle H \rangle \approx p_k^i - \eta \langle \langle G_k \rangle \rangle \tag{3.6}$$

where $\eta$ is an hyperparameter called **learning rate**.

Since gradients are evaluated stochastically, they are subject to statistical noise. Assuming Gaussian noise for simplicity, one can model:

$$\frac{\partial}{\partial p_k} \langle H \rangle \sim \mathcal{N}\left(\langle G_k \rangle, \sigma\right)$$

so that the update rule can be recast as a discretized stochastic differential equation:

$$p_k^{i+1} = p_k^i - \delta_t \langle G_k \rangle + \mathcal{N}\left(0, \sqrt{2\delta_t T}\right)$$

where $\delta_t$ can be interpreted as a discrete time step, and $T$ plays the role of an effective temperature.

The key insight is that, in the limit $t \to \infty$, this Langevin dynamics samples the variational parameters from a **Boltzmann distribution**:

$$\Pi(p_1, \ldots, p_N) \propto e^{-\langle H \rangle / T}$$

Hence, parameter configurations with lower expected energy are statistically favored. The parameter $T$ acts like a physical **temperature** in a statistical mechanical system: as $T \to 0$, the distribution collapses onto the global minimum of the energy landscape, ideally corresponding to the ground state of the system.

Furthermore, the variance of the gradient estimator can be identified with the temperature as:

$$\sigma^2 = \mathrm{Var}(G_k) = \frac{2T}{\delta_t} \qquad \text{with} \qquad \eta = \delta_t \tag{3.7}$$

To reduce the gradient noise and improve convergence, one can gradually lower the temperature following a **simulated annealing schedule**, where at each iteration the temperature satisfies:

$$T_1 > T_2 > \cdots > T_{N_{\text{iter}}}$$

This strategy allows for broad exploration of the parameter space at high temperatures, followed by refined sampling around the global minimum as the temperature decreases.

Solving Eq. (3.7) for $T$ shows that decreasing the temperature can be achieved either by increasing the number of samples per iteration (reducing variance) or by decreasing the learning rate $\eta$. In practice, both techniques are combined, and the learning rate is made to **decay during the iterations**:

$$\eta = \eta\left(n_{iter}\right)$$

The explicit decay law for the learning rate is task–specific, but typically power or exponential decays are implied.

Now that all the tools needed have been presented, here follows a scheme of an iterative sequence of steps for implementing VMC algorithms:

1. *Ansatz choice*: choose an arbitrary wavefunction $\psi_{\theta_i}(\mathbf{x})$ dependent on a set of parameters $\theta_i, i = \ldots, N$.

2. *Energy evaluation*: estimate the energy functional $E[\psi]$ via Monte Carlo sampling.

3. *Parameters update*: update ansatz' parameters to minimize the energy

4. *iteration until convergence.*

Where the convergence can be observed by exploiting the **zero variance property**: as $E[\psi]$ gets closer to $E_0$, the variance associated to it $\sigma_E^2 \to 0$. in fact:

$$\mathrm{var}(E_{\mathrm{loc}}) = \langle E_{\mathrm{loc}}^2 \rangle - \langle E_{\mathrm{loc}} \rangle^2 = \sum_x |\Psi(x)|^2 \, E_{\mathrm{loc}}(x)^2 - \langle H \rangle^2 =$$

$$\sum_x \sum_{x_1} H_{x,x_1} \Psi(x_1) \sum_{x_2} H_{x,x_2} \Psi(x_2) - \langle H \rangle^2 = \langle H^2 \rangle - \langle H \rangle^2 \, .$$

so if $\Psi$ is an eigenstate of $H$, then

$$\langle H^2 \rangle = \langle H \rangle^2 = E_0^2, \quad \text{and} \quad \mathrm{var}(E_{\mathrm{loc}}) = 0. \tag{3.8}$$

# Chapter 4

# Machine learning

Machine Learning (ML) is a branch of artificial intelligence which implements algorithms capable of extracting patterns from data to improve their performance through experience, rather than by being explicitly programmed with task-specific rules.

In general, a **model** is represented as a family of functions that depend on $N$ parameters $\theta_i$:

$$f(\mathbf{x}; \theta_1, \ldots, \theta_N)$$

The learning process happens during the **training** of the model, in which the parameters are optimized by minimizing a **loss function** $L(\theta)$. After training, the learned mapping should perform well also on unseen data, which means the system must capture general statistical structure rather than memorize the training set.

ML indeed is not a single paradigm, but rather a general framework that includes: *supervised*, *unsupervised* and *reinforcement* learning.

**Supervised learning** The model is used to learn patterns from given data. Main tasks of supervised learning include:

- *Classification*: discrete output to classify a given input into pre-determined classes (e.g. image recognition)
- *Regression*: continuous output to approximate an underlying function given a dataset (e.g. noisy function fitting)

In this context, the model is trained upon a **labelled dataset**, which means that the dataset contains input data and the associated labels $(X, Y)$. The model is trained by minimizing a loss, which typically represents the difference between the model's predictions and the labels. A common loss for regression tasks is the Mean Squared Error (MSE):

$$\mathrm{MSE}(\theta) = \frac{1}{M} \sum_{i=1}^{M} \left( f(x^{(i)}; \theta) - y^{(i)} \right)^2$$

**Unsupervised learning** In unsupervised learning the data are not labelled: the dataset contains only inputs $X$ with no associated targets $Y$. The objective is not to match given labels, but to discover statistical structure in the data. Typical tasks include:

- *Clustering*: find groups of similar samples in the dataset.
- *Dimensionality reduction*: learn a low dimensional representation that preserves relevant information.

The model is trained by optimizing an objective function that rewards compact representations, structure discovery or data likelihood, rather than the fit to explicit labels.

**Reinforcement learning** The model is an agent which interacts with an environment by taking actions. For every action taken by the agent, the environment returns the new state (effect of the action) and a reward, based on the effectiveness of the move. The agent is trained to learn a **policy**(a function used to sample the actions) that maximizes the expected reward:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

## 4.1 Artificial neural networks

Artificial neural networks are a class of ML models loosely inspired by biological neurons. An artificial neuron is a computational unit which receives an $N$–dimensional input, computes an affine transformation through internal weights and biases, and applies a pointwise non–linear activation. Neurons are organized in **layers**, whose composition defines the network. This architecture is extremely flexible and can be applied in supervised, unsupervised and reinforcement learning.

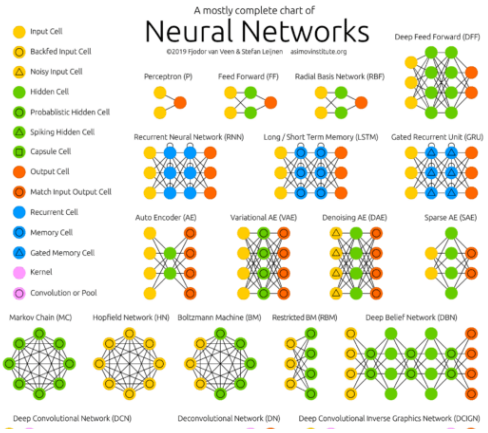



Figure 4.1: A chart of the most common neural network architectures.

More recent **deep learning** (DL) implies the use of deep neural networks, namely architectures with many layers.
There exist many neural network architectures, each one designed for a specific class of tasks. The three most widespread ones include:

- *Multilayer perceptrons (MLPs)*: fully connected feedforward networks, commonly used for classification and regression.
- *Convolutional neural networks (CNNs)*: networks with convolutional and pooling layers, used to exploit local correlations in the input (e.g. image recognition).
- *Recurrent neural networks (RNNs)*: networks with recurrent connections that maintain a hidden state over time, used for sequential data such as time series and language.

### 4.1.1 Perceptrons

The first example of neural network literature, yet the simplest one, introduced by Rosenblatt in his seminal work [17], is the **perceptron**. A perceptron is a linear neuron with threshold activation. It operates on the input (an $N$–dimensional vector $\vec{x}$) by applying a linear transformation:

$$a = \sum_i w_i x_i + b$$

where $\vec{w}$ is the vector of **weights** and $b$ the **bias**. This quantity is then compared to a threshold value $T$, and the perceptron outputs $+1$ if $a > T$, or $-1$ otherwise. This threshold evaluation constitutes the **activation function**, so that the perceptron action can be written as:

$$o = f\left(\sum_i w_i x_i + b\right)$$

The main limitation of the perceptron is that the hard threshold activation produces a discrete, non–differentiable output. This prevents the use of efficient gradient based optimization and makes the output highly sensitive to small changes in the weights. Modern neural networks replace the threshold with smooth non-linear activations, which are differentiable and output real values, greatly increasing both trainability and expressivity.

### 4.1.2 Training neural networks

Training a neural network requires adjusting weights and biases in order to minimize a loss function. Modern neural networks are trained with gradient-based optimization, which requires the gradient of loss with respect to every parameter in the network. One of the most widely used approaches is the **stochastic gradient descent**, which updates parameters in the direction opposite to the gradient of the loss computed on small random batches of data:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}_B(\theta)$$

In deep neural networks, the loss function $\mathcal{L}$ typically depends on a very large number of parameters (weights and biases). Computing all the partial derivatives $\partial\mathcal{L}/\partial\theta$ by applying the chain rule naively would be prohibitively expensive. The **backpropagation algorithm** provides an efficient way to compute these gradients by propagating error signals backward through the network's computational graph.

Consider a feed forward neural network, in which the information is propagated only in one direction through layers, with layers indexed by $l = 1, 2, \ldots, L$. For each layer $l$, the pre-activation vector is defined as

$$a^{(l)} = W^{(l)} x^{(l-1)} + b^{(l)},$$

where $W^{(l)}$ and $b^{(l)}$ are respectively the weight matrix and bias vector of layer $l$, and $x^{(l-1)}$ is the activation of the previous layer. The activation of layer $l$ is obtained by applying the nonlinearity $f$ component-wise:

$$x^{(l)} = f\big(a^{(l)}\big)\,.$$

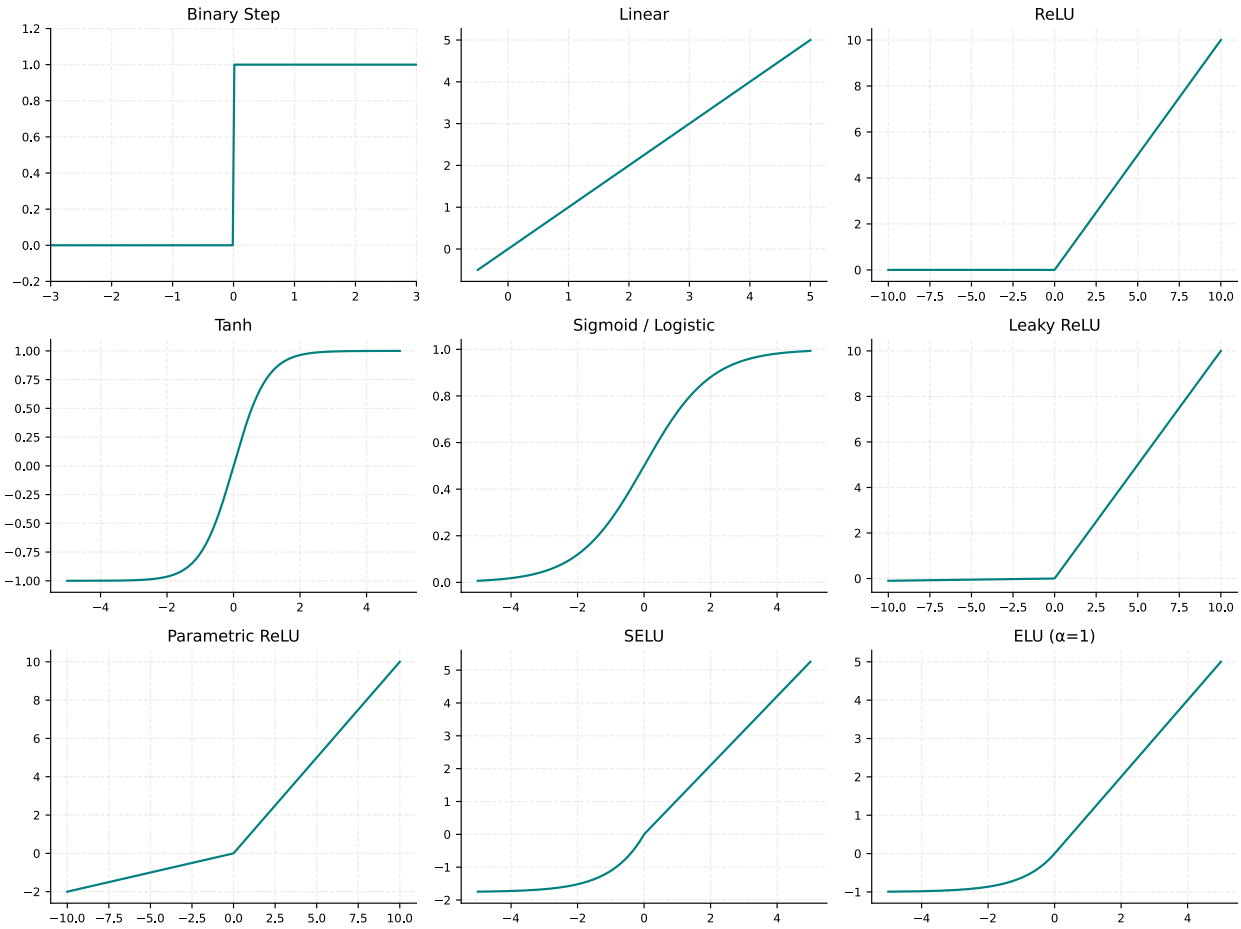



Figure 4.2: A chart of the most common activation functions.

To perform backpropagation, the *error term* (or local gradient) is introduced as:

$$\delta^{(l)} := \frac{\partial \mathcal{L}}{\partial a^{(l)}},$$

which measures how sensitive the loss is to changes in the pre-activations of layer $l$. By applying the chain rule, the error at layer $l$ can be expressed recursively in terms of the error at the next layer:

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot f'\left(a^{(l)}\right),$$

where $\odot$ denotes the element-wise product and $f'$ is the derivative of the activation function.

Once the error terms $\delta^{(l)}$ are known for all layers, the gradients of the loss function with respect to the trainable parameters follow directly. For each layer $l$, one has:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(x^{(l-1)}\right)^T, \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}.$$

These expressions show that the gradient of the weights is simply the outer product between the error at layer $l$ and the activations of the previous layer, while the gradient of the bias is

the error itself. After computing these gradients for all layers, the parameters are updated using an optimization algorithm such as stochastic gradient descent (SGD) or one of its variants (e.g. Adam).

This mechanism ensures that even very deep networks with millions of parameters can be trained efficiently, since all gradients are obtained with a single backward pass whose computational cost is comparable to that of the forward pass.

# Chapter 5

# Restricted Boltzmann Machines

The Restricted Boltzmann Machine (RBM) architecture was introduced in the mid 1980s as a simplified version of the original Boltzmann Machine proposed by Ackley, Hinton and Sejnowski in 1985 [2]. The original model was a fully connected stochastic neural network that defined a probability distribution over binary variables through an energy function, but exact inference and learning were computationally intractable because of the dense connectivity. In the RBM this connectivity is "restricted": intralayer connections (visible to visible and hidden to hidden) are removed, and only visible to hidden connections remain. This structural constraint does not severely reduce the expressive power of the model, and it has a key computational consequence: units in the same layer become conditionally independent, making inference and training tractable.

The RBM is composed of two layers: a *visible* and a *hidden* one, where the probability associated to each configuration of visible units $\boldsymbol{\sigma}$ and hidden ones $\boldsymbol{h}$ is given by the Boltzmann distribution:

$$P(\sigma, h) = \frac{e^{-E(\sigma,h)}}{Z},$$

where $Z$ is the partition function. Because of the bipartite structure, hidden units are conditionally independent given the visible units configuration $\boldsymbol{\sigma}$:

$$P(\mathbf{h}|\boldsymbol{\sigma}) = \prod_j P(h_j|\boldsymbol{\sigma}),$$

this means that the hidden units can be independently sampled once the visible configuration is known:

$$P(h_j = 1|\boldsymbol{\sigma}) = \text{logistic}\big(b_j + \sum_i W_{ij}\sigma_i\big),$$

where

$$\text{logistic}(x) = \frac{1}{1 + e^{-x}}$$

is the sigmoid activation function. Given a visible configuration $\boldsymbol{\sigma}$, each hidden unit $h_j$ has an explicit probability, independent from the other hidden units, so that each hidden configuration can be directly sampled in parallel. In this framework, an RBM parametrizes a family

of probability distributions by assigning an energy value to each configuration. Due to the Boltzmann form, lower–energy configurations have higher probability, so learning amounts to adjusting the parameters such that high–probability data configurations are assigned low energy.

## 5.1 RBMs as universal approximators

**Kolmogorov–Arnold representation theorem** [18] demonstrates that any continuous $N$–dimensional function can be represented as a finite superposition of nonlinear one variable functions. This result implies the existence of parametric models that, in principle, can approximate any continuous function on a compact domain, and interpreting neural networks as such models suggests that they could act as universal function approximators. However, each neural network architecture possesses its own specific parametrization, so the universal approximation property must be proved separately for each case.

In the case of RBMs, Le Roux and Bengio [19] proved several important properties, including universal approximation. Their analysis relies on the **Kullback–Leibler divergence**, a quantity used to measure how much a probability distribution $P$ differs from a target one $\hat{P}$ in terms of Shannon entropy:

$$D_{\mathrm{KL}}(\hat{P} \,\|\, P) = \sum_x \hat{P}(x) \, \log \frac{\hat{P}(x)}{P(x)}$$

Minimizing this quantity means approximating the target distribution arbitrary well.

The first theorem proposed shows that adding a single hidden unit improves the approximation given from the RBM, if it doesn't already perfectly model the target distribution:

**Theorem 1.** *Let $\hat{P}$ be an arbitrary distribution over $\{0,1\}^N$, and let $R_P$ be an RBM with marginal distribution $P$ over the visible units such that $KL(\hat{P}\|P) > 0$. Then there exists an RBM $R_{P'}$ composed of $R_P$ and an additional hidden unit $h_i$ with parameters $(W^{(i)}, b_i)$ whose marginal distribution $P'$ over the visible units achieves $KL(\hat{P}\|P') < KL(\hat{P}\|P)$.*

The second theorem proves that RBMs can model any discrete probability distribution:

**Theorem 2.** *(Universal Approximation) Any distribution over $\{0,1\}^N$ can be approximated arbitrarily well (in the sense of the KL divergence) with an RBM with $k+1$ hidden units where $k$ is the number of input vectors $v$ whose probability is not* 0*.*

Although this theorem proves a universal approximation property for RBMs, its practical use is limited. The reason is that the number of hidden units required by the theorem scales with the number of input configurations having nonzero probability, which typically grows exponentially with the number of visible units. As a consequence, applying this construction to realistic systems would require an intractable number of parameters, defeating the efficiency and compactness that make RBMs appealing in the first place. To address this limitation, Montúfar and Ay [20] proposed an improved formulation, showing that the same universal approximation property can be achieved with significantly fewer hidden units, as reported below.

**Theorem 3.** *Any distribution $p$ on binary vectors of length $n$ can be approximated arbitrarily well by an RBM with $k-1$ hidden units, where $k$ is the minimal number of pairs of binary vectors, such that the two vectors in each pair differ in only one entry and such that the support set of $\mathcal{P}$ is contained in the union of these pairs.*

This result links the expressive power of the model to the number of connected pairs of configurations rather than to the total number of nonzero configurations, providing a much tighter and practically relevant bound on the required number of hidden units. In physical or structured datasets, where the support of the distribution is typically localized or correlated, this reduction can be dramatic. Each hidden unit can be interpreted as capturing a local correlation between configurations that differ by a single bit, so that the RBM effectively builds a piecewise representation of the probability landscape. This perspective not only makes the universal approximation result constructive, but also connects the representational capacity of RBMs to the topological structure of the data manifold.

## 5.2 Wavefunction representation with RBMs

Quantum wavefunctions can be represented by interpreting a Restricted Boltzmann Machine (RBM) as a **spin model**, with binary-valued units where the visible nodes encode the physical degrees of freedom as spins $\sigma_i \in \{-1, +1\}$. The RBM defines a probability amplitude over visible configurations by marginalizing over the hidden variables:

$$\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) = \sum_{\boldsymbol{h}} \exp\left(\sum_i a_i \sigma_i + \sum_j b_j h_j + \sum_{i,j} W_{ij} \sigma_i h_j\right),$$

where $a_i$, $b_j$ are the visible and hidden biases, respectively, $W_{ij}$ are the complex weights connecting visible and hidden units, and $\boldsymbol{\theta} = \{a, b, W\}$ is the full parameter set. As quantum wavefunctions are generally complex-valued, all parameters of the network are taken to be complex.

Assuming binary hidden units $h_j \in \{-1, +1\}$, the marginalization can be carried out analytically, yielding a closed form expression:

$$\begin{aligned}
\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}) &= \sum_{\{h\}} \exp\left[\sum_{ij} W_{ij}\sigma_i h_j + \sum_j b_j h_j + \sum_i a_i \sigma_i\right] \\
&= \exp\left(\sum_i a_i \sigma_i\right) \prod_j \left(\exp\left[\sum_i W_{ij}\sigma_i + b_j\right] + \exp\left[-\sum_i W_{ij}\sigma_i - b_j\right]\right) \qquad (5.1) \\
&= \exp\left(\sum_i a_i \sigma_i\right) \prod_j 2\cosh\left(\sum_i W_{ij}\sigma_i + b_j\right)
\end{aligned}$$

To construct a wavefunction for a given quantum system, each degree of freedom (either spin or occupation number) is mapped to a visible unit of the RBM. While previous implementations focused on spin models [11, 12], here the focus is put on systems of particles confined in a three dimensional box.

The cubic box is discretized into a lattice of linear size $L$, consisting of $L^3$ sites spaced by $\Delta x$. Each site is indexed by a triple $(i, j, k)$, and its physical coordinate is defined as the center of the corresponding cubic cell:

$$\vec{r}_{ijk} = \Delta x \left( i + \frac{1}{2},\ j + \frac{1}{2},\ k + \frac{1}{2} \right)$$

Each lattice site is associated with a visible node $\sigma_{ijk} \in \{-1, +1\}$, which encodes the occupation number at that site via:

$$n_{ijk} = \frac{1}{2}(1 + \sigma_{ijk}) \in \{0, 1\}$$

A complete physical configuration of the system in the second quantization formalism, $|n_1, \ldots, n_N\rangle$, is therefore represented by a visible configuration of the RBM:

$$\boldsymbol{\sigma} \in \{-1, +1\}^{L^3}$$

which serves as the input to the variational wavefunction $\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$ defined by the network.

In practice, a 3D lattice is flattened into a one dimensional vector by mapping the 3D indices $(i, j, k) \in \{0, \ldots, L-1\}^3$ to a linear index $\alpha \in \{0, \ldots, L^3 - 1\}$ using the standard row-major convention:

$$\alpha = i + L \cdot j + L^2 \cdot k$$

The inverse mapping from $\alpha$ to $(i, j, k)$ is given by:

$$i = \alpha \bmod L \tag{5.2}$$

$$j = \left\lfloor \frac{\alpha}{L} \right\rfloor \bmod L \tag{5.3}$$

$$k = \left\lfloor \frac{\alpha}{L^2} \right\rfloor \tag{5.4}$$

The wavefunction is defined by the RBM up to a normalization constant, which is irrelevant for sampling purposes. In fact, sampling is performed using the Metropolis algorithm, where the acceptance probability for a proposed transition $\boldsymbol{\sigma} \to \boldsymbol{\sigma}'$ is:

$$A(\boldsymbol{\sigma} \to \boldsymbol{\sigma}') = \min\left(1,\ \frac{|\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma}')|^2}{|\psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})|^2}\right)$$

It is convenient to work in logarithmic space, as this simplifies the computation of the acceptance ratio. The logarithm of the RBM wavefunction reads:

$$\log \psi(\boldsymbol{\sigma}) = \sum_i a_i \sigma_i + \sum_j \log\left[ 2\cosh\left( \sum_i W_{ij}\sigma_i + b_j \right) \right] \tag{5.5}$$

Consequently, the logarithm of the ratio can be computed directly as:

$$\log\left(\frac{\psi(\boldsymbol{\sigma}')}{\psi(\boldsymbol{\sigma})}\right) = \sum_i a_i(\sigma_i' - \sigma_i) + \sum_j \log\left(\frac{\cosh(\theta_j')}{\cosh(\theta_j)}\right) \tag{5.6}$$

where $\theta_j$ are precomputed auxiliary quantities (often referred to as *lookup tables*):

$$\theta_j = \sum_i W_{ij}\sigma_i + b_j$$

Since only a subset of the visible units changes at each step, only the corresponding entries if the lookup tables need to be updated. Once the log-ratio is computed, the acceptance probability can be recovered by exponentiating the log-ratio.

Finally, this closed form of RBM provides a simple expression for the **variational derivatives**, which are implied in the chosen optimization scheme:

$$\begin{aligned} D_{a_i}(\boldsymbol{\sigma}) &= \frac{1}{2}\,\sigma_i^z, \\ D_{b_j}(\boldsymbol{\sigma}) &= \frac{1}{2}\,\tanh(\theta_j), \\ D_{W_{ij}}(\boldsymbol{\sigma}) &= \frac{1}{2}\,\tanh(\theta_j)\,\sigma_i^z \end{aligned} \tag{5.7}$$

# Chapter 6

# Neural Network quantum states

As previously mentioned, the problem of determining the ground state wavefunction of a quantum many–body system becomes rapidly intractable as the number of particles $N$ increases. This is due to the exponential growth of the Hilbert space: in general, a quantum wavefunction assigns a complex amplitude to each possible configuration of the system. Since the number of such configurations scales as $d^N$, where $d$ is the local Hilbert space dimension (e.g., $d = 2$ for a spin-$\frac{1}{2}$ system), the amount of information required to store or manipulate the full wavefunction grows exponentially with $N$. Even storing the wavefunction becomes practically impossible beyond modest values of $N$. To overcome this limitation, Carleo and Troyer [3] introduced the use of Restricted Boltzmann Machines (RBMs) as variational *ansätze* to efficiently compress and represent many–body quantum states in a compact, trainable form:

$$\psi_{\boldsymbol{\theta}}(\boldsymbol{x}) = F\big(\boldsymbol{x}, \boldsymbol{\theta}\big)$$

where $F$ is the output of the RBM (5.1). In this way, the variational state is defined as a family of functions which act on the basis:

$$\psi_{\boldsymbol{\theta}} : \boldsymbol{x} \to \mathbb{C}$$

The final request is that the evaluation of $\psi_{\boldsymbol{\theta}}(\boldsymbol{x})$ must happen with a polinomial cost in the number of degrees of freedom of the problem.

A **neural network quantum state** (NQS) is indeed defined as a variational wavefunction for which $\psi_{\boldsymbol{\theta}}(\boldsymbol{x})$ is partially or completely determined by a neural network whose parameters $\boldsymbol{\theta}$ coincide with the variational ones. In this framework, a variational optimization problem is traduced into a machine learning one. As it will be discussed later, the main advantage of this approach is that the number of parameters of the network is fixed for every $N$, while the computational cost still remains prohibitive for growing number of bodies.

## 6.1 State of the art

Carleo and Troyer work has been of great influence in machine learning applications in physics. their original paper explored spin systems, including the transverse Ising model and the antiferromagnetic Heisenberg model, by employing a restricted Boltzmann machine as a

wavefunction ansatz. Here, a list of some relevent applications of NQS and neural networks is presented.

- *Fermionic neural network states for ab-initio electronic structure* [4]: in this article is presented an extension of NQS to model interacting fermionic problems. The authors directly map fermionic degrees of freedom to spin ones, and then use NQS to perform electronic structure calculations, and then compare different methods, including the RBM.

- *Dilute neutron star matter from neural-network quantum states* [5]: in this article the authors model low density neutron matter using hidden-nucleon NQS, along with variational Monte Carlo and stochastic optimization methods.

- *Machine learning the deuteron* [21]: in this article the authors leverage machine learning techniques to address the nuclear two-body bound state problem, implying a simple neural network as ansatz.

- *Solving quasiparticle band spectra of real solids using neural network quantum states* [6]: in this article is demonstrated the effectiveness of artificial neural networks for ab initio calculations of extended periodic materials in condensed matter physics. The method accurately simulates the ground state energies of solids in one, two, and three dimensional systems, achieving chemical accuracy. Additionally, it efficiently extracts quasiparticle band spectra, which are essential for understanding the properties of solid state systems.

- *Variational Monte Carlo Calculations of A  4 Nuclei with an Artificial Neural Network Correlator Ansatz* [7]: in this article, the authors compute the binding energies and point-nucleon densities of $A \leq 4$ nuclei derived from a leading-order pionless effective field theory Hamiltonian. They also compare the artificial neural network (ANN) wavefunction to more traditional parametrizations based on two and three body Jastrow functions.

- *Ab initio quantum chemistry with neural network wavefunctions* [10]: this review outlines the main neural network ansätze used to represent electronic wavefunctions (e.g. RBMs, autoregressive models, deep architectures), and describes how they integrate with variational Monte Carlo to capture strong electron correlation. The authors also highlight current challenges and future developments of NQS methods within ab-initio quantum chemistry.

## 6.2 Many electron atoms wavefunction

The first scenario in which NQS has been implied in this work is the one with many electron atoms, building on Redaelli's code [1].

The physical configuration of the system is represented as mentioned in section 5.2: the electrons are simulated in a three dimensional box of physical length $\ell$ and size $L = \sqrt[3]{n_v}$,

where $n_v$ is the number of visible units of the network, and the nucleus is put at the center of the box. Each visible node corresponds to a physical occupation number, and the configurations are sampled with the Metropolis algorithm.

The RBM used here uses a visible layer of length $2n_v$, which corresponds to two different **spin canals**: visible nodes in the first $[0, \dots, n_v - 1]$ values represent **spin up** electrons, while the remaining represent the **spin down** ones.
It is important to note that this representation of the physical configuration in terms of visible nodes **automatically enforces the Pauli exclusion principle**. Each site in a given spin channel can be occupied by at most one particle, since visible units take only binary values. Consequently, configurations that would violate the exclusion principle are intrinsically excluded from the model's configuration space. This allows the ansatz to be simplified by avoiding the explicit inclusion of Slater determinant states, since the antisymmetry required by the Pauli exclusion principle is already embedded in the binary occupation representation.

The energies are calculated as in equation (3.4), where the ratio is evaluated according to the expression in (5.6). The Hamiltonian is defined on the discretized lattice as:

$$H = T + V$$

Where $V$ is the potential corresponding to the atom to be studied.

### 6.2.1 Kinetic energy

The **discrete kinetic operator** is obtained from a finite difference discretization of the Laplacian operator $\nabla^2$ on a regular lattice, where each grid point is equally spaced by $\Delta x = \frac{\ell}{L}$. For simplicity, consider the one dimensional case (the generalization to three dimensions is straightforward):

$$\nabla^2 \psi(x_i) = \frac{\partial^2}{\partial x^2} \psi(x) \Big|_{x=x_i} = \frac{\psi(x_{i+1}) - 2\psi(x_i) + \psi(x_{i-1})}{\Delta x^2}$$

This leads to a matrix representation of the kinetic operator acting on the lattice wavefunction:

$$\hat{T} = -\frac{\hbar^2}{2m\Delta x^2} \begin{pmatrix} -3 & 1 & 0 & \cdots & 0 & \cdots & 0 \\ 1 & -2 & 1 & 0 & \cdots & \cdots & 0 \\ 0 & 1 & -2 & 1 & 0 & \cdots & 0 \\ \vdots & 0 & 1 & \ddots & \ddots & 1 & 0 \\ 0 & \vdots & 0 & 1 & -2 & 1 & 0 \\ \vdots & 0 & \vdots & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -3 \end{pmatrix} \tag{6.1}$$

The values $-3$ at the boundaries represent corrections due to Dirichlet boundary conditions, which impose $\psi = 0$ at the edges of the box.

Finally, the local kinetic energy at a given configuration $\mathbf{x}$ is computed as:

$$T_{loc}(\mathbf{x}) = \sum_{\mathbf{x}'} T_{\mathbf{x}\mathbf{x}'} \frac{\psi(\mathbf{x}')}{\psi(\mathbf{x})} \tag{6.2}$$

### 6.2.2 Coulomb interaction

The Coulomb potential can be implemented analogously to the kinetic operator. Considering first the nucleus–electron attraction, in Hartree atomic units one has:

$$V_{ne}(\vec{r}) = -\frac{Z}{|\vec{r}|}$$

This operator is **local** in the coordinate basis, meaning it acts by pointwise multiplication with the wavefunction:

$$(\hat{V}_{ne}\psi)(\vec{r}) = V_{ne}(\vec{r})\,\psi(\vec{r})$$

In other words, the matrix representation of the Coulomb attraction is diagonal in this basis, and the attractive electromagnetic contribution to the energy can be evaluated as:

$$E_{loc}^{ne} = -\sum_{i} \frac{Z}{|\vec{r}_i|}$$

A similar structure holds for the repulsive electron–electron interaction $V_{ee}$, which depends on the pairwise distances between particles:

$$E_{loc}^{ee} = \sum_{i<j} \frac{1}{|\vec{r}_i - \vec{r}_j|}$$

Therefore, the total Coulomb (electromagnetic) energy contribution is given by:

$$V(\vec{r}) = V_{ne}(\vec{r}) + V_{ee}(\vec{r}) \tag{6.3}$$

so that the local energy contribution is:

$$E_{loc}^{EM} = E_{loc}^{ne} + E_{loc}^{ee} \tag{6.4}$$

## 6.3 Deuteron wavefunction

A more complex context in which NQS have been applied in this work is the study of the deuteron wavefunction. The deuteron is a bound state of a proton and a neutron, defined by the total spin $S = 1$ and total isospin $T = 0$. To fully describe this system, the extra isospin quantum number is needed, so that the visible layer of the RBM used here is of size $4n_v$, which correspond to the 4 **spin- isospin canals**, which are organized as follows:

$$[p\uparrow\,|p\downarrow\,|n\uparrow\,|n\downarrow] \tag{6.5}$$

where $p$ denotes the $T_z = +\frac{1}{2}$ state, corresponding to the proton, $n$ the $T_z = -\frac{1}{2}$ state corresponding to the neutron, and $\uparrow$ / $\downarrow$ denote spin up and down, respectively.

The pauli principle is again automatically satisfied, since the same considerations done for atoms hold also when adding the isospin quantum number.

The sampling of configuration is carried out again with the Metropolis algorithm, although with some modifications with respect to the one implied with atoms.
The center of mass of the system is fixed at the center of the box. This constraint does not affect the physical properties of the system, since it corresponds to a symmetry of the deuteron, which is described by the Hamiltonian defined in Eq. (2.9) in the case $A = 2$. In this case the Hamiltonian can be written as

$$H = T + V(|\mathbf{r}_i - \mathbf{r}_j|),$$

which is translationally invariant, so that the motion of the center of mass can be separated from the relative motion of the two nucleons and is governed by a free particle kinetic term:

$$H \;=\; \frac{\mathbf{P}^2_{\mathrm{CM}}}{2M} \;+\; \left[\frac{\mathbf{p}_r^2}{2\mu} \;+\; \sum_p v_p(r) O_p\right]$$

Fixing the center of mass is therefore equivalent to choosing a reference frame in which the nucleons are at rest, and that does not affect in any way the physical properties of the deuteron.

To practically implement this constraint, the nucleons are initialized in a specular way with respect to the center of the box. The condition that the coordinates of the nucleons have to satisfy for this to happen is that, on each axis, one has:

$$x_p + x_n = L - 1$$

The COM is maintained fixed during all the sampling by making the Metropolis proposal **specular** to the center of the box. The proposal procedure works as follows:

**Algorithm 2** Metropolis proposal with COM symmetry preservation

1: Choose randomly one of the two particles, denoted as $i$
2: Identify the available nearest–neighbour sites of particle $i$
3: Propose and store a move of $i$ to one of the free neighbouring sites
4: For the other particle $j$, propose and store a move in the direction

$$\Delta\mathbf{r}_j = -\,\Delta\mathbf{r}_i,$$

so that the center of mass remains fixed at the box center
5: Check whether the proposed moves would lead to overlapping configurations (i.e. both particles ending up in the same site, which can only occur if the moves meet at the box center)
6: If no overlap occurs, compute the Metropolis acceptance probability and apply the move if accepted

Moreover, since the nuclear interaction in general does not conserve the spin, during the sampling the value of a spin of one randomly chosen nucleon is flipped with probability $p_s$.

Using the lattice definition (6.5), this means that a random particle is shifted by $+n_v$ if it is originally spin up, or by $-n_v$ otherwise.
In a similar manner, both isospin values are flipped with probability $p_t$. This means applying a shift of $+n_v$ on the lattice for the proton and one of $-n_v$ for the neutron. Given all of this, the overall Metropolis algorithm works as follows:

**Algorithm 3** Metropolis sampling for the deuteron system with COM symmetry

**Initialization:**
  Place the two nucleons $(p, n)$ such that $\vec{r}_p + \vec{r}_n = L - 1$
  Assign initial spin and isospin values according to Eq. (6.5)
**for** $i = 1$ to $N_{\text{sweeps}}$ **do**
  Draw a random number $r \in [0, 1)$
  **if** $r < p_m$ **then** ▷ spatial move
    Choose randomly one of the two nucleons $i$
    Identify its available nearest neighbour sites
    Propose a move $\Delta\vec{r}_i$ to one of them
    Propose a specular move $\Delta\vec{r}_j = -\Delta\vec{r}_i$ for the other nucleon
    Check for overlap (both nucleons on same site)
  **else if** $r < p_m + p_s$ **then** ▷ spin flip
    Choose randomly one nucleon and flip its spin by shifting of $\pm n_v$ on the lattice
  **else if** $r < p_m + p_s + p_t$ **then** ▷ isospin flip
    Choose randomly one nucleon and flip its isospin: by shifting of $\pm 2n_v$ on the lattice)
  **end if**
  Compute Metropolis acceptance probability:

$$A = \min\left(1, \frac{|\Psi(\boldsymbol{\sigma}')|^2}{|\Psi(\boldsymbol{\sigma})|^2}\right)$$

  Accept the proposed move with probability $A$
**end for**

### 6.3.1 ArgonneV6 potential

To fully describe the deuteron, its energy evaluation is required. The calculation of the kinetic energy local contribution is carried out as in the atomic case (see Sec. 6.2.1), whereas the ArgonneV6 potential (2.9) must be implemented explicitly. The main difficulty in implementing this potential is that it is **non-local**, which means that its contribution cannot be computed through pointwise multiplication with the wavefunction, as in the case of the Coulomb interaction. As a result, the matrix representing this potential is generally non-diagonal in the computational basis.

The local energy contribution must be evaluated as in (3.5), and the non-locality of the potential requires computing the contribution of each spin/isospin configuration $(s, t)$ at a given interparticle distance $r$.

The key idea is that, starting from a given configuration $(\vec{r}, s_i, t_i, s_j, t_j)$, the local energy

is computed as a sum over all possible spin/isospin final states. defining $\mathbf{\Gamma} = (s_i, t_i, s_j, t_j)$:

$$E_{\text{loc}}(r, \mathbf{\Gamma}) = \sum_{s_i', s_j', t_i', t_j'} \left[ \sum_{p=1}^{6} v^{(p)}(r) \left\langle s_i s_j t_i t_j \middle| O^{(p)}(r, \mathbf{\Gamma}) \middle| s_i' s_j' t_i' t_j' \right\rangle \right] \cdot \frac{\Psi(\vec{r}, s_i', s_j', t_i', t_j')}{\Psi(\vec{r}, s_i, s_j, t_i, t_j)} \tag{6.6}$$

Since $s_i, s_j, t_i, t_j \in \left\{-\frac{1}{2}, \frac{1}{2}\right\}$, each Monte Carlo evaluation involves a sum over 16 spin/isospin combinations, making the sampling computationally more challenging than in the atomic case.

The $O^{(p)}$ operators are defined in (2.8) and can be efficiently implemented as precomputed matrices. To this end, it is useful to introduce a compact basis for the two particle spin/isospin states. A two particle spin state can be mapped to a single index using:

$$\texttt{MatrixIndex}(a, b) = 2 \cdot \delta_{a,1} + \delta_{b,1}$$

so that the mapping is:

$$\begin{aligned} |\downarrow\downarrow\rangle &\to 0 \\ |\downarrow\uparrow\rangle &\to 1 \\ |\uparrow\downarrow\rangle &\to 2 \\ |\uparrow\uparrow\rangle &\to 3 \end{aligned} \tag{6.7}$$

Once this projection is established, operator contributions in Eq. (6.6) can be computed by reading the appropriate $(i, j)$-th matrix element.

The Pauli matrices are defined as in (2.1), and the first operators are straightforward. The identity $O^{(1)} = \mathbb{I}$ is trivial, while $O^{(2)} = \vec{\sigma}_1 \cdot \vec{\sigma}_2$ can be computed noting that each spin operator $\vec{\sigma}_i$ acts only on the $i$-th particle's subspace:

$$\vec{\sigma}_1 \cdot \vec{\sigma}_2 = \sum_{\alpha=x,y,z} (\sigma^\alpha \otimes \mathbb{I})(\mathbb{I} \otimes \sigma^\alpha) = \sum_{\alpha=x,y,z} \sigma^\alpha \otimes \sigma^\alpha \tag{6.8}$$

Substituting the Pauli matrices into this expression yields the final form:

$$O^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 2 & 0 \\ 0 & 2 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{6.9}$$

The same argument holds for the isospin operator $O^{(3)} = \vec{\tau}_1 \cdot \vec{\tau}_2$, whose matrix has the same structure as $O^{(2)}$.

The fourth operator is defined as:

$$O^{(4)} = (\vec{\sigma}_1 \cdot \vec{\sigma}_2)(\vec{\tau}_1 \cdot \vec{\tau}_2)$$

Since spin and isospin operators act on separate Hilbert spaces, this operator factorizes as:

$$\left\langle s_i s_j t_i t_j \middle| O^{(4)} \middle| s_i' s_j' t_i' t_j' \right\rangle = \left\langle s_i s_j \middle| O^{(2)} \middle| s_i' s_j' \right\rangle \cdot \left\langle t_i t_j \middle| O^{(3)} \middle| t_i' t_j' \right\rangle = \langle i | O^{(2)} | j \rangle \cdot \langle k | O^{(3)} | l \rangle$$

where $i, j, k, l$ label the spin/isospin configurations using the indexing in Eq. (6.7).

The most subtle term is the tensor operator $S_{12}$, which depends explicitly on the direction $\hat{r}$ between the particles:

$$S_{12} = 3(\vec{\sigma}_1 \cdot \hat{r})(\vec{\sigma}_2 \cdot \hat{r}) - \vec{\sigma}_1 \cdot \vec{\sigma}_2$$

Again each term acts on separate subspaces, so the expression can be expanded as:

$$(\vec{\sigma}_1 \cdot \hat{r})(\vec{\sigma}_2 \cdot \hat{r}) = \left( \sum_{\alpha=x,y,z} \hat{r}_\alpha \, \sigma^\alpha \otimes \mathbb{I} \right) \left( \sum_{\beta=x,y,z} \hat{r}_\beta \, \mathbb{I} \otimes \sigma^\beta \right) = \sum_{\alpha,\beta=x,y,z} \hat{r}_\alpha \hat{r}_\beta \, \sigma^\alpha \otimes \sigma^\beta$$

and $\vec{\sigma}_1 \cdot \vec{\sigma}_2$ is given by (6.8).

Putting everything together leads to:

$$S_{12} = \sum_{\alpha,\beta=x,y,z} (3\hat{r}_\alpha \hat{r}_\beta - \delta_{\alpha\beta}) \cdot \sigma^\alpha \otimes \sigma^\beta$$

Thus, for each fixed $\hat{r}$, one can construct a $4 \times 4$ matrix in the computational basis $|s_i, s_j\rangle \to |i\rangle = \texttt{MatrixIndex}(s_i, s_j)$. The general form of this matrix is:

$$S_{12} = \begin{pmatrix} T_{zz} & T_{zx} - iT_{zy} & T_{xz} - iT_{yz} & T_{xx} - T_{yy} - 2iT_{xy} \\ T_{zx} + iT_{zy} & -T_{zz} & 2(T_{xx} + T_{yy}) & T_{xz} - iT_{yz} \\ T_{xz} + iT_{yz} & 2(T_{xx} + T_{yy}) & -T_{zz} & T_{zx} - iT_{zy} \\ T_{xx} - T_{yy} + 2iT_{xy} & T_{xz} + iT_{yz} & T_{zx} + iT_{zy} & T_{zz} \end{pmatrix}$$

where:

$$T_{\alpha\beta} = 3\hat{r}_\alpha \hat{r}_\beta - \delta_{\alpha\beta}$$

Finally, the calculation of $O^{(6)}$ contribution to local energy can be evaluated as the $O^{(4)}$ one, since same again the spin and isospin terms are separable:

$$\left\langle s_i s_j t_i t_j \middle| O^{(6)} \middle| s'_i s'_j t'_i t'_j \right\rangle = \left\langle s_i s_j \middle| O^{(5)} \middle| s'_i s'_j \right\rangle \cdot \left\langle t_i t_j \middle| O^{(3)} \middle| t'_i t'_j \right\rangle = \langle i | O^{(5)} | j \rangle \cdot \langle k | O^{(3)} | l \rangle$$

The radial operators $v_p(r)$ have been implemented by translating the original Fortran code made by the Argonne group, which defines them as properly weighted combinations of the terms:

$$\text{Yukawa central average:} \quad \overline{y}_\pi(r) = \frac{e^{-x}}{x} \left(1 - e^{-c_\pi r^2}\right)$$

$$\text{Yukawa tensor average:} \quad \bar{t}_\pi(r) = \left(1 + \frac{3}{x} + \frac{3}{x^2}\right) \cdot \frac{e^{-x}}{x} \left(1 - e^{-c_\pi r^2}\right)$$

$$ws(r) = \left(1 + e^{(r - r_{\text{ws}}) \cdot a_{\text{iws}}}\right)^{-1}$$

$$ws_0 = \left(1 + e^{-r_{\text{ws}} \cdot a_{\text{iws}}}\right)^{-1}$$

$$wsp(r) = ws(r) \cdot \left(1 + a_{\text{iws}} \cdot e^{-r_{\text{ws}} \cdot a_{\text{iws}}} \cdot ws_0 \cdot r\right)$$

$$\text{Corrected pion terms:} \quad \begin{aligned} y'_{\pi^0} &= y_{\pi^0} - f^2 \cdot d_{\pi^0} \cdot \frac{ws(r) \cdot r}{ws_0} \\ y'_{\pi^\pm} &= y_{\pi^\pm} - f^2 \cdot d_{\pi^\pm} \cdot \frac{ws(r) \cdot r}{ws_0} \end{aligned}$$

$$\text{tensor term:} \quad t_\pi^2(r) = \left[\bar{t}_\pi(r)\right]^2$$

The radial functions $V^{(p)}(r)$ are defined as combinations of partial potentials $p_{ST}^{xy}$ corresponding to different spin-isospin channels, labeled as $p_{ST}$, where $S$ is the total spin and $T$ the total isospin of the nucleon pair.

These are obtained as linear combinations of more fundamental interaction channels, noted as $p_{ST}^{xy}$, where $x, y \in \{p, n\}$ denote proton or neutron configurations. Each depends on the interparticle distance $r$, the Yukawa functions, and Woods-Saxon cutoff corrections. The full expressions of the partial potentials extracted from the code implementation are:

$$\begin{aligned}
p_{11}^{pp} &= -7.62701 \cdot t_\pi^2 + 1815.4920 \cdot wsp(r) + 1847.8059 \cdot (ws(r) \cdot x)^2 + y'_{\pi^0} \\
p_{11}^{np} &= -7.62701 \cdot t_\pi^2 + 1813.5315 \cdot wsp(r) + 1847.8059 \cdot (ws(r) \cdot x)^2 - y'_{\pi^0} + 2y'_{\pi^\pm} \\
p_{11}^{nn} &= -7.62701 \cdot t_\pi^2 + 1811.5710 \cdot wsp(r) + 1847.8059 \cdot (ws(r) \cdot x)^2 + y'_{\pi^0}
\end{aligned}$$

$$\begin{aligned}
p_{t1}^{pp} &= 1.07985 \cdot t_\pi^2 - 190.0949 \cdot ws(r) \cdot x - 811.2040 \cdot (ws(r) \cdot x)^2 + t_{\pi^0} \\
p_{t1}^{np} &= 1.07985 \cdot t_\pi^2 - 190.0949 \cdot ws(r) \cdot x - 811.2040 \cdot (ws(r) \cdot x)^2 - t_{\pi^0} + 2t_{\pi^\pm} \\
p_{t1}^{nn} &= 1.07985 \cdot t_\pi^2 - 190.0949 \cdot ws(r) \cdot x - 811.2040 \cdot (ws(r) \cdot x)^2 + t_{\pi^0}
\end{aligned}$$

$$\begin{aligned}
p_{10} &= -8.62770 \cdot t_\pi^2 + 2605.2682 \cdot wsp(r) + 441.9733 \cdot (ws(r) \cdot x)^2 - y'_{\pi^0} - 2y'_{\pi^\pm} \\
p_{t0} &= 1.485601 \cdot t_\pi^2 - 1126.8359 \cdot ws(r) \cdot x + 370.1324 \cdot (ws(r) \cdot x)^2 - t_{\pi^0} - 2t_{\pi^\pm}
\end{aligned}$$

$$\begin{aligned}
p_{01}^{pp} &= -11.27028 \cdot t_\pi^2 + 3346.6874 \cdot wsp(r) - 3y'_{\pi^0} \\
p_{01}^{np} &= -10.66788 \cdot t_\pi^2 + 3126.5542 \cdot wsp(r) - 3(-y'_{\pi^0} + 2y'_{\pi^\pm}) \\
p_{01}^{nn} &= -11.27028 \cdot t_\pi^2 + 3342.7664 \cdot wsp(r) - 3y'_{\pi^0} \\
p_{00} &= -2.09971 \cdot t_\pi^2 + 1204.4301 \cdot wsp(r) - 3(-y'_{\pi^0} - 2y'_{\pi^\pm})
\end{aligned}$$

The code then defines partial potentials by averaging over isospin states:

$$\begin{aligned}
p_{11} &= \frac{1}{3}(p_{11}^{pp} + p_{11}^{nn} + p_{11}^{np}) \\
p_{01} &= \frac{1}{3}(p_{01}^{pp} + p_{01}^{nn} + p_{01}^{np}) \\
p_{10} &= p_{10} - 0.3 \cdot p_{LS0}
\end{aligned}$$

The singlet-singlet channel $p_{00}$ is defined directly in the code. $p_{t1}$ and $p_{t0}$ represent the triplet and singlet tensor channels, obtained similarly by averaging over different isospin configurations.

Finally, the six radial functions used in the Argonne $v_6$ model are obtained as:

$$\begin{aligned}
V^{(1)}(r) &= \tfrac{1}{16}\,(9\,p_{11} + 3\,p_{10} + 3\,p_{01} + p_{00}) \\
V^{(2)}(r) &= \tfrac{1}{16}\,(3\,p_{11} - 3\,p_{10} + p_{01} - p_{00}) \\
V^{(3)}(r) &= \tfrac{1}{16}\,(3\,p_{11} + p_{10} - 3\,p_{01} - p_{00}) \\
V^{(4)}(r) &= \tfrac{1}{16}\,(p_{11} - p_{10} - p_{01} + p_{00}) \\
V^{(5)}(r) &= \tfrac{1}{4}\,(3\,p_{t1} + p_{t0}) \\
V^{(6)}(r) &= \tfrac{1}{4}\,(p_{t1} - p_{t0})
\end{aligned}$$

## 6.4 Complete algorithm

Now that all the tools have been discussed, it is possible to introduce the full algorithm powering NQS in both atoms and deuteron scenarios. The differences between these two cases rely only in the energy calculation and in the metropolis proposal, while the optimization scheme remains the same. The following algorithm has been implemented in C++, building on Redaelli, Lazzarino and Paravizzini's work [12, 11, 1]. The full explanation of every component of the code is available in the appendix.

**Algorithm 4** NQS complete training algorithm

Initialize lattice and particles
**for** $i = 1$ to $N_{\text{iter}}$ **do**
    Equilibrate the Metropolis sampler by performing $N_{\text{sweeps}} \times T_f$ sweeps, where $T_f$ is the thermalization factor
    **for** $j = 1$ to $N_{\text{sweeps}}$ **do**
        Propose a Metropolis move
        Accept the move with probability

$$A = \min\left(1, \frac{|\Psi(\boldsymbol{\sigma}')|^2}{|\Psi(\boldsymbol{\sigma})|^2}\right)$$

        **if** $r \in [0, 1) \leq A$ **then**
            Update configuration: $\boldsymbol{\sigma} \leftarrow \boldsymbol{\sigma}'$
        **end if**
        Compute local energy $E_{\text{loc}}$ of the current configuration
        Compute log-derivatives:

$$O_{\boldsymbol{\theta}} = D_{\boldsymbol{\theta}} \log \Psi_{\boldsymbol{\theta}}(\boldsymbol{\sigma})$$

    **end for**
    Collect $E_{loc}$ for all the threads
    calculate $\langle E_{loc} \rangle$ and errors over the threads
    Compute gradient estimator:

$$\frac{\partial E}{\partial \theta_k} = 2\,\mathrm{Re}\left[\langle O_k^* E_{\text{loc}} \rangle - \langle O_k^* \rangle \langle E_{\text{loc}} \rangle\right]$$

    Update parameters via SGD:

$$\theta_k \leftarrow \theta_k - \eta \cdot \frac{\partial E}{\partial \theta_k}$$

    Update learning rate:

$$\eta(i) = \eta_0 \left(\frac{1}{1 + \frac{i}{\eta_1}}\right)^p$$

**end for**

# Chapter 7

# Results

This chapter reports the results from RBM–NQS simulations, based on sections 5.2 and 6, to atoms and deuteron.

For atomic systems, all simulations are conducted within a spatial domain whose length is a multiple of the Bohr radius, $a_0 = \hbar/(m_e c\alpha)$, where $\alpha$ is the fine structure constant, $c$ is the speed of light, and $m_e$ is the electron mass. These physical scales are widely adopted in atomic and molecular physics, as they naturally accommodate the predominant influence of Coulomb interactions. The Hartree, defined as $m_e c^2 \alpha^2$, serves as the fundamental unit of energy throughout our simulations.

Since the normalization constant is not required for the application of the Metropolis algorithm, wavefunctions and their moduli are presented in arbitrary units. For consistency and clarity, all wavefunctions are scaled with a numerical approximations of the respective normalization constant.

## 7.1 Atoms

This section explores the results on three atomic systems:

- Helium ($Z = 2$) with parallel spins
- Helium with antiparallel spins
- Lithium ($Z = 3$)

the results presented include spatial distribution plots as well as plots of the wavefunctions significative sections. Before mentioning the best result, some study on the RBM has been made to find the best learning rate update schedule and the best hidden layer size.

### 7.1.1 learning rate study

#### power law decay study

The effect of the exponent $p$ on the learning rate decay

$$\eta(i) = \eta_0 \left( \frac{1}{1 + \frac{i}{\eta_1}} \right)^p \tag{7.1}$$

has been studied in the cases

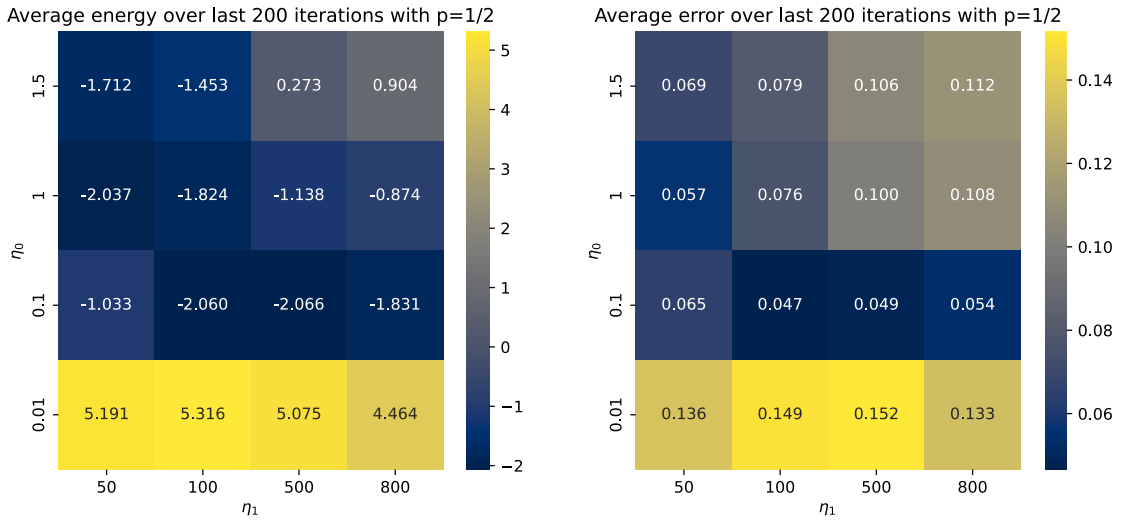
$$p \in \{\frac{1}{2}, 1, 2\}$$

The study has been made by training an RBM with $n_v = 1331$ ($L = 11$) and $n_h = 5$ on Helium with both spins up.

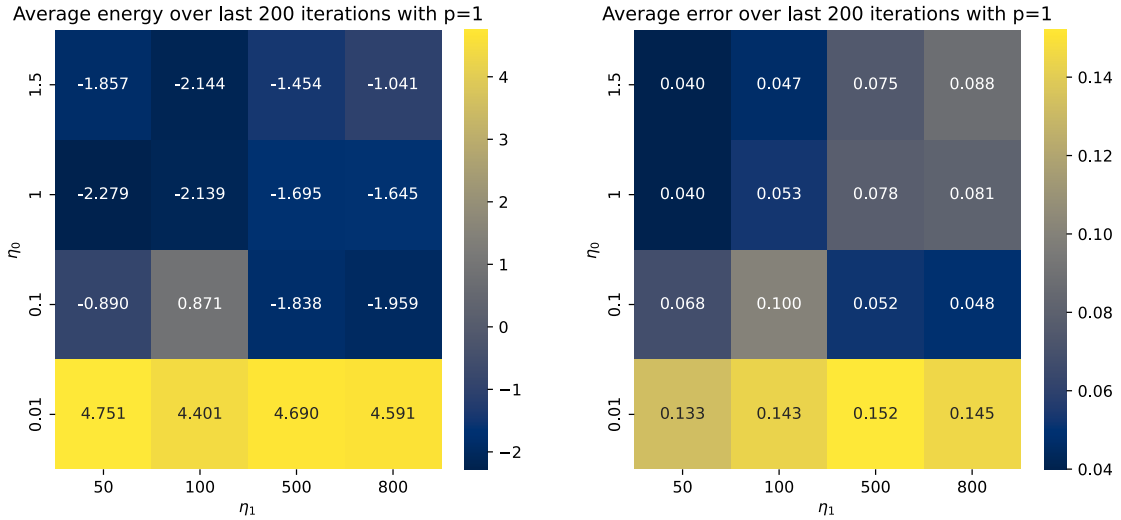
The same training, starting from Gaussian initialized weights and carried out for $n_{iter} = 6000$, has been performed by using the hyperparameters:

$$\eta_0 \in \{1.5, 1, 0.1, 0.01\}, \qquad \eta_1 \in \{50, 100, 500, 800\}$$

The heatmaps of Fig. 7.1 show the result for each case, by marking the average energy and errors over the last 200 iterations.

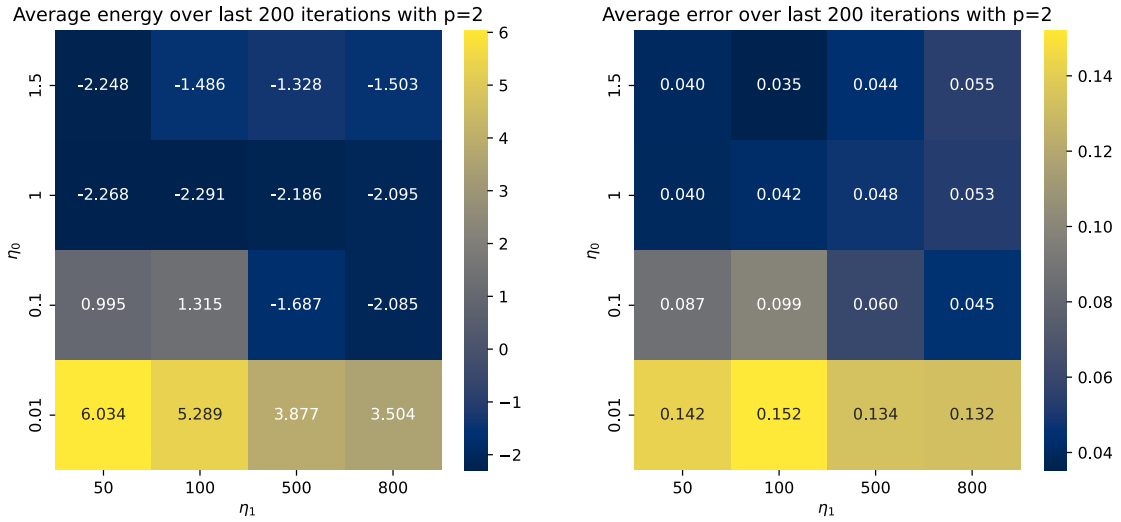



(a) $p = 0.5$




(b) $p = 1$

Figure 7.1: Heatmap energy/error for $n_h = 5$ with $p = 0.5$ and $p = 1$




(c) $p = 2$

Figure 7.2: Heatmap energy/error for $n_h = 5$ with $p = 2$

Recalling the variational principle (3.1.2), we identify the best case scenario with the lowest energy one, because it is the closest to the actual ground state. On the other hand, the zero variance property (3.8) allows to identify the state with the lowest error as an eigenstate of the Hamiltonian of the problem. It follows immediately that lowering the energy must mean lowering the error, as the network approaches the true ground state of the problem. Thus, for $p = 1$ the case of $(\eta_0, \eta_1) = (1, 50)$ is identified as the best, for $p = 2$ the case of $(\eta_0, \eta_1) = (1, 100)$, while the result obtained with $p = \frac{1}{2}$ can be considered of worse accuracy compared to more aggressive power laws.

This suggests that the best cooling strategy in the simulated annealing interpretation implies starting with a "hot" system ($\eta_0 = 1$), and "cooling down" in a relatively fast way: $\eta_1 = 50/100$ means that the denominator in (7.1) increments by one unit every $50/100$ iterations. This argument can also be considered when observing the fact that a more aggressive cooling schedule obtained with $p = 2$ leads to even better results than the lighter $p = 1$ case. The reason is that the RBM is allowed to explore the parameter space broadly during the early stages of training, and then to concentrate in effective regions around the minima in a fast manner, which leads to a greater lowering of energy in the long run.

Moreover, by looking at the $\eta_0 = 0.1, 0.01$ rows, where the system already starts with a low learning rate compared with the other cases, one can observe that starting at low temperature and decreasing it slowly leads also to good approximations, as confirmed to the lower energy for progressively higher values of $\eta_1$ in the same row. On the other hand, cooling down aggressively results in a complete early annealing of the RBM, which remains in high energy regions for the whole training.

The most extreme case is the last row ($\eta_0 = 0.01$), where the temperature is already too low at the start, and the parameters space cannot be explored sufficiently, even if the cooling down is the slowest.

Figures 7.3 and 7.4 show the results of the same study for $n_h = 3, 7$ in the best cases of $p = 1, 2$.

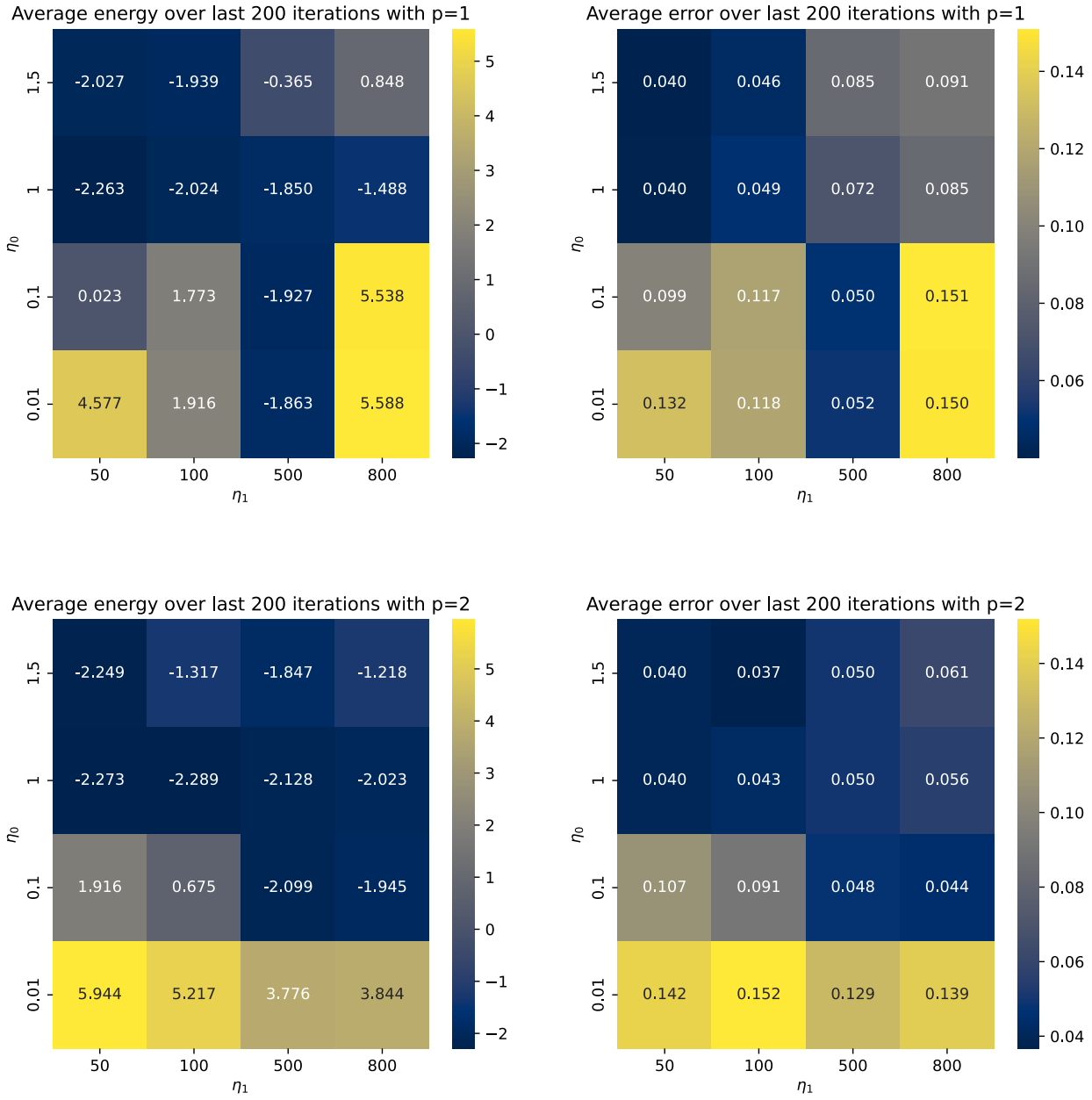



Figure 7.3: Heatmap energy/error for $n_h = 7$ with different learning rate decay power laws $p$

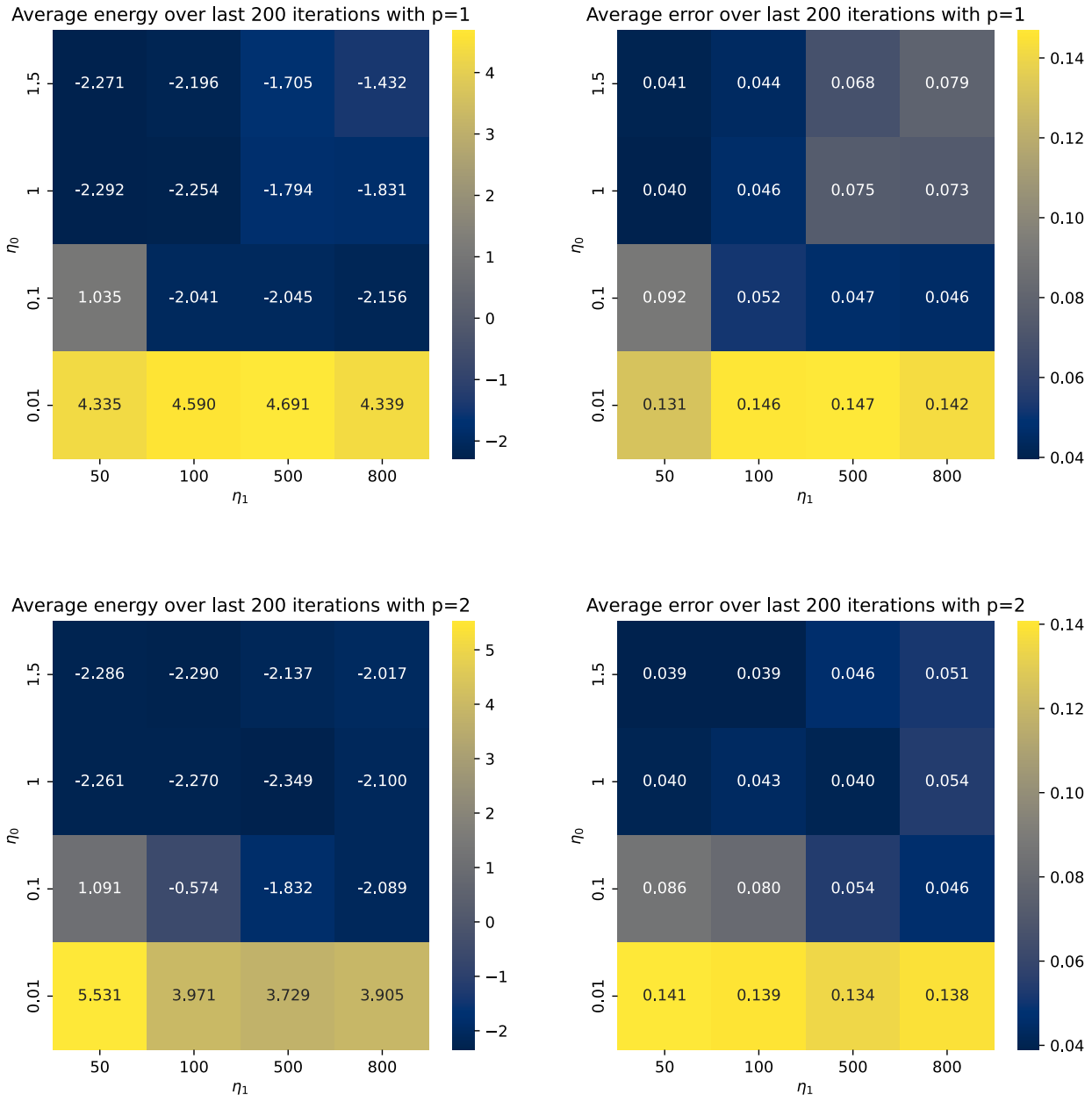



Figure 7.4: Heatmap energy/error for $n_h = 3$ with different learning rate decay power laws $p$

Here the same considerations done for the case of $n_h = 5$ hold. From now on, all the presented results are obtained using this cooling schedule: The RBM is initially allowed to explore the parameter space broadly by starting with $\eta_0 = 1$, and the learning rate then decreases in a progressive yet relatively fast manner (following a power law decay with $p = 2$ and $\eta_1 = 100$). This drives the optimization towards low energy regions at an early stage, while still retaining enough flexibility to explore the neighborhood of the minimum effectively.

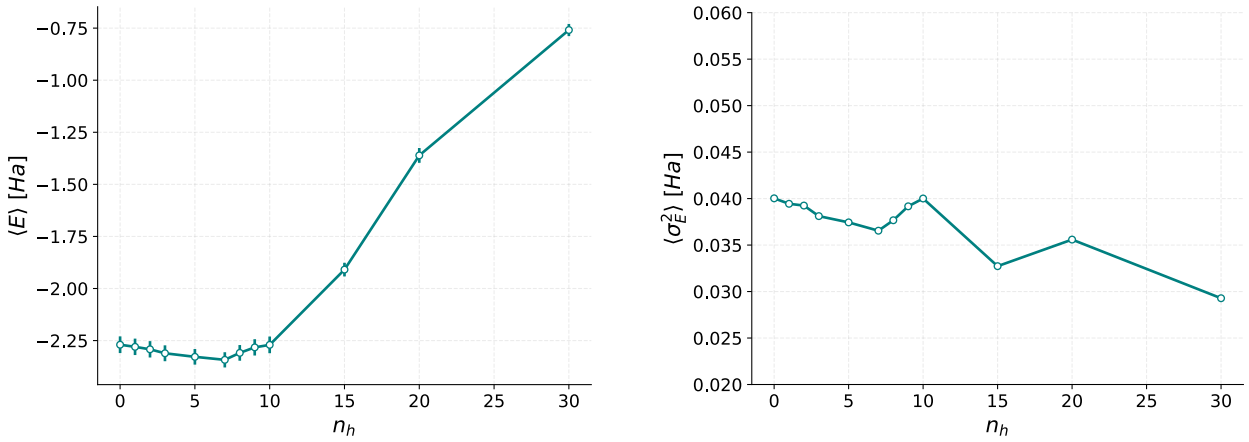
### Hidden nodes study

For Helium with both spins up, a study of the network's expressiveness has been conducted by training an RBM with $n_v = 1331$ with different numbers of hidden units in:

$$n_h \in [0, 30]$$

the objective of this section is to prove that RBM expressiveness increments with the size of its hidden layer, if it hasn't already learned the wavefunction perfectly.

The plot below shows the average energy over the last 200 iterations for different hidden layers sizes, along with the variances reached.




(a) Average energy during last 200 training iterations for helium with parallel spin, with different values of $n_h$.

(b) Average variance on energy during last 200 training iterations for helium with parallel spin, with different values of $n_h$.

Average energy and error during the last 200 training iterations for different $n_h$.

Starting from $n_h = 0$ and adding the new hidden nodes *adiabatically*, by taking the weights of the previously trained network and adding a single node at a time, it is possible to reduce the final energy in an initial range of $n_h$. However, as the number of hidden nodes grows, the training becomes progressively more unstable and the fluctuations around the variational minimum are amplified. The total number of variational parameters is

$$n_{\text{par}} = 2n_v + n_h + 2n_v n_h,$$

which grows linearly with $n_h$. As $n_h$ increases, the dimensionality of the parameter space rapidly becomes large, while a discretized box with $L = 11$ sites provides only a limited number of physically accessible electronic configurations. This enhances the statistical noise of the Monte Carlo estimates and makes the optimization landscape more difficult to navigate.

Consequently, for $n_h \gtrsim 9$ the RBM becomes effectively overparametrized for the present problem: the additional expressive capacity does not correspond to physically relevant degrees of freedom, and the optimization tends to converge to higher energy stationary points rather than improving the variational bound. Interestingly, increasing $n_h$ still results in a reduction of the energy variance. According to the zero variance property (3.8), a vanishing variance

signals convergence to an eigenstate of the Hamiltonian. In this case the variance remains finite but decreases, which suggests that the network might be approaching some excited state solutions (higher energy eigenstates or approximations), corresponding to local minima in parameter space that become easier to fall into as the number of parameters grows, if the training setup is fixed. This will be shown more clearly with spatial distribution plots in the next section.

From now on, all the result presented will rely on an RBM with 7 hidden units, since it is the case we found at lowest energy, considering the observations made above.

### 7.1.2 Spatial distribution plots

We perform a full training of the RBM on the Helium with two spins up over $n_{iter} = 2 \times 10^4$ iterations. The RBM trained uses $n_v = 1331, L = 11, \ell = 4a_0, \eta_0 = 1, \eta_1 = 100, p = 2$. The plots of the average energy and errors during training are shown in Fig. 7.6.

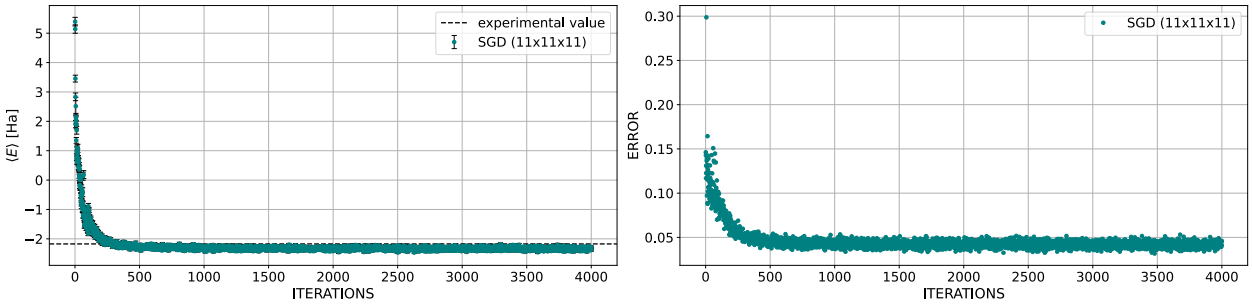



Figure 7.6: Energy and error during training (first 4000 iterations) for Helium atom with both spins up and $n_h = 7$

#### Radial density function

The **radial density** $\rho(r)$ is defined as the probability density per unit volume of finding an electron at distance $|r|$ from the nucleus (regardless of the direction):

$$\rho(r) = \frac{1}{4\pi r^2} \left\langle \sum_{i=1}^{N} \delta\left(r - |\mathbf{r}_i - \mathbf{R}_N|\right) \right\rangle = \frac{1}{4\pi r^2} \sum_{i=1}^{N} \int |\psi(\mathbf{r}_1, \ldots, \mathbf{r}_N)|^2 \, \delta\left(r - |\mathbf{r}_i - \mathbf{R}_N|\right) \, d\mathbf{r}_1 \ldots d\mathbf{r}_N$$

where $\mathbf{R}_N$ denotes the position of the nucleus, $\mathbf{r}_i$ the position of each electron and $\delta$ is the Dirac delta distribution.

For spherically symmetric systems such as atomic helium, the radial density encodes how tightly the electronic cloud is bound to the nucleus, providing a compact one dimensional representation of the fully three–dimensional probability distribution $|\psi(\mathbf{r})|^2$. By taking the trained RBM, a single iteration of $n_{sweeps} = 5 \times 10^5$ sweeps has been made to collect the same number of electron positions samples according to the Helium wavefunction distribution $|\psi(\sigma)|^2$. Since the nucleus is placed at the center of the box $(X_N, Y_N, Z_N) = (5, 5, 5)$, the discretized squared distance of any sampled lattice point from the nucleus is

$$n^2 = (n_x - X_N)^2 + (n_y - Y_N)^2 + (n_z - Z_N)^2,$$

where $n_x, n_y, n_z$ are the discrete lattice coordinates obtained with Eq.(5.4). Since $L = 11$, the nucleus sits in $(X_N, Y_N, Z_N) = (5, 5, 5)$ and $n_x, n_y, n_z \in \{0, \ldots, 10\}$. Because each displacement $(n_x - X_N)$ can take values in $\{-5, \ldots, 5\}$, the maximum accessible squared distance is

$$n^2_{\max} = 25 + 25 + 25 = 75.$$

Since the radial density does not depend on the angular variables, each shell at fixed $n^2$ must be normalized by its multiplicity, i.e. by the number of lattice points lying at that same distance. This corresponds to the $4\pi r^2$ factor in the discrete case. The multiplicity is computed by explicit enumeration:

**Algorithm 5** Computation of the discrete shell multiplicities $M(n^2)$

allocate array $M[0, \ldots, 75] \leftarrow 0$
**for** $n_x = 0$ to 10 **do**
  **for** $n_y = 0$ to 10 **do**
    **for** $n_z = 0$ to 10 **do**
      compute

$$n^2 = (n_x - X_N)^2 + (n_y - Y_N)^2 + (n_z - Z_N)^2$$

      $M[n^2] \leftarrow M[n^2] + 1$
    **end for**
  **end for**
**end for**

Finally, the radial density $\rho(r)$ is reconstructed by histograming the sampled distances up to the maximum squared displacement $n^2_{\max} = 75$, and normalizing each shell by its multiplicity, i.e.

$$\rho(n) \propto \frac{\text{counts}(n)}{N_{samples} M(n)},$$

which removes the discrete geometric factor associated with the number of lattice sites at fixed radius.

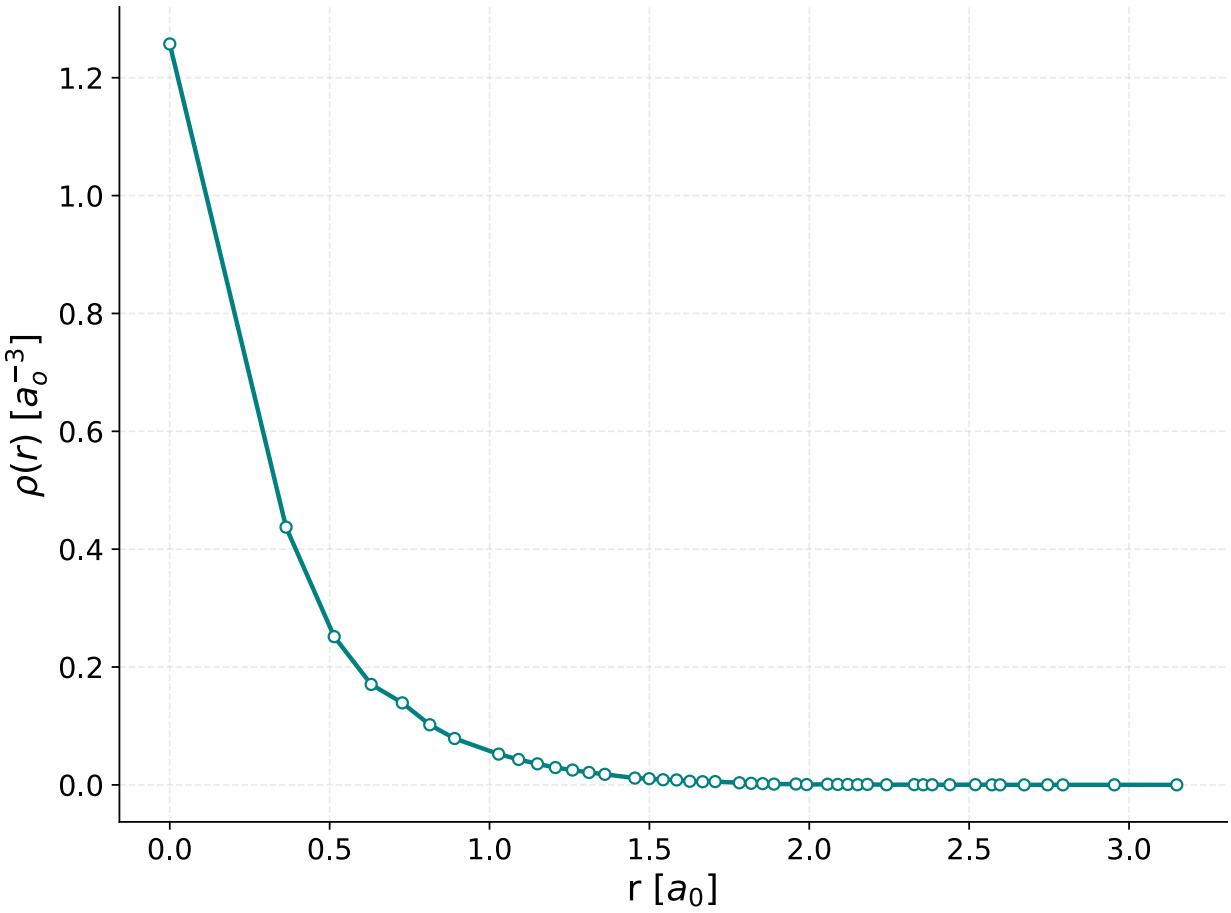



Figure 7.7: $\rho(r)$ for Helium atom with both spins up

The resulting curve shown in Fig. 7.7 follows a qualitative behavior of the form

$$\rho(r) \propto e^{-\alpha r}$$

This exponential decay reflects the behavior of the ground state wavefunction of the helium atom. Since the electrons are bound, the many–body wavefunction decays exponentially with distance from the nucleus, and this is directly visible in the radial density profile.

The pronounced peak around $r \to 0$ reflects the high probability of finding electrons near the nucleus, due to the attractive Coulomb potential. The rapid decay beyond $r \sim 1\, a_0$ indicates that electrons are strongly localized: they rarely possess enough energy to overcome the Coulomb attraction and populate regions far from the nucleus.

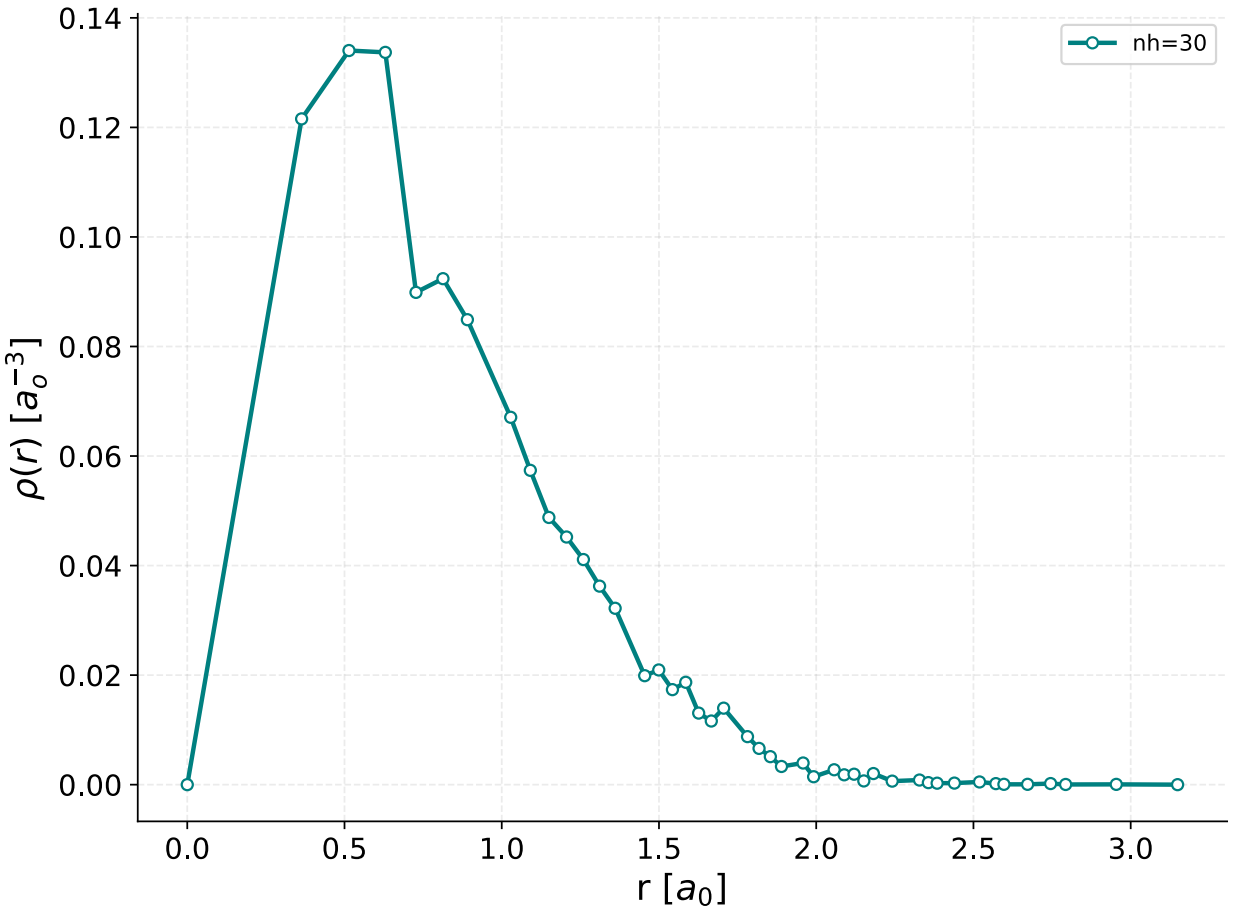



Figure 7.8: $\rho(r)$ for Helium atom with both spins up with $n_h = 30$

The case of the best solution we found for $n_h = 30$ is shows in Fig. 7.8. The electrons appear to be localized in a region around $\sim 0.5a_0$ far away from the nucleus, which can be interpreted as an higher energy orbital that comes from an excited state, as discussed in Sect. 7.1.1.

### Radial distribution function

The **radial distribution function** $P(r)$ is obtained in the same way as the radial density, but without dividing the histogram by the shell multiplicities, so that

$$P_2(r) = 4\pi r^2 \rho(r). \tag{7.2}$$

This function is normalized;

$$\int_0^\infty P(r)\, dr = 1,$$

and therefore represents the probability of finding an electron at a distance between $r$ and $r + dr$ from the nucleus.

The plot below shows the $P(r)$ obtained first by taking:

$$P_1(r) \propto \frac{counts}{N_{samples}}$$

e.g. in the same way as the $\rho(r)$, but without dividing for the multiplicity associated with each shell ($P_1$). Then, the same function has been calculated in a different way, by applying 7.2 on each discretized distance on the previously calculated $\rho(r)$ ($P_2$). The two plots in Fig. 7.9 represent the same quantity, but finite size effects are more evident in the first implementation, as one can see from the greater fluctuations.

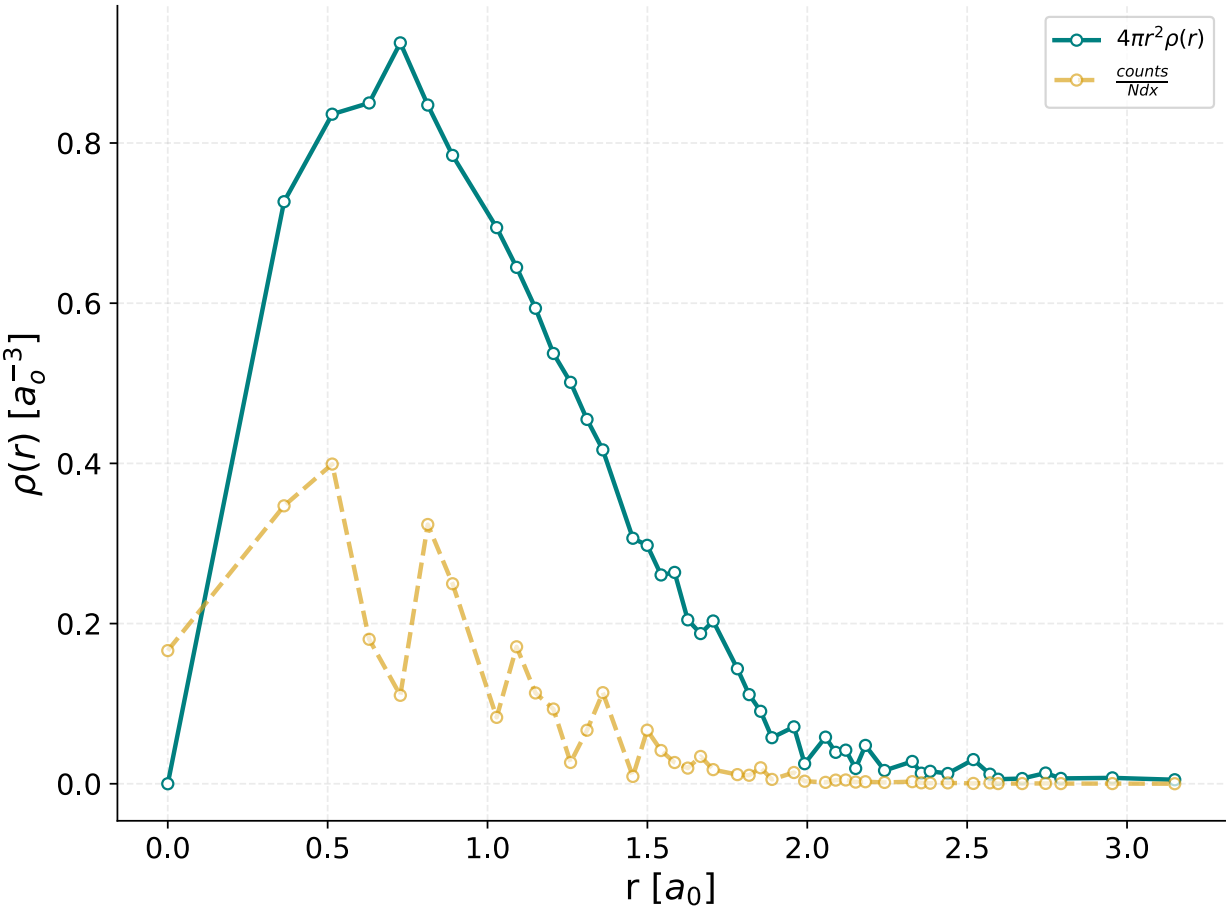



Figure 7.9: $P(r)$ for Helium atom with both spins up

The behavior near the nucleus shows that the radial probability $P(r)$ of finding an electron exactly at the nucleus vanishes. This is not a contradiction with the fact that the electron density $\rho(r)$ is maximal at $r = 0$. The point is that $\rho(r)$ is a density per unit volume, while $P(r)$ is the radial probability. Although the density is highest at the nucleus, the volume element $4\pi r^2\, dr$ shrinks to zero as $r \to 0$. As a consequence, the probability of finding an electron attached to the nucleus vanishes even if the local density is large. In the $P_1$ plot, the fact that $P(0) \neq 0$ is a finite size effect, because some of the sampled positions stay in the first bin $[0, dr]$, and thus the histogram $P_1(0)$ is non-null. This effect is eliminated in the

$P_2$ plot because of the multiplication for $r = 0$, giving a more physical feeling of the plotted quantity. The peak around $\sim 0.5/0.7a_0$ reflects the highest probability zone, where Coulomb attraction and repulsion are balanced. The exponential decay after $r \sim 1a_0$ reflects, again, the same behavior of the true ground state Helium wavefunction.

### Radial pair distribution function

Another important quantity is the **radial pair distribution function** $g(r)$, which quantifies the probability of finding two electrons at distance $r$. It can be defined as:

$$g(r) = \frac{1}{N\Delta V_n} \left\langle \delta\big(r - \left|\mathbf{r}_1 - \mathbf{r}_2\right|\big) \right\rangle ,$$

Where $N$ is the total number of samples, $r_1, r_2$ denote the lattice positions for the electrons and $\Delta V_n$ denotes the multiplicity of the $n$-th shell, which in the discrete case is calculated as in algorithm (5), but taking a maximum of 300 values, because now the coordinates are taken with respect to each electron, instead of the nucleus.

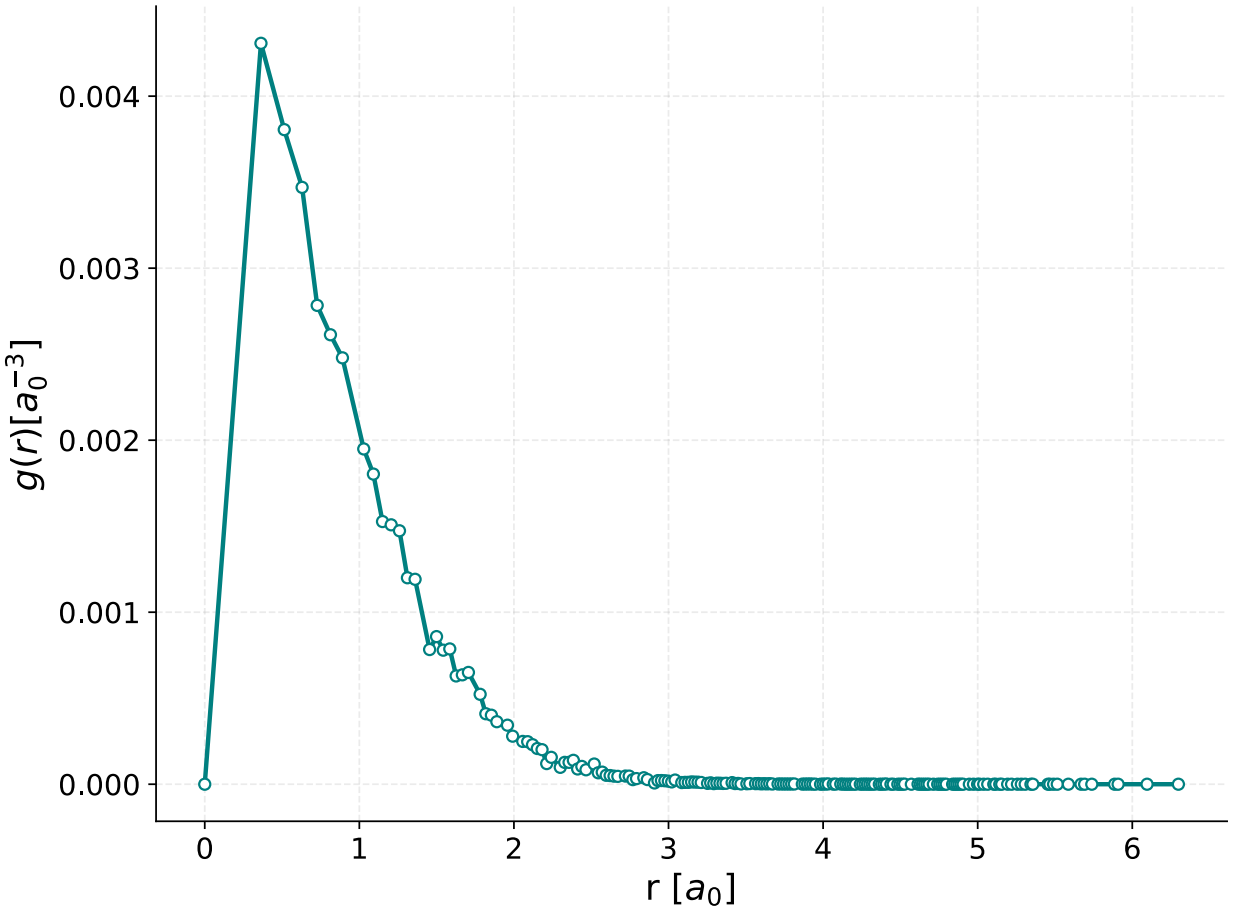



Figure 7.10: $g(r)$ for Helium atom with both spins up

As required by the Pauli principle, the function vanishes at short distances: two electrons with the same spin cannot occupy the same spatial point, so the probability of finding them at zero separation must be exactly zero, and this is reflected by $g(0) = 0$.

At intermediate distances, $g(r)$ develops a pronounced peak around $r \simeq 0.6$–$0.8\, a_0$, which corresponds to the most probable separation between the two electrons. This peak results from the competition between the attractive interaction with the nucleus, which tends to pull both electrons towards the atomic core, and the antisimmetry constraint imposed by their parallel spins, which prevents them from approaching each other too closely. The structure of the peak is therefore a direct manifestation of exchange effects and electron–electron correlation in the ground state.

For larger separations, $g(r)$ rapidly decays to zero: distances much greater than the typical size of the atom become increasingly unlikely, and in the present discretized lattice the maximal accessible separation is itself limited.

### 7.1.3 Helium with antiparallel spins

The singlet Helium systems (one spin up and one spin down) has been simulated to train an RBM with visible layer size $n_v = 1331$, $L = 11$, $\ell = 4fm$, $\eta_0 = 1, \eta_1 = 100, p = 2$ for different hidden layer sizes. Since an in–depth study has already been made, here we focus on fewer values of $n_h$, with the aim of finding the one that leads to the lowest energy, in order to use that RBM to produce spatial distribution plots. The results are shown in the plot below:

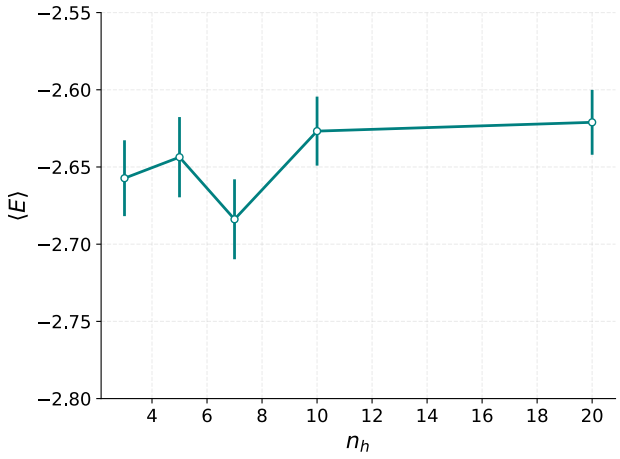



Figure 7.11: Final energies and error for RBM with different hidden layer sizes on helium with antiparallel spins

Again, the best values are obtained for $n_h = 7$, as for the case of parallel spins. This result is not surprising, since the system to be learned is closely similar to the one studied before, except for different electron-electron simmetry correlations. From now on, the plots will show results of an RBM with $n_h = 7$. The plot below shows the energy during training for the best case scenario, by marking the experimental value as a qualitative benchmark.

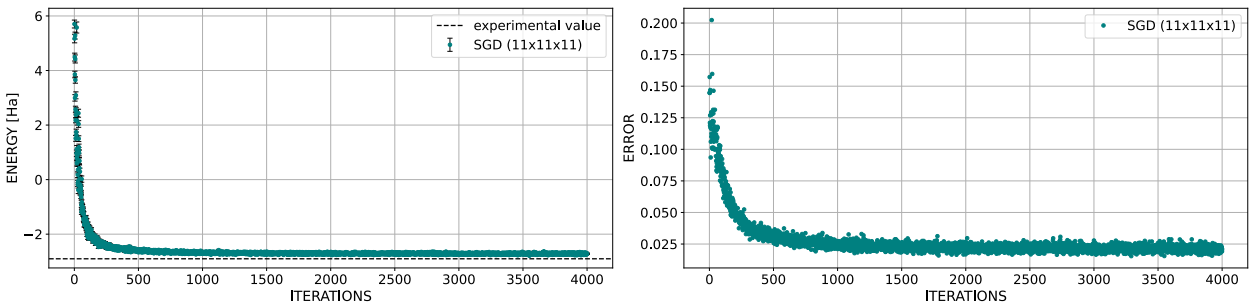



Figure 7.12: average energy and errors during training (first 4000 iterations) for RBM with $n_h = 7$ on helium with antiparallel spins

It should be emphasized that the experimental ground–state energy of helium cannot be used as the target value of the training. Because the system is represented in a finite and discretized box, the Hamiltonian actually solved by the RBM differs from the physical continuous one. As a consequence, its eigenvalues are not expected to coincide with the experimental energy, even if a proper tuning of physical length parameters $L, \ell, n_v$ can scale the values obtained to give a better approximation. For this reason, the experimental value is shown only as a qualitative reference for the energy scale, and not as the value that the optimization is supposed to reproduce.

The same spatial distribution plots $(\rho(r), P(r), g(r))$ calculated for Helium with two spin ups have been obtained also for the antiparallel case.

### Radial density function for both spins

The first plot shows the radial density $\rho(r)$ with respect to spin up and spin down electrons separately:

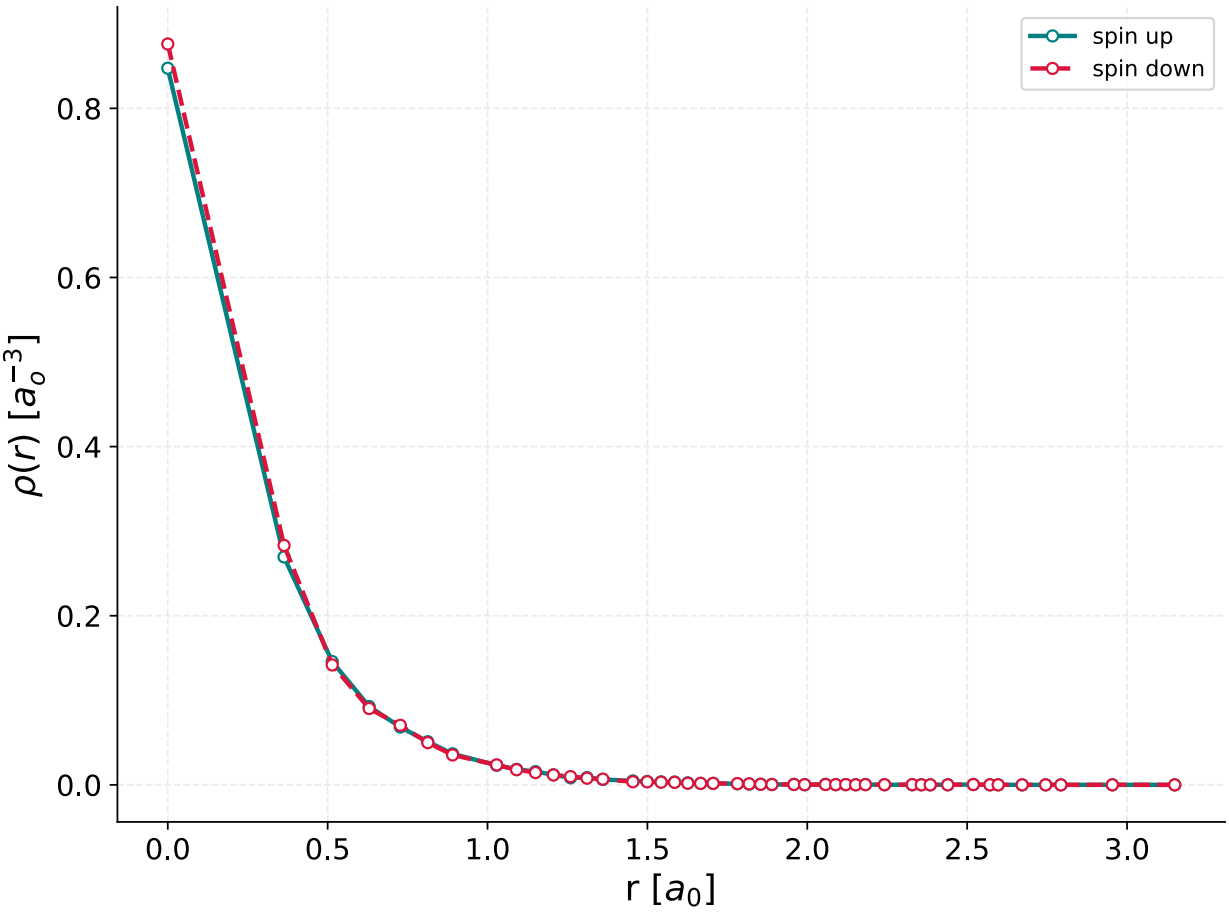



Figure 7.13: $\rho(r)$ for Helium atom with antiparallel spins

The plot shows the same behavior as Figure 7.7, and the same considerations made in that case apply. The agreement for spin up and spin down plots reflects the fact that their sampling is treated in the same way by the RBM as the Coulomb interaction is independent from spin, so that the spatial behavior during the sampling is the same, and the network seems to have correctly learned such symmetry.

### Radial distribution function for both spins

The plot below shows the $P(r)$ plot, calculated as $P(r) = 4\pi r^2 \rho(r)$, for spin up and spin down electrons separately.

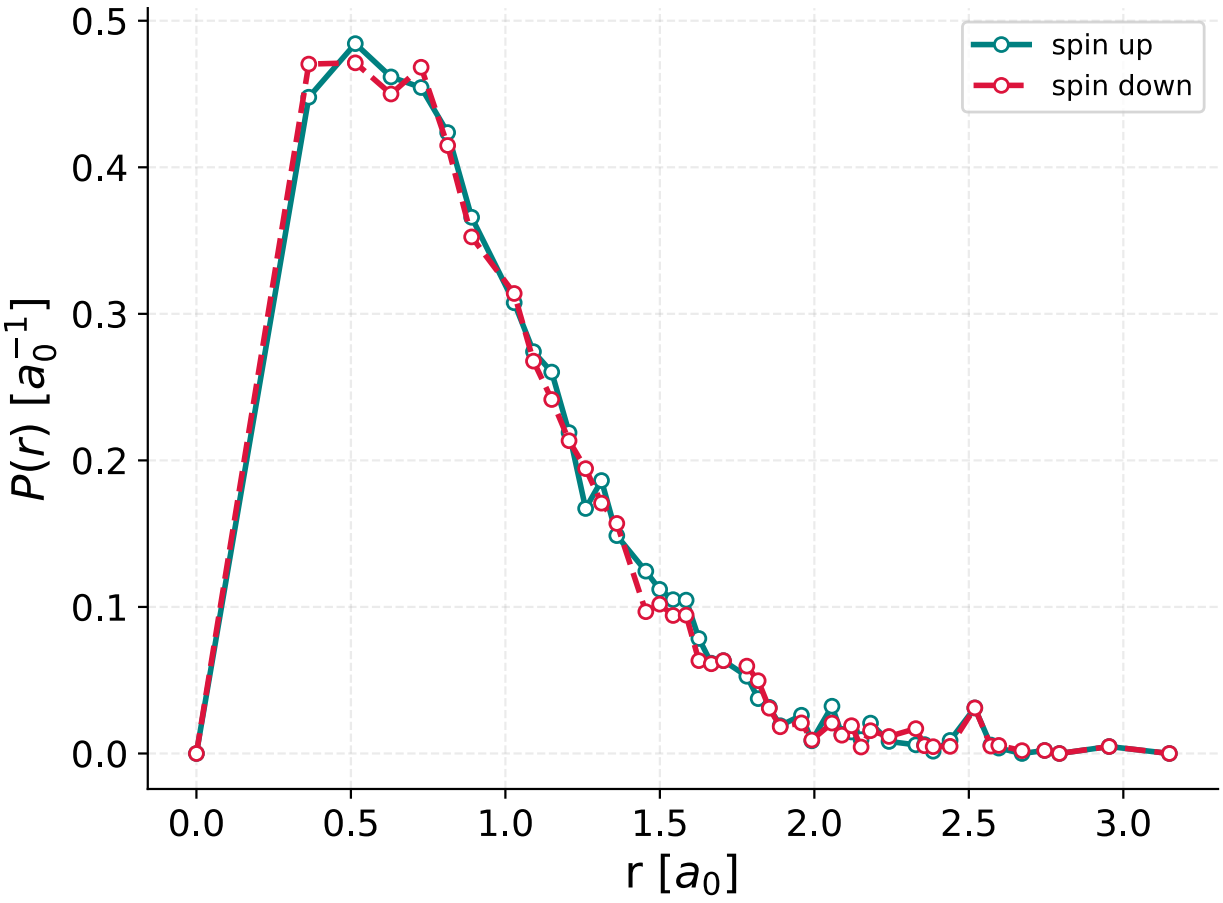



Figure 7.14: $P(r)$ for Helium atom with antiparallel spins.

Again, the profile of $P(r)$ matches that shown in Figure 7.9, and the same considerations discussed previously apply here. Due to the same symmetry arguments as in figure 7.13, both spin components exhibit identical radial behavior, except for statistical fluctuations due to Monte Carlo noise.

### Radial pair distribution function

Fig. 7.15 shows the pair distribution function $g(r)$ for the antiparallel case, obtained in the same way as for the parallel configuration.

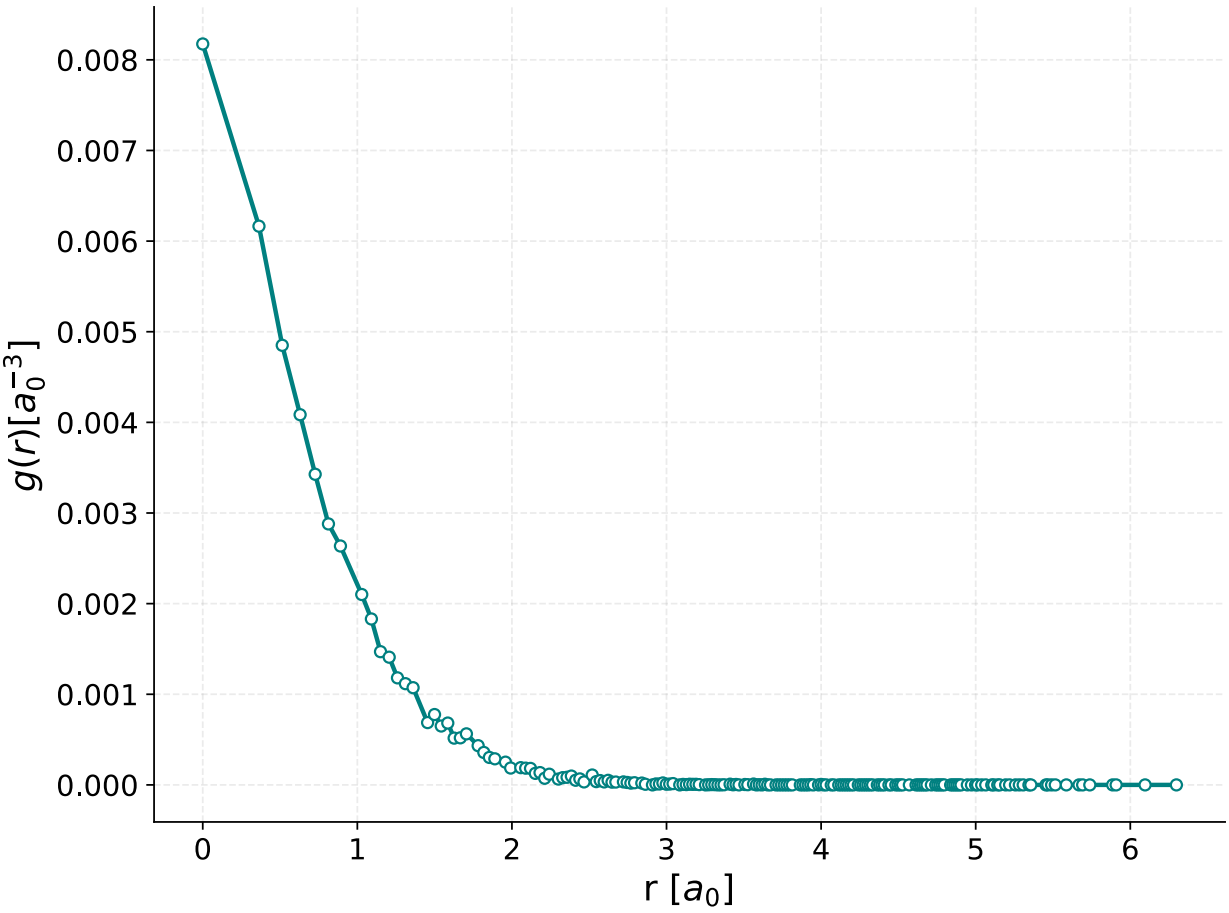



Figure 7.15: $g(r)$ for Helium atom with antiparallel spins.

This plot provides the clearest evidence that the RBM correctly captures the Pauli exclusion principle. In contrast to the parallel-spin case, the probability of finding two electrons at the same spatial location ($r \to 0$) is non-zero. This feature also results in higher probabilities at short electron-electron distances and in smoother oscillations of $g(r)$. In this case, the correlation is purely due to Coulomb repulsion, with no additional constraints from the symmetry of the wavefunction. Therefore, nothing forbids the presence of two electrons on the same physical site of the lattice because, in this case, they move in different canals on the RBM.

Figures 7.16 through 7.23 provide examples of different sections for the wavefunction $|\psi|$, compared for both spin states. The sections are obtained by fixing 4 out of total 6 spatial coordinates in different scenarios.

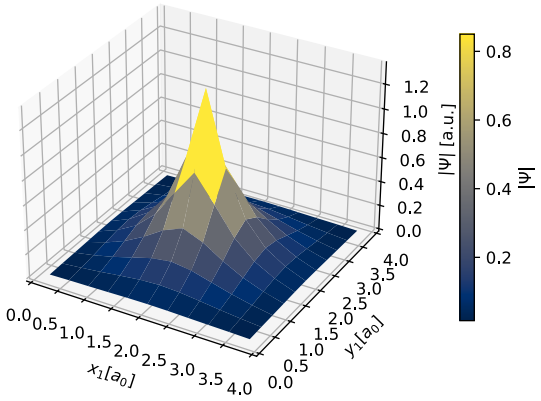



Figure 7.16: Motion of a spin up electron on the equatorial $xy$ plane with a spin down one on the nucleus. Here $x_2 = y_2 = z_2 = z_1 = 5$.

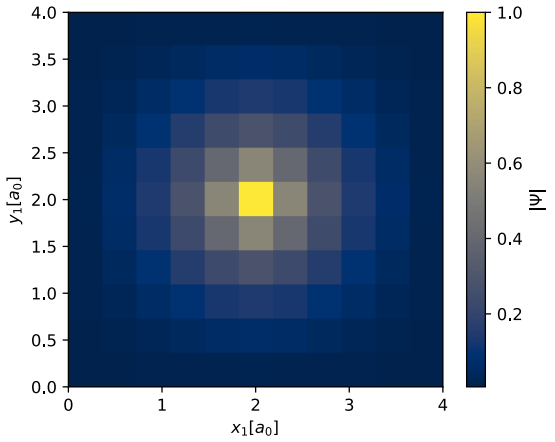



Figure 7.17: heatmap of the section on $xy$ equatorial plane.

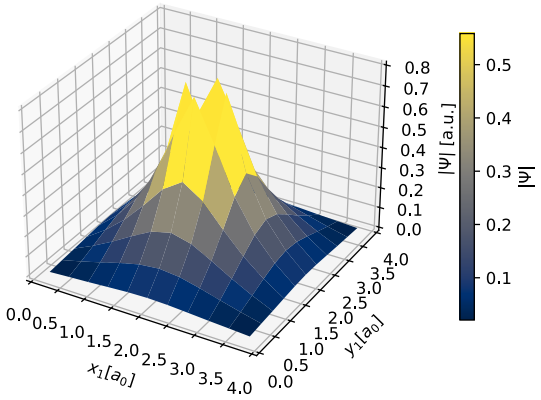



Figure 7.18: Motion of a spin up electron on the equatorial $xy$ plane with a spin up one on the nucleus. Here $x_2 = y_2 = z_2 = z_1 = 5$.

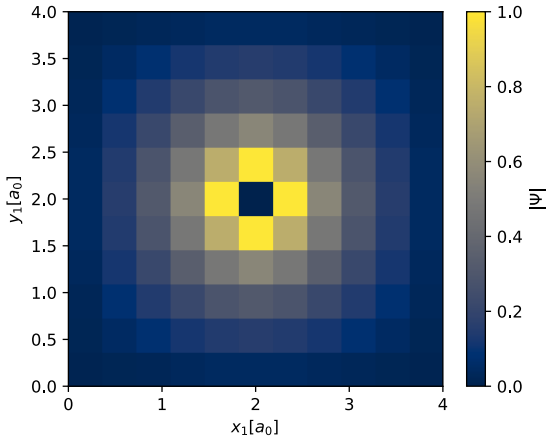



Figure 7.19: heatmap of the section on $xy$ equatorial plane.

Here the Pauli principle reflects in a "symmetry hole" in the nucleus for the 2 up case (7.18), while the antiparallel spin case shows the expected spatial symmetry of the wavefunction. As it is expected, the probability assigned to configurations which present electrons near the nucleus is maximal.
The plots below show vertical sections for both spin cases, providing another insight of the Pauli principle.

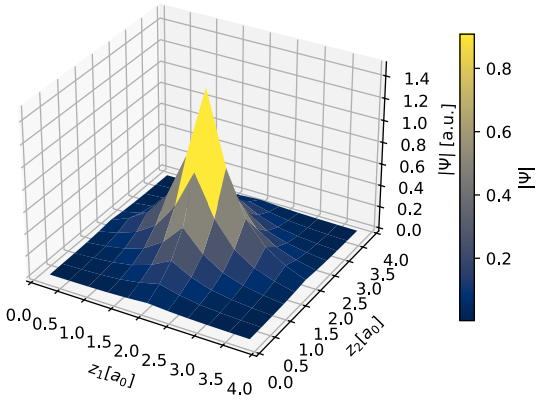



Figure 7.20: Motion of one spin up and one spin down electrons on a vertical line passing through the nucleus. Here $x_2 = y_2 = x_1 = y_1 = 5$

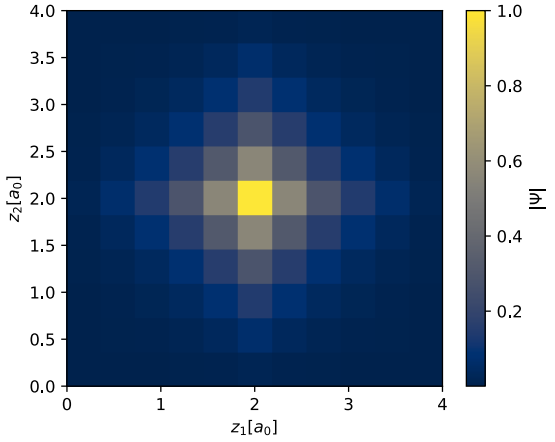



Figure 7.21: heatmap of the section on a vertical line passing through the nucleus for antiparallel spins.

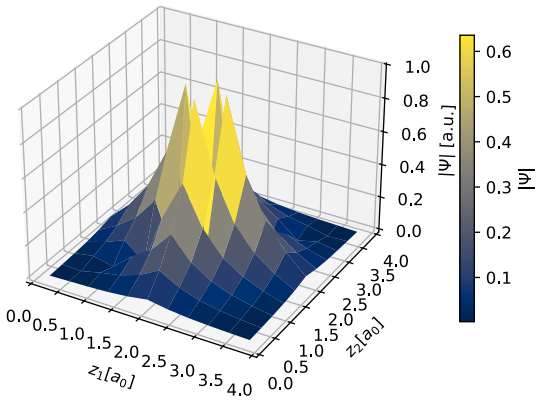



Figure 7.22: Motion of two spin up electrons on a vertical line passing through the nucleus. Here $x_2 = y_2 = x_1 = y_1 = 5$.

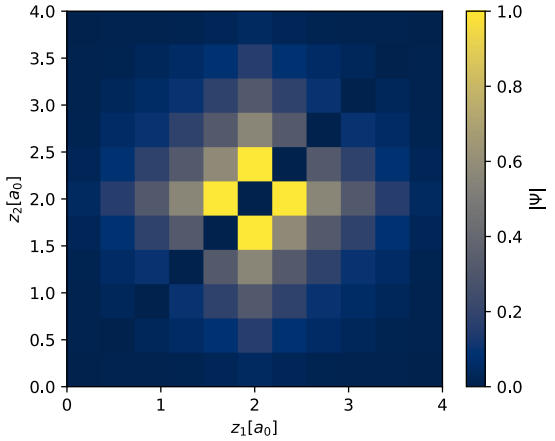



Figure 7.23: heatmap of the section on a vertical line passing through the nucleus for parallel spins.

Here the Pauli principle forbids all the configurations with $z_1 = z_2$, and this is reflected by the diagonal line in 7.23. Again, the antiparallel case shows the expected spatial symmetry, while leaving higher probability in the nucleus neighborhood.

### 7.1.4 Lithium

In this section, the focus is on the lithium atom ($Z = 3$) with two electrons with spin up and one with spin down. With its three electrons and spectrum of interaction effects, this system offers a nontrivial step beyond the two electron systems such as helium. The objective is to evaluate the expressive capacity of the RBM wavefunction ansatz in capturing both the one-

and two-electron correlation phenomena inherent in a three-electron atomic system.
Using the same parameters as before, a brief study of the hidden layer size was made in this case, and it is shown in the following plot.

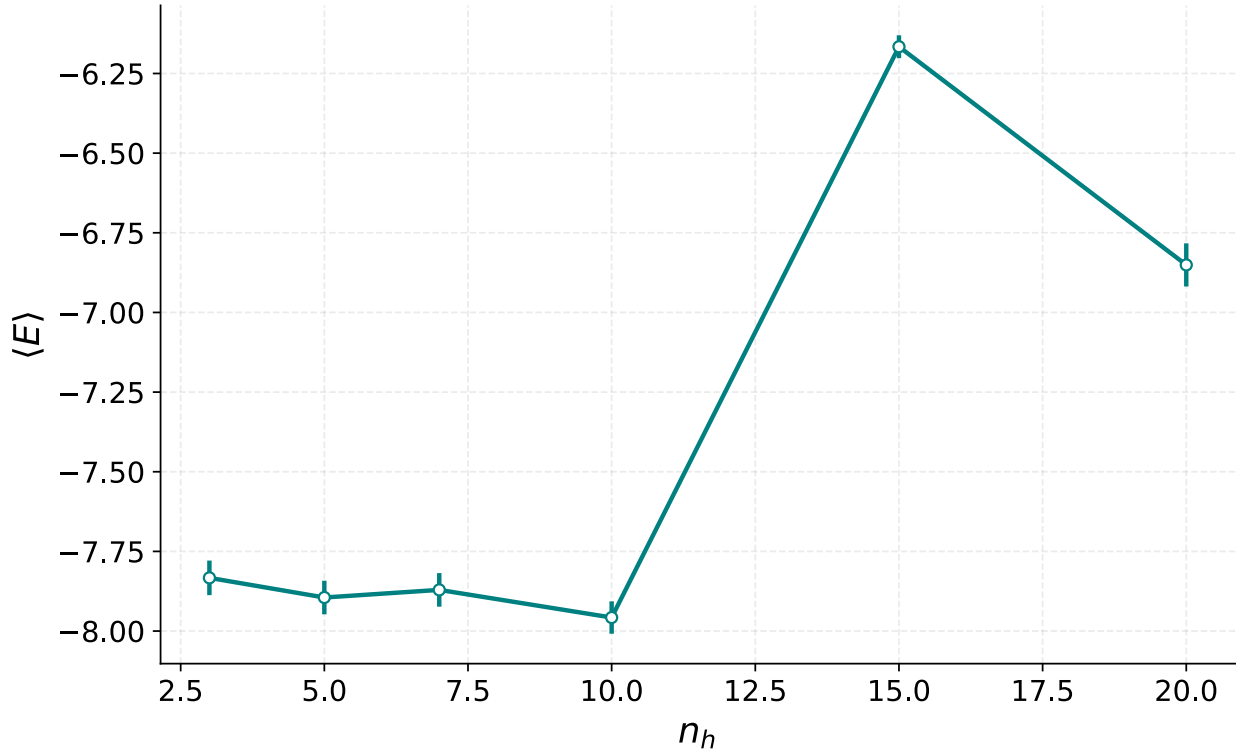



Figure 7.24: last energy values with error bars for RBM with different hidden layer size trained on lithium atom.

Here the same considerations done for helium hold. Since the three electron system introduces more degrees of freedom and more complexity due to spin simmetry and electronic correlations, here the best RBM needs 3 extra hidden units with respect to the helium case, so that the best configuration can be identified with the one with $n_h = 10$. Fig. 7.25 demonstrates the behavior of energy during the training in this scenario.

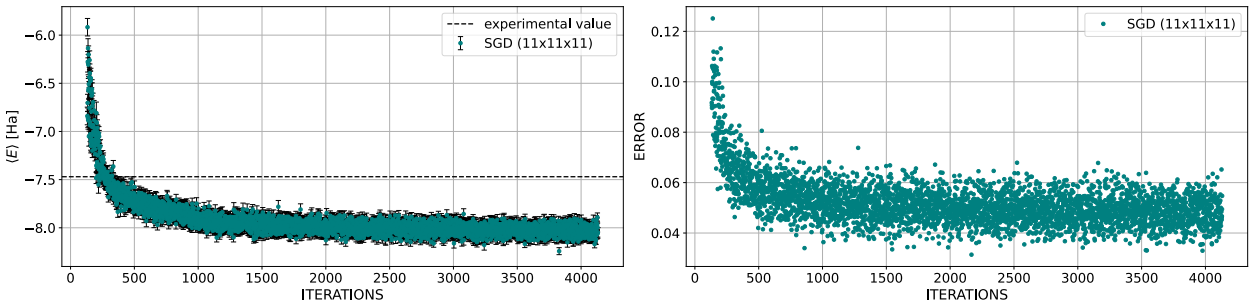



Figure 7.25: Energy and error during training (first 4000 iterations) for lithium with $n_h = 10$.

Again, the comparison with the experimental value is plotted just to give an intuitive

feel about the magnitude of the energy reached, and the experimental value is not intended to be assumed as the expected one. To obtain an energy eigenvalue comparable with the experimental value, a more in depth study of $L, n_v, \ell$ is needed. Here, the case of hydrogen made by Redaelli [1] is presented as an example:

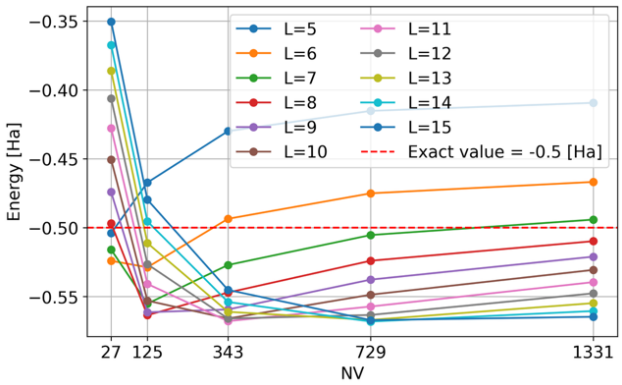



Figure 7.26: Redaelli's study [1] on different ground state energy eigenvalue, compared with the RBM spatial parameters.

The result obtained for Lithium can be roughly compared to the red line in this plot ($L = 8$), in which the final ground state energy eigenvalue is below the expected one. However, reproducing the experimental value is not the scope of this work, since the interest is to prove that RBMs can correctly learn wavefunctions by reproducing qualitatively coherent results.

Using this trained RBM, the spatial plots, and some sections of the wavefunction, have been calculated.

### Spatial distribution plots

The plots of $\rho(r), P(r), g(r)$ for the lithium atom provide a meaningful representation of the Pauli principle effects on the sampling of the electrons positions, marking a summary of the property seen for both helium cases.

The first plot shows the radial density function $\rho(r)$ calculated separately for the two spin up electrons and the spin down one:

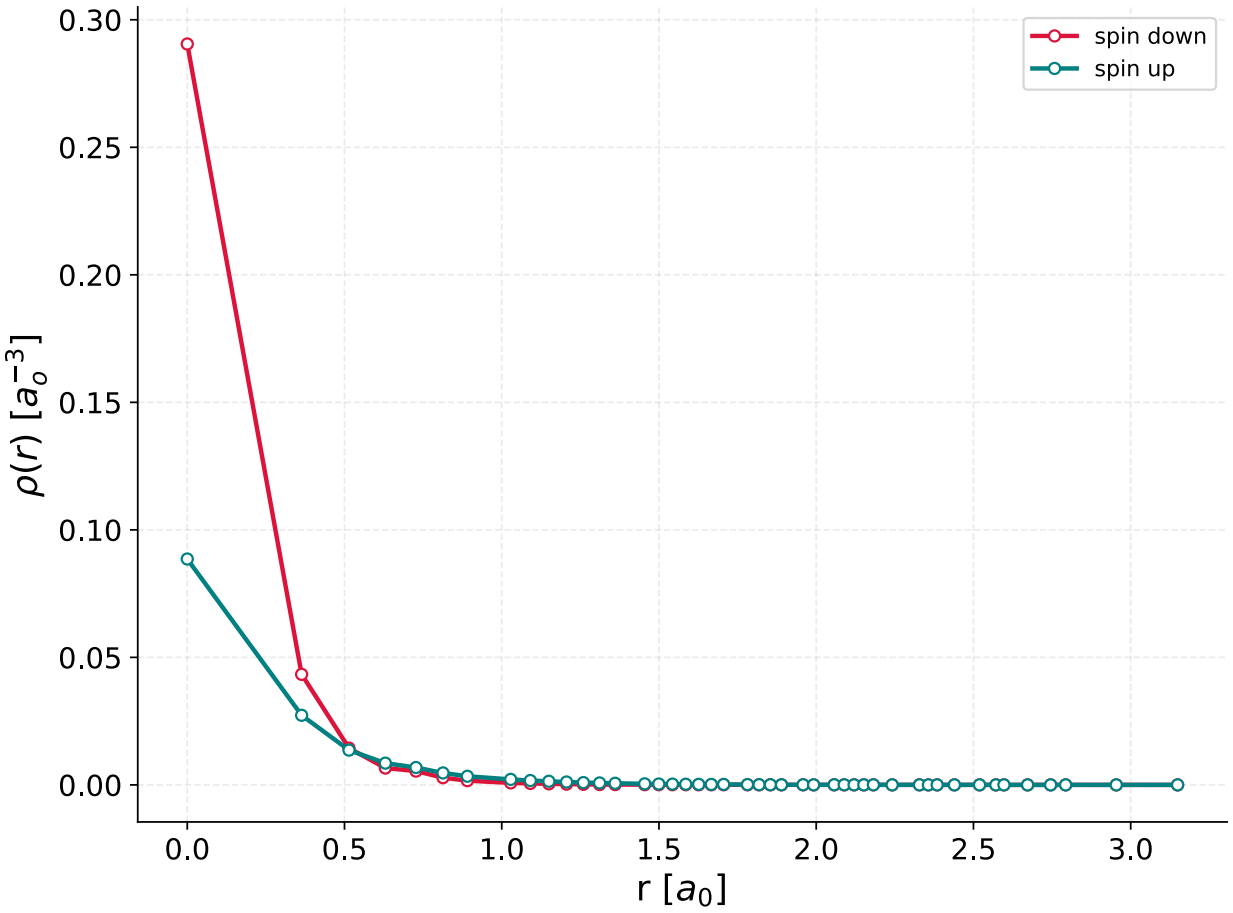



Figure 7.27: $\rho(r)$ for lithium with two spin up electrons and a spin down one.

The much higher peak at $r \to 0$ for the spin down electron is a symmetry effect. In the ground state of lithium ($1s^2\,2s^1$), the spin down electron always occupies the inner $1s$ shell and is therefore much more likely to be found near the nucleus. Conversely, the spin up density is suppressed at short distances because one spin up electron is already in the $1s$ orbital, forcing the second spin up (the $2s$ electron) to remain on average at a larger radius due to the antisymmetry of the spatial wavefunction for same spin electrons.

This symmetry considerations are confirmed by looking at the $P(r)$ plot:

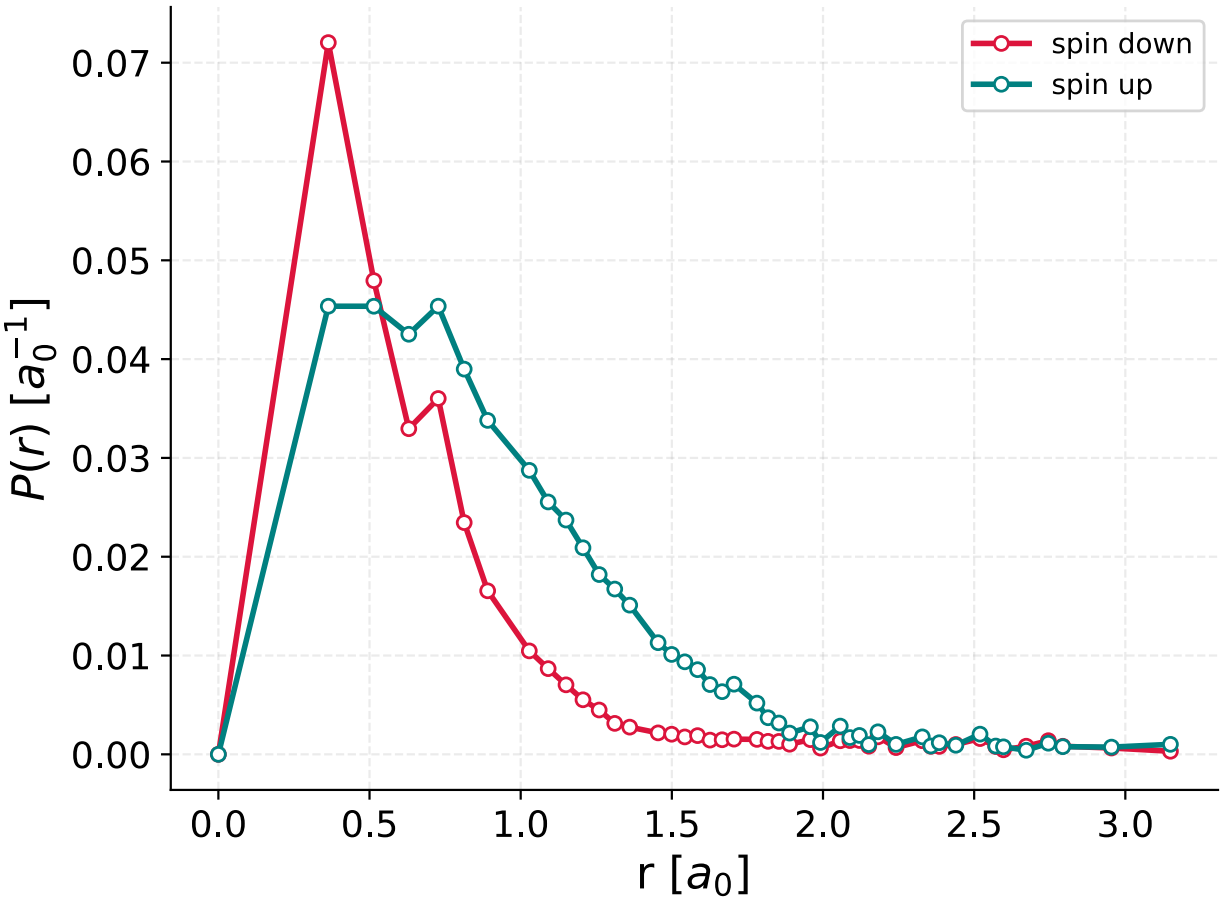



Figure 7.28: $P(r)$ for lithium with two spin up electrons and a spin down one.

Here it is evident that the spin down electron is more localized around the nucleus, as the probability of finding it in the region around $\sim 0.4\,a_0$ is maximal. In contrast, the spin up density is more spread out because it represents the average distribution of two electrons: one inner $1s$ electron, localized near the nucleus similarly to the spin down one, and one outer $2s$ electron at larger distance. This mixed character is reflected in the plateau visible in the intermediate region around $r \sim 0.4$–$0.7\,a_0$, where the contributions from the inner and outer spin up electrons overlap.

Fig. 7.29 demonstrates the pair distribution function $g(r)$ evaluated separately on up–up and down–up spin pairs. The behavior for $r \to 0$ summarizes the observations done for helium: antiparallel pairs have symmetric spatial wavefunction, that makes it highly probable to find them at short distances, meanwhile parallel pairs exhibit $g(0) = 0$ due to Pauli principle, that forbids to have two spin up electrons on the same lattice site.

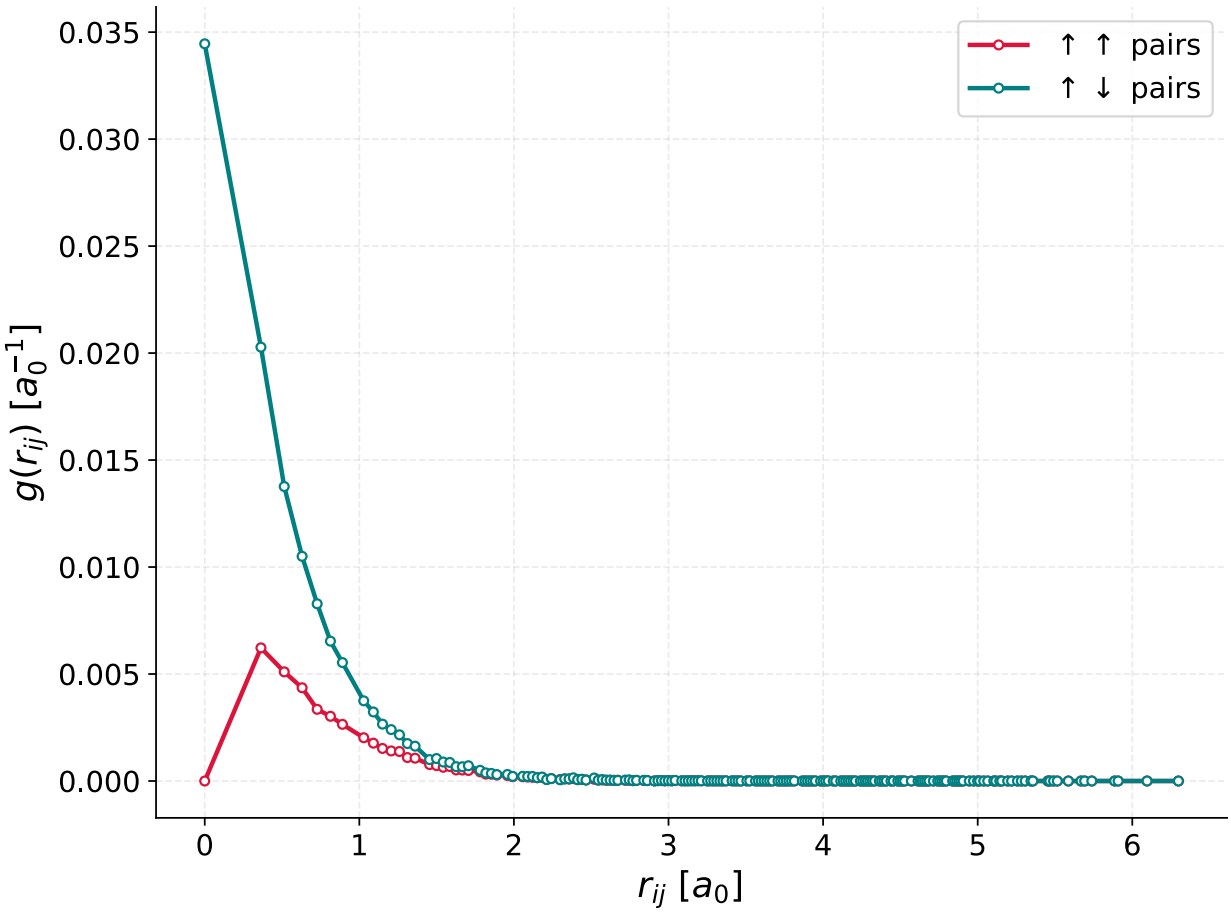



Figure 7.29: $g(r)$ for lithium with two spin up electrons and a spin down one, plotted for parallel and antiparallel pairs.

Figures **??** through **??** shows some sections of the $|\psi|$ with 7 fixed coordinates out of a total of 9.

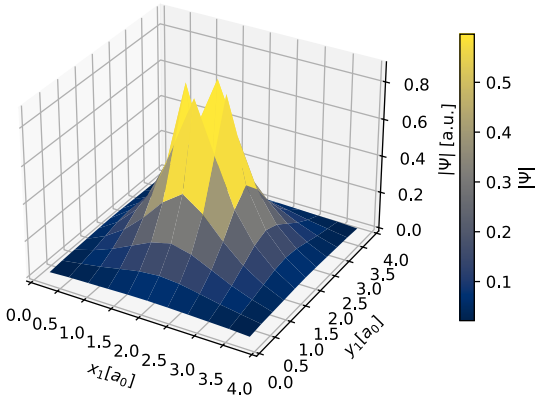



Figure 7.30: Motion of a spin up electron on the equatorial $xy$ plane. Here $x_2 = y_2 = z_2 = x_3 = y_3 = z_3 = 5$ (two electrons on the nucleus) and $z_1 = 5$.

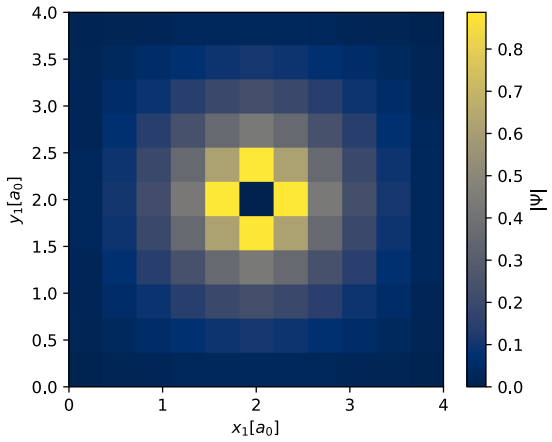



Figure 7.31: heatmap of the section on $xy$ equatorial plane.

Fig. **??** represents the motion on the $xy$ plane of the outer $2s$ electron, since the spin down and the other spin up electrons are fixed into the nucleus position. from these figures, the spherical symmetry is evident, and the Pauli principle forces a "symmetry hole" in the center of the box. As one might expect, the probability of finding the electron increases as the electron gets closer to the nucleus, and this is due to Coulomb attraction.

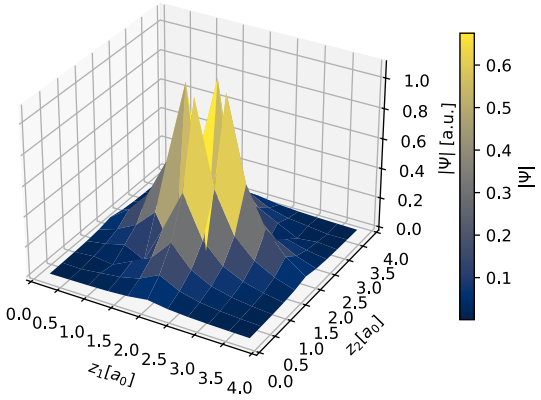



Figure 7.32: Motion of two spin up electrons on a vertical line passing through the nucleus. Here $x_1 = x_2 = x_3 = y_1 = y_2 = y_3 = z_3 = 5$.

.

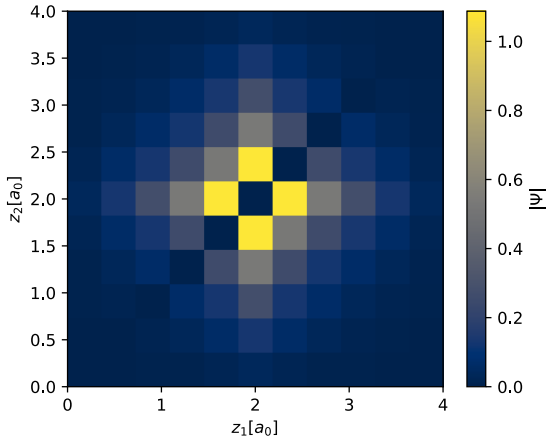



Figure 7.33: heatmap of the section on a vertical line passing through the nucleus.

In Fig. **??**, Pauli principle imposes that $z_1 = z_2 \Rightarrow |\psi| = 0$, and this is visualized on the diagonal line of the heatmap.

In fig.**??** the null vertical line reflects the Pauli repulsion between the moving spin up

electron and the fixed one. The wavefunction is also symmetric with respect to the spin up and spin down electrons' motion.

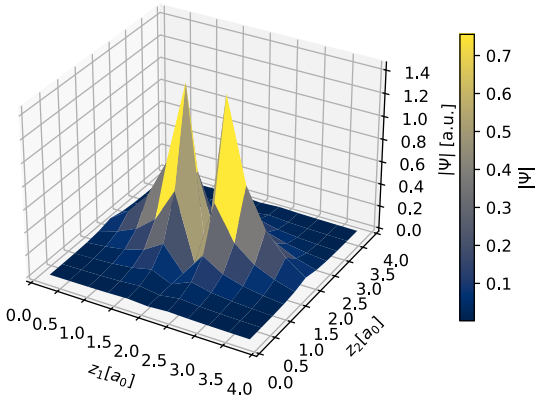



Figure 7.34: Motion of one spin up and one spin down electrons on a vertical line passing through the nucleus. Here $x_1 = x_2 = x_3 = y_1 = y_2 = y_3 = z_3 = 5$.
.

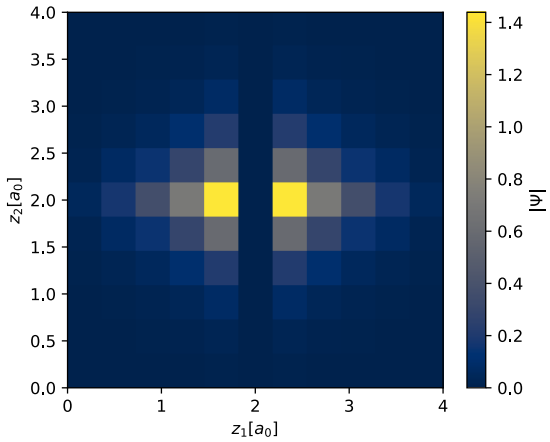



Figure 7.35: heatmap of the section on a vertical line passing through the nucleus.

## 7.2 Deuteron

We run deuteron simulations using the nuclear metrical unit system ($\hbar = c = 1$ dimensionless), where energy and momenta are expressed in $MeV$ (mega–electronvolt) or $fm$. We used the values for the neutron and proton masses mentioned in 2.4, and the lengths are expressed in $fm$. Due to the short range of the Argonne potential, which vanishes at distances beyond $2fm$ and has a high repulsive core around $0.5fm$, and due to the highest computational cost carried by the $4n_v$ lattice and the local energy evaluation, fewer visible units are implied with respect to atoms. Here we present results for a box mesh of $L = 9$ and thus $n_v = 729$, in a trap with physical length of $\ell = 3.5fm$. This value of $\ell$ is necessary to balance the discretization, which can degrade the representation of the kinetic energy if $dx \lesssim 0.3$. The annealing schedule defined is the same as the one used for atoms.

Figures **??** show the behavior of energy during a full training with $n_h = 10$.

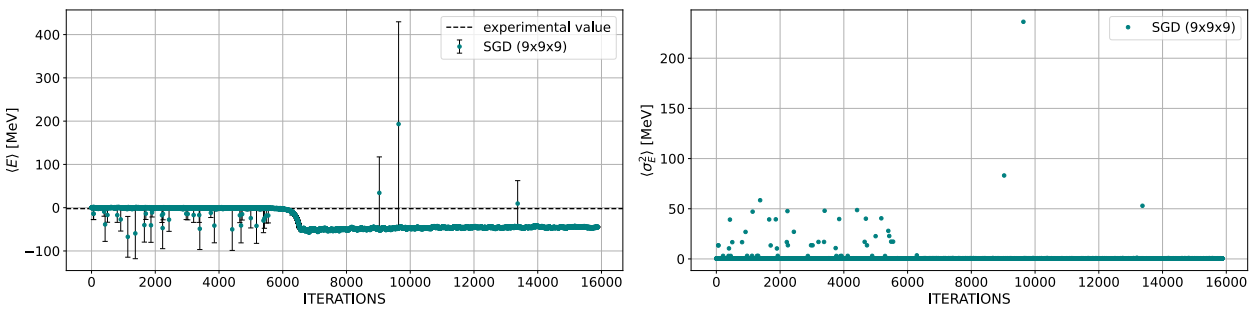



Figure 7.36: Energy during training for deuteron with ArgonneV6 potential.

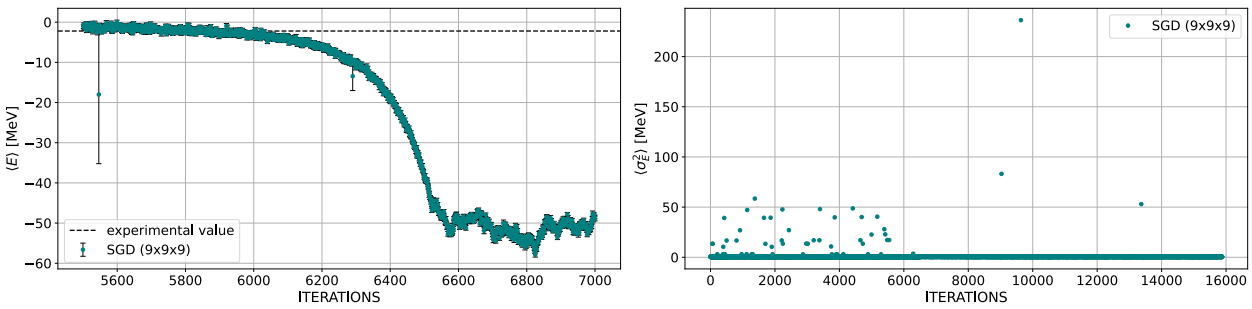
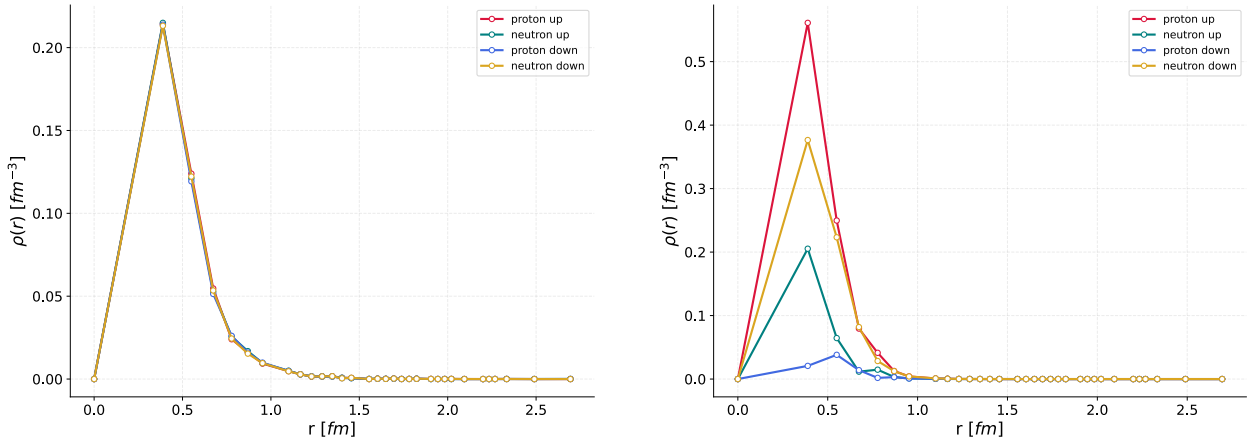



Figure 7.37: Energy during training for deuteron with ArgonneV6 potential in the energy drop region.

The plot shows that the energy remains coherent with the experimental value for the first 6000 iterations, and then it rapidly drops to $\sim -50MeV$. To investigate this behavior, the RBM weights have been saved before and after the drop, and spatial distribution plots have been calculated.




(a) $\rho(r)$ for the deuteron, separated by particle type. Weights taken at $n_{\text{iter}} = 6000$.

(b) $\rho(r)$ for the deuteron, separated by particle type. Weights taken at $n_{\text{iter}} = 12000$.

Figure 7.38: $\rho(r)$ for deuteron in different stages of training.

Figure 7.38a displays the radial density function for the deuteron before the large drop in energy and demonstrates a physically coherent behavior. Since the distances $r$ are taken with respect to the center of mass, we conclude that the most probable distance between nucleons is around $[0.9, 1]fm$, which corresponds to the attractive part of the nuclear potential. The fact that all the plots for different spins overlap is accurate, because there is no physical reasons for proton and neutrons to behave differently during their motion, due to the symmetry constraints imposed on the sampling scheme. In addition, the density decays exponentially after the peak, which confirms the expected results.

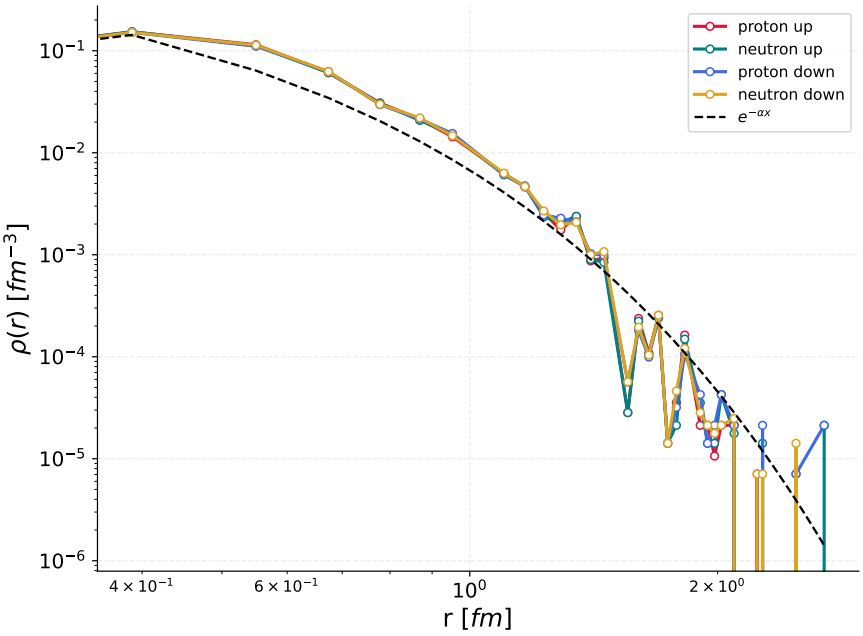
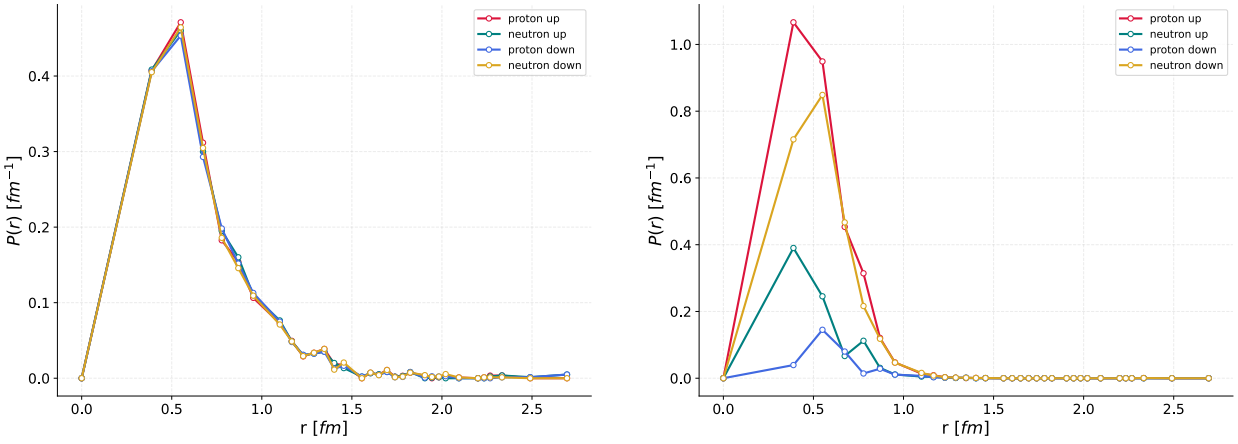



Figure 7.39: $\rho(r)$ for the deuteron, separated by particle type, with logarithmic scale on both axis and an approximate exponential fit. Weights taken at $n_{\text{iter}} = 6000$.

On the other hand, figure 7.38b shows different behaviors depending on the state of each particle, which is an unphysical result. This can be due to an overfitting phenomenon or an implementation error for the RBM, which is confirmed by the unphysical energies reached during this phase of training.

We consider the densities obtained around $n_{\text{iter}} = 6000$ to be close approximations of the deuteron radial distribution. However, instabilities seen at later epochs are still not under control and we plan future work to resolve this.

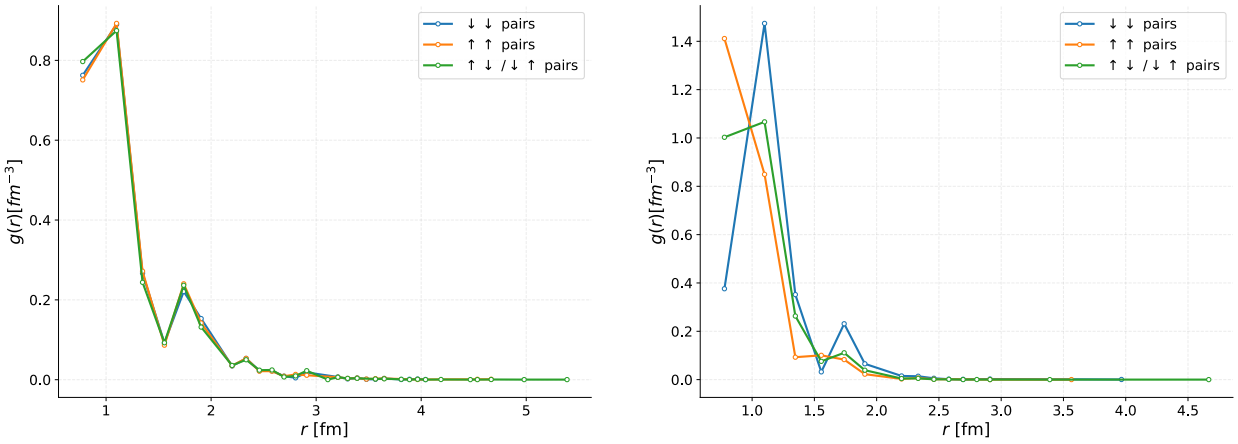



(a) $P(r)$ for the deuteron, separated by particle type. Weights taken at $n_{iter} = 6000$.

(b) $P(r)$ for the deuteron, separated by particle type. Weights taken at $n_{\text{iter}} = 12000$.

Figure 7.40: $P(r)$ for deuteron in different stages of training.

The $P(r)$ plot confirms the observations made above, as it shows that the most probable distance between nucleons is around $\sim [1, 1.2] fm$, coherently with the range of nuclear interactions range.




(a) $g(r)$ for the deuteron, separated by particle type. Weights taken at $n_{iter} = 6000$.

(b) $g(r)$ for the deuteron, separated by particle type. Weights taken at $n_{\text{iter}} = 12000$.

Figure 7.41: $g(r)$ for deuteron in different stages of training.

The radial pair distribution function in the first scenario shows, again agreement with the expected qualitative behavior. 7.41a shows that the most probable distance between particles sits around $\sim [1, 1.2] fm$, which is consistent with the expectations after the $P(r)$ plot, and so with the nuclear interactions range. The second plot shows the same problematic behavior as for the $\rho(r)$ and $P(r)$.

Overall, we reported this case because it summarizes the main issues encountered when training the RBM on the deuteron: the energy either remains higher than the expected value or drops to unphysically overbound states. An open problem is to find the best hyperparameters $n_v, \ell, L$ to discretize the simulation box so that the nuclear potential can exploit its attractive core, while preventing the kinetic energy from becoming too large, since $dx = \frac{\ell}{L}$ appears in the denominator in (6.1).

If $dx$ is too large, the energy remains high because the attractive part of the potential (acting only in the short range $[0.5, 1]$ fm) does not contribute significantly, as the minimum distance between nucleons is $2dx$ due to the imposed center of mass symmetry. Moreover, if $dx$ is too small, the energy also increases because the kinetic term dominates.

The occurrence of unphysically overbound states, when it happens, likely signals implementation errors.

Further improvements in the code are needed to fix such issue, and this could be the starting point for a future work.

# Chapter 8

# Conclusions and outlook

My contribution to this area of research has been, in the first place, to complete a previously exploratory implementation [1] of RBMs for atoms in a box. This implied improvements to the hidden layer in the network that allowed an in–depth study of the helium and lithium systems. We then investigated how the hyperparameters $\eta_0, \eta_1, p$ and $n_h$ affect training. In particular, we observed that the best cooling schedule corresponds to an initially hot regime followed by a relatively fast temperature decrease, and that an excessively large number of hidden units tends to destabilize the training. Finally, we extended the code by implementing the nuclear potential, approximated by the semi–realistic Argonne V6 model [13], in order to give an initial study of the application of RBMs in nuclear physics.

The investigation of nuclear systems is, however, far from complete. Although the RBM is able to produce qualitatively correct observables for the deuteron, issues such as overfitting and unphysical energy values still emerge at late training stages. These instabilities suggest that further methodological improvements in the sampling, in the optimization scheme or in the parametrization of the wavefunction will be required. Therefore, resolving the deuteron wavefunction with an RBM will be the subject of future work.

One of the most promising aspects of this architecture is its potential scalability. In principle, extending the method from the deuteron to an $N$-nucleon system only requires modifications in the sampling procedure (in particular, the loss of centre of mass symmetry), while the number of variational parameters remains fixed or only weakly dependent on the system size. This property makes RBMs an attractive candidate for future applications in nuclear physics, where conventional approaches such as Configuration Interaction (CI) and Diffusion Monte Carlo (DMC) become rapidly computationally demanding as the number of particles increases. RBM based NQS could therefore provide a more tractable and potentially scalable alternative for predicting nuclear observables, along with the the hypothesis of transfer learning that the presented feature might suggest.

Overall, this thesis offers a first step towards the use of RBMs in ab initio nuclear simulations. The results obtained for helium, lithium and the deuteron highlight both the promise of this approach and the technical challenges that remain open, providing several clear avenues for future work.

# Appendix A

# Code

Here follows a brief presentation of the main components of the implemented code. To give a complete overview, we also report Redaelli's part of the summary [1]. This section is not intended to be a full explanation of the code, but to be a brief summary of each part of it.

## A.1 Readoptions.cc

The procedure of setting up the simulations relies on an appropriately prepared input file to be filled from the user.

The input file contains one parameter per line, in the form `keyword = value`. The available keywords are listed in the following table.

| | |
|---|---|
| `nv` | number of visible units. |
| `nh` | number of hidden units. |
| `sigma` | variance of the Gaussian used to initialize the network parameters. |
| `loadtype` | 0: random weights (Gaussian); 1: visible weights from file and random hidden weights; 2: all weights from file; 3: all weights from file with newly initialized hidden units. |
| `restartfn` | name of the file containing the weights to be loaded. |
| `model` | Hamiltonian to be used (Coulomb3d for atoms, ArgonneV6 for nuclei). |
| `M` | number of particles. |
| `spin_up` | number of spin–up particles. |
| `spin_down` | number of spin–down particles. |
| `seed` | random generator seed (set to `-1` for random value). |
| `printstates` | if `true`, sampled configurations are printed to file. |
| `filestates` | name of the output file for configurations. |
| `nsweeps` | number of Monte Carlo sweeps per iteration. |
| `niter` | number of iterations. |
| `get_pars` | every given number of iterations, print the weights to file. |
| `Nprotons` | atomic number. |
| `traplength` | physical length of the box. |
| `isospin` | `1` for nuclei, `0` otherwise (sets the visible layer size to $4n_v$). |
| `iso_up` | number of protons. |
| `iso_down` | number of neutrons. |
| `eta` | $\eta_0$. |
| `eta1` | $\eta_1$. |
| `label` | label for the output files (weights, acceptance, learning rate);for example, for weights files the filename is `weights<label>.dat`. |
| `trainingfn` | name of the file that outputs energies and errors during training. |

**ReadParametersFromFile** method reads this input file and returns a map, in which each element can be accessed by its label in the left column of the input file.

## A.2 optH2.cc

This code is the main program file that orchestrates the execution of the simulation, it uses MPI for parallel processing and handles different model types.

The initialization of MPI is performed using MPI_Init. Subsequently, the rank of the current process is determined using MPI_Comm_rank, while the total number of processes is obtained via MPI_Comm_size.

Following the initialization, the function ReadOptionsFromFile is invoked to parse the command-line arguments in a std::map.

Then the code defines a neural network wavefunction object wavef of class Nqs, initialized with the parameters provided in the options.

It sets a different seed for each process using the provided seed and the process rank. If the process is rank 0, it prints information about the run, including the number of nodes,

the model, and the number of sweeps per process.

It uses a try-catch block to handle potential exceptions during the simulation. A series of if-else if statements checks the model type and executes the corresponding simulation, following for each one this steps:

- Creates a Hamiltonian.
- Defines a Sampler with the wavefunction and Hamiltonian.
- If printstates is true and the rank is 0, it sets the file states in the sampler.
- Sets learning parameters eta and eta1.
- Defines an Sgd optimizer.
- Defines a Variational object with the sampler and optimizer.
- Runs the training using var.RunTraining

If the model is unsupported the catch block handles exceptions by printing an error message with the rank and the exception details, then aborts MPI. At the end it uses MPI_Barrier to synchronize all processes after the calculation and if the rank is 0, it prints a success message. The it finalizes MPI using MPI_Finalize.

The main implied is an extentsion of Redaelli's one, that includes also the ArgonneV6 model section, in which it defines the sampler as a SamplerH2 object that implements the modified Metropolis scheme for the deuteron.

## A.3 nqs.cc

This code defines a class Nqs that implements a Neural Quantum State (NQS) wavefunction, specifically using a Restricted Boltzmann Machine (RBM) architecture. It's designed to:

- *Represent a quantum wavefunction*: it uses an RBM to approximate the wavefunction of a quantum system.
- *Calculate wavefunction properties*: it provides methods to compute the wavefunction's value, ratios of wavefunction values for different configurations, and derivatives of the wavefunction.
- *Manage the RBM's parameters*: it includes functions to load, set, and get the parameters of the RBM (weights and biases).

These procedures are implemented with the methods that follow.

- **LogVal Method:** It calculates the logarithm of the wavefunction for a given state, which is a vector of integers representing the spin configuration of the system. It also computes the RBM energy for the given state and returns it as a complex number.

- **LogPoP Method:** It computes the logarithm of the ratio of probabilities $\Psi(\text{state}')/\Psi(\text{state})$, where state' is a state with some spins flipped compared to state. It uses look-up tables (Lt_) to efficiently calculate the change in the log probability due to the spin flips.
- **PoP Method:** It calculates the ratio of probabilities $\Psi(\text{state}')/\Psi(\text{state})$ and it exponentiates the result of LogPoP.
- **InitLt Method:** It initializes the look-up table Lt_ and it calculates and stores intermediate values that are used to speed up the computation of LogPoP.
- **UpdateLt Method:** It updates the look-up table Lt_ after spin flips.
- **DerLog Method:** It calculates the derivatives of the logarithm of the wavefunction with respect to the RBM parameters. These derivatives are crucial for variational Monte Carlo methods, where they are used to optimize the wavefunction.
- **Get/Set methods**: this series of methods can be used to return (get) or to modify (set) all the class data members.
- **Nqs (constructor) method**: this new constructor takes he RBM parameters as inputs from the main: nv and nh set the size of both layers, the boolean "isospin" variable doubles the size of the visible layer and calculates the total number of parameters consequently if true, the variable loadtype is used to choose the selected weights loading scheme and the "filename" string variable serves as the name of the input weights file (by default it is set to "none", so that for loadtype=0 it is ignored).
- **LoadParametersFromFile methods**: these methods implement four different weights and learning rate initializations for the RBM. When the input weights file is needed, it must be prepared with the number of visible units in the first row of the file, the number of hidden units in the second one, the learning rate on the third one and the number of iteration (in case of restart) in the fourth one, and then all the network's weights, one for each line. Such format is already produced from Variational.cc during training. The loading modalities implemented are:

  0. All random (Gaussian) weights.
  1. Visible units weights loaded from file, random (Gaussian) hidden units weights.
  2. All weights loaded from file.
  3. All weights loaded from file, with new hidden nodes, initialized randomly (Gaussian). In this case the input file must contain the number of hidden units to be added right after the original $n_h$.

## A.4 Sampler.cc

This code defines a class called `Sampler` that implements a Monte Carlo sampling algorithm specifically designed for studying quantum many–body systems (except for deuteron). The central class, named `Sampler`, utilizes the Metropolis-Hastings algorithm to generate

a Markov chain of spin configurations. These configurations represent different possible arrangements of spins in a lattice model. The `Sampler` class takes a wavefunction object and a Hamiltonian object as templates, allowing for flexibility in the choice of the model being simulated. The wavefunction object encapsulates the quantum state of the system, while the Hamiltonian object encodes the interactions between the spins.

- **Move Method:** The `Move()` method is responsible for proposing and accepting spin flips. Within `Move()`, the `RandSpin()` function generates a proposed spin flip, adhering to any constraints imposed by the model, such as connectivity or spin conservation. The acceptance probability of this proposed flip is then calculated. This calculation involves evaluating the wavefunction (`wf_.PoP()`) for both the current and proposed configurations, with the acceptance probability determined by the ratio of the squared magnitudes of these values. This acceptance probability is then compared against a random number drawn from a uniform distribution. If the probability exceeds the random number, the proposed spin flip is accepted, and the configuration is updated accordingly, otherwise it is rejected. This iterative process of proposing and accepting spin flips generates a sequence of configurations forming a Markov chain. The `Move()` method also handles the necessary updates to the wavefunction's internal lookup tables (`wf_.UpdateLt()`) upon accepting a flip, ensuring consistency in the representation of the quantum state.

  **MeasureEnergy Method:** The accepted configurations are then employed to calculate observables, such as the energy of the system, using the `MeasureEnergy()` method. This method leverages the Hamiltonian object to compute the energy for a given spin configuration. By accumulating these measurements over a large number of accepted configurations, one can estimate the expectation values of these observables.

- **Sample Method:** In addition to the `Move()` method, the `Sampler` class also provides a `Sample()` method, which offers a streamlined approach to performing Monte Carlo simulations. This method encapsulates the core logic of the simulation, including initialization, thermalization, and the sampling loop. Within `Sample()`, the `InitRandomState()` function generates an initial random spin configuration, and the `wf_.InitLt()` function initializes the wavefunction's lookup tables. Then, a thermalization phase is executed, where a specified number of spin flips are performed and discarded to ensure that the system reaches equilibrium. After thermalization, the main sampling loop begins, where spin flips are proposed and accepted using the `Move()` method, and measurements are performed using the `MeasureEnergy()` and `CalcDerlogs()` methods.

## A.5 SamplerDeut.cc

This code define a class "SamplerH2" which implements the modified sampling for the deuteron. The flattened lattice state, the SpinIndexAll vector containing the indexes of occupied sites are stored as data members of this class, and the sampling happens substantially by acting on them. The methods that calculate energy and outputs are the same ones used by the original sampler.cc.

- **InitRandomState method**: initializes the lattice by filling state with $-1$s, and then generates two occupied positions as mentioned in 6.3. The SpinIndexAll vector is filled with the generated lattice positions.

- **OppositeDirection method**: given a direction in the lattice (an integer $r \in [0, \ldots, 5]$ for 3D box), returns the opposite one.

- **RandSpinH2 method**: this method proposes the metropolis move at each configuration. It picks one nucleon by randomly choosing a position in SpinIndexAll, checks which nearest neighbours of the ste are occupied, and stores the proposed move in the vector flips. After that, it calculates the proposed move on the other particle by applying the OppositeDirection method, saving the proposal in the mirrorflips vector.

- **FlipSpin method**: implements the spin flip by choosing a random particle, checking if it is spin up or down based on its position on the state vector, and than translates it by $\pm n_v$. After that, the SpinIndexAll vector is updated consequently.

- **FlipPN method**: this method exchanges proton and neutron by translating both of $\pm n_v$ in the state vector and updates the SpinIndexAll consequently.

- **MoveH2 method**: this method implements the Metropolis algorithm as mentioned in 6.3, using the methods mentioned above. If the move is accepted, it updates the state and SpinIndexAll vectors consequently and increments the accepted moves counter.

- **Plot method**: this method can be used to obtain spatial distribution plots. It produces a single run of samples without training and outputs all the sampled positions to an external file.

## A.6 Coulomb3d.cc

This code defines a class `Coulomb3d` that simulates a system of interacting particles in a 3-dimensional space, specifically addressing a Coulomb interaction within a trap.

- **Coulomb3d (Constructor) Method:** It initializes a `Coulomb3d` object. It takes parameters such as number of spins (`nspins`), number of particles (`Mag`), trap length (`traplength`) and so on. It calculates the lattice dimension `l_` based on `nspins` and it initializes `KEconst_` and `nucleus_`. It calls `InitLattice()` and `Init()`.

- **InitLattice Method:** It initializes the lattice and it checks if the number of spins is compatible with a cubic lattice, it aborts if not compatible.

- **Get/Set Methods:** to give access to data members.

- **Init Method:** It initializes pre-computed quantities, it resizes `mel_` and `flipsh_` and it assigns values to them.

- **FindConn Method:** It finds non-zero matrix elements of the Hamiltonian. It takes the state index, flipsh, mel, and Nup as input, it resizes mel and flipsh, it assigns precomputed matrix elements and it calculates Coulomb potential terms. It also finds neighbors and implements boundary conditions.
- **MinFlips() Method:** It returns the minimum number of flips per Monte Carlo step, which is two.
- **CalculateCoo Method:** It calculates the coordinates of a particle taking a particle index as input. It returns a vector of coordinates.
- **FindNeighbours Method:** It finds the neighbors of a site `s` taking a site index `s` as input. It returns a vector of neighbor indices.
- **PbcH, PbcI and PbcV Methods:** These are functions for finding neighbors with periodic boundary conditions (PBC).
- **H, I and V Methods:** These are functions for finding neighbors without periodic boundary conditions.
- **Distance Method:** It calculates the distance between two sites `s1` and `s2`, it uses `CalculateCoo` to get the coordinates and it returns the distance.

## A.7 ArgonneV6.cc

This code implements the nuclear Hamiltonian in the same template class used for atoms in the Coulomb3d case. The most important data members to be mentioned here are the mel vector and the flipsh one, which will contain the matrix element of the potential and the corresponding ratios $\frac{\psi'}{\psi}$, respectively.

- **FindConn method**: this method evaluates the local energy contribution of the kinetic energy and puts it into the diagonal, corresponding to mel[0]. Then, it evaluates the ArgonneV6 contribution, and puts off diagonal terms in mel[i], along with the corresponding transitions on the lattice, stored in the vector flipsh[i].
- **FindNeighborsGlobal method**: extends the previous one to the $4n_v$ lattice, by calculating physical distances translating the indexes in the first canal.
- **MatrixIndex method**: implements the coupled spin–spin base mentioned in 6.7 by mapping two spin indexes into a number in $[0, \ldots, 3]$.
- **SpinCouplingInit method**: initializes the spin coupling matrix (6.9) by performing the same calculations mentioned in 6.3.
- **UnitVector method**: uses the already present coordinates method to build the normalized unit vector going from the first input site to the second.
- **ComputeS12 method**: this method computes the $4 \times 4$ matrix of the tensor operator at a given $\hat{r}$. it does so performing the calculations mentioned in 6.3.

- **AV6Operators method**: this method computes the radial operators of the ArgonneV6 potential as properly weighted combination of the basic terms mentioned in 2.8. This implementation is a translation of the original Argonne group Fortran code.

## A.8 Variational.cc

This code defines a template class named `Variational` designed to perform Variational Monte Carlo simulations. The `Variational` class orchestrates the interplay between different components:

- *Hamiltonian*: represents the system's Hamiltonian.
- *Nqs*: represents the variational wavefunction.
- *Sampler*: implements the Monte Carlo sampling.
- *Optimizer*: handles the optimization of the variational parameters.

The `Variational` class aims to optimize the parameters of a quantum wavefunction to approximate the ground state of a given Hamiltonian. It uses Monte Carlo sampling to estimate expectation values and gradient-based optimization to update the wavefunction's parameters.

- **Sample Method:** The `Sample()` method begins by retrieving the MPI rank and size, which are essential for parallel processing. It then initiates the Monte Carlo sampling process by calling the `Sample()` method of the `Sampler` object. Following the sampling, the method obtains the sampled spin configurations and their corresponding indices directly from the `Sampler`. To prepare for energy calculations, the `elocs_` vector is resized. The `elocs_` vector is subsequently populated with the real part of the sampling energies, extracting this information from the `Sampler`. The mean of these local energies (`elocmean_`) is then computed. Finally, the method retrieves the logarithmic derivatives (`derLogs_`) from the `Sampler`.
- **CalcSigmaElocmean Method:** It calculates the standard deviation of the mean local energy (`sigmaElocmean_`) using block averaging to estimate the statistical error. It divides the data into blocks and computes the variance of the block averages.
- **TrainingGradient Method:** It calculates the gradient of the energy with respect to the variational parameters, used during the training phase. Then it uses the difference between the local energies and the mean local energy to compute the gradient.
- **RunTraining Method:** The `RunTraining` method is the heart of the Variational Monte Carlo simulation. First, it sets things up for parallel computing using MPI. If loadtype$\neq$ 0, it sets the learning rate and iteration number to correctly restart from the given weights file. Then, it gets ready to save the results (training energies and errors) and statistics (acceptance, learning rate)to a file. The method then enters a loop, and this loop is where the optimization unfolds step by step. Inside the loop, it does the following:

1. **Sampling**: it calls the `Sample()` method.
2. **Energy Evaluation**: it calculates the energy of the system based on the sampling. Importantly, it makes sure all processors agree on the average energy by communicating and averaging across them.
3. **Gradient Calculation**: it calls the `TrainingGradient()` method and then, again, it combines information from all processors to get the best direction to adjust the parameters.
4. **Parameter Update**: it adjusts the parameters of the wavefunction based on the calculated mean gradient, improving the wavefunction's approximation of the true ground state.
5. **Printing values to file**: prints energies and errors during training on output file. Every given number of iterations, sets up a reset file containing RBM's weights.

The method repeats these steps many times, refining the wavefunction in each iteration. It also includes checks to make sure the parallel computation is running smoothly.

- **UpdateParameters a Method:** Each method updates the variational parameters using the optimizer.
- **SetOutputFileTraining Method:** It opens an output file for writing training data.
- **PrintStats Method:** It prints statistics about the simulation to the output file and to standard output. It includes the iteration number, mean local energy, standard deviation of the mean local energy, and the values of the variational parameters.
- **CheckParametersConsistency Method:** It uses MPI communication to check if the parameters of the variational wavefunction are consistent across all MPI processes, every fifty iterations.
- **SetUpReset method**: collects all RBM weights and outputs a file containing $n_v, n_h, \eta, n_{iter}$ in the first 4 rows and then all the weights.

## A.9 SGD.cc

This code defines a class `Sgd` that implements a stochastic gradient descent (SGD) optimizer, a fundamental algorithm used for training machine learning models.

- **Sgd (Constructor) Method:** It initializes an `Sgd` object. It takes parameters such as learning rate (`eta`), `eta1`, learning rate decay flag (`etaDecay`), L2 regularization (`l2reg`), momentum (`momentum`), and dropout probability (`dropout_p`). It initializes `npar_` to $-1$, the `decay_factor_` to 1 and `niter_` to 0.
- **SetNpar Method:** It sets the number of parameters (`npar_`).

- **Update Method:** It updates the parameters using the SGD algorithm. It first asserts that the number of parameters (`npar_`) is greater than 0. If learning rate decay is enabled (`etaDecay` is true), it updates the learning rate `eta_` and it also applies dropout regularization. It updates the parameters using the following relation:

$$\theta_{t+1} = (1 - \mu)\theta_t - \eta\big(\nabla f(\theta_t) + \lambda\theta_t\big), \tag{C.1}$$

  where:

  - $\theta_t$ is the vector of parameters at iteration $t$.
  - $\mu$ is the momentum coefficient.
  - $\eta$ is the learning rate.
  - $\nabla f(\theta_t)$ is the gradient of the objective function at $\theta_t$.
  - $\lambda$ is the L2 regularization coefficient.

  Then it increments the iteration counter `niter_`.

- **Get/Set methods**: used for restart with $\eta_0$ from file.

# Bibliography


[1] Eleonora Redaelli. Neural-network quantum states in a three dimensional box. *Università degli studi di Milano*, Bachelor thesis, 2023/2024.

[2] Geoffrey E. Hinton David H. Ackley and Terrence J. Sejnowski. A learning algorithm for boltzmann machines. *Cognitive science*, 9(1):147–169, 1985.

[3] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017.

[4] Kenny Choo, Antonio Mezzacapo, and Giuseppe Carleo. Fermionic neural-network states for ab-initio electronic structure. *Nature Communications*, 11(1), May 2020.

[5] Bryce Fore, Jane M. Kim, Giuseppe Carleo, Morten Hjorth-Jensen, and Alessandro Lovato. Dilute neutron star matter from neural-network quantum states, 2022.

[6] Nobuyuki Yoshioka, Wataru Mizukami, and Franco Nori. Solving quasiparticle band spectra of real solids using neural-network quantum states. *Communications Physics*, 4(1), May 2021.

[7] Corey Adams et al. Variational monte carlo calculations of a 4 nuclei with an artificial neural-network correlator ansatz. *Physical Review Letters*, 127.2, 2021.

[8] Jan Hermann, Zeno Schätzle, and Frank Noé. Deep-neural-network solution of the electronic schrödinger equation. *Nature Chemistry*, 12(10):891–897, September 2020.

[9] David Pfau, James S. Spencer, Alexander G. D. G. Matthews, and W. M. C. Foulkes. Ab initio solution of the many-electron schrödinger equation with deep neural networks. *Phys. Rev. Res.*, 2:033429, Sep 2020.

[10] Spencer J. Choo K. et al. Hermann, J. Ab initio quantum chemistry with neural-network wavefunctions. *Nat Rev Chem*, 7:692–709, 2023.

[11] Lorenzo Lazzarino. Fermionic neural-network quantum states in a three dimensional box. *Università degli studi di Milano*, Bachelor thesis, 2021/2022.

[12] Giovanni Maria Paravizzini. Neural-network quantum states in a three dimensional box. *Università degli studi di Milano*, Bachelor thesis, 2021/2022.

[13] R. B. Wiringa, V. G. J. Stoks, and R. Schiavilla. Accurate nucleon-nucleon potential with charge-independence breaking. *Phys. Rev. C*, 51:38–51, Jan 1995.

[14] Hal Tasaki. Introduction to the "second quantization" formalism for non-relativistic quantum mechanics: A possible substitution for sections 6.7 and 6.8 of feynman's "statistical mechanics", 2023.

[15] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 06 1953.

[16] Giuseppe Carleo. Neural-network quantum states lecture notes beijing, 2017.

[17] F. Rosenblatt. The perceptron: a probabilistic model for information stor- age and organization in the brain. *Psychological review*, 65(6):386, 1958.

[18] V. Arnold. On the representation of functions of several variables as a superposition of functions of a smaller number of variables, 2009.

[19] Nicolas Le Roux and Yoshua Bengio. Deep belief networks are compact universal approximators. *Neural computation*, 22(8):2192–2207, 2010.

[20] Guido F Montufar, Johannes Rauh, and Nihat Ay. Expressive power and approximation errors of restricted boltzmann machines. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc., 2011.

[21] J.W.T. Keeble and A. Rios. Machine learning the deuteron. *Physics Letters B*, 809:135743, October 2020.

[22] Bogdan Mihaila. Matrix elements of the argonne v18 potential, 2011.

[23] J.J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.

[24] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009.

[25] Dirk P. Kroese and Reuven Y. Rubinstein. Monte carlo methods. *Wiley Interdisciplinary Reviews: Computational Statistics*, 4(1):48–58, 2012.

[26] Stefano Forte, Luca Rottoli, et al. *Fisica quantistica*. Zanichelli, 2018.

[27] Giuseppe Carleo et al. Neural-network quantum states. *Bulletin of the American Physical Society*, 63, 2018.

[28] Giuseppe Carleo et al. Machine learning and the physical sciences. *Reviews of Modern Physics*, 91(4):045002, 2019.

[29] David H. Ackley, Geoffrey E. Hinton, and Terrence J. Sejnowski. A learning algorithm for boltzmann machines. *Cognitive Science*, 9(1):147–169, 1985.

[30] Kenny Choo, Antonio Mezzacapo, and Giuseppe Carleo. Fermionic neural-network states for *ab-initio* electronic structure. *Nature Communications*, 11(1):2368, 2020.

[31] Davide Emilio Galli. Laboratorio di simulazione numerica. Lecture notes, Università degli Studi di Milano, 2024.

[32] The Asimov Institute. The neural network zoo. `https://www.asimovinstitute.org/neural-network-zoo/`. Accessed: 2025-11-26.